



中国研究生创新实践系列大赛
“华为杯”第二十届中国研究生
数学建模竞赛

学 校

云南大学

参赛队号

23106730076

队员姓名

1.杨双萍

2.郑润泽

3.黄照煜

中国研究生创新实践系列大赛

“华为杯”第二十届中国研究生

数学建模竞赛

题 目： 出血性脑卒中临床智能诊疗建模

摘 要：

出血性脑卒中是一种常见但危险的脑血管疾病，其治疗和管理需要及时而准确的诊断以及有效的预测模型。本研究旨在通过分析入院患者的临床数据和影像学特征，建立数学模型，评估患者的血肿扩张风险、血肿周围水肿进展以及长期预后。

针对问题一，是关于血肿扩张的计算以及概率预测问题。第一个小问要求计算患者是否发生血肿扩张以及扩张时间。根据所给数据，首先计算出患者的发病时间，再在发病时间 48 小时后的时间里根据血肿体积的**相对体积变化**或者**绝对体积变化**，确定患者是否在发病 48 小时内发生了血肿扩张。用血肿扩张最后的影像时间减去发病时间得到血肿扩张的时间。第二个小问要求根据所给的数据预测所有的 160 名患者血肿扩张的概率。由于样本过于不平衡，我们首先对样本使用 **SMOTE** 进行过采样。我们使用了五个可用于本题的模型分别是 **XGBoost**、**MLP**、**LightGBM**、**SVC**、**LR**，使用了准确率、召回率、精确度、F1 以及 AUC 指标对模型进行了评估，并绘制出了 ROC 曲线，结果显示 **XGBoost** 模型效果最好。最后我们使用 **XGBoost** 模型对全部血肿扩张概率进行了预测。

针对问题二，首先根据流水号将每位病人每次检查的水肿体积数据提取出，并且计算出对应的时间横轴，然后使用**分段形式的多段式拟合**方法来拟合，根据**不同阶数**和**不同分段点数**来拟合出较为合理的模型，然后得出残差。先用**主成分分析方法**对整理完的患者数据进行处理，然后再利用**手肘法**对其分析聚类个数，得到聚类个数之后使用 **K-Means** 算法将所有患者进行分类，得到不同的亚组，再整合完分类之后的病人数据与上小题得到的模型进行拟合预测，从而得到残差。将所有参与多次检查的病人水肿体积变化整理成一个矩阵，判断每位患者水肿体积情况是否恶化，再与每位患者接受的治疗方式进行**灰色关联分析**，得到治疗方式的对显著性顺序。对于第四小问，采用**多元线性回归模型**，完成基于治疗方法完成与否的分布统计的数据探索后，建立不同治疗方法对于患者血肿体积与水肿体积影响的关系模型，得出治疗方法中的脑室引流和降颅压治疗对平均血肿体积具有显著的正相关关系，而降颅压治疗对平均水肿体积也有显著的正相关关系的结论。

针对问题三，我们利用**方差筛选法**和**互信息法**结合、**距离相关系数法**和**递归特征消除法**结合，然后利用这两种方法对所有患者的个人史、疾病史、发病相关及影像结果进行特征筛选，然后将得到的主要特征分别带入**随机森林算法**、**支持向量机算法**、**XGBoost 算法**、**LightGBM 算法**中，并且我们使用**网格搜索**以及**随机搜索**来对其进行**超参数调优**，结合交叉验证训练从而得到十六个不同的模型，再根据指标挑选出拟合度高的模型，然后通过**集成学习投票的方式**对 mRS 评分等级进行预测。为解决第三小问，将任务划分为三个子问题：个人史、疾病史和治疗方法对于预后的关联性分析、不同位置的血肿与水肿体积对于预后的关联性分析和信号强度特征与形状特征对于预后的关联性分析。针

对各子问题的变量特征，分别采用 **Fisher 精确检验**、**Levene 检验**、**有序 Logistics 回归模型**和**单因素方差分析**来探究患者的预后（90 天 mRS）和个人史、疾病史、治疗方法及影像特征的关联关系。最后根据建立的模型得到了各变量间的关联关系，并参考相关临床诊疗方案和案例，对出血性脑卒中患者预后提出了有关疾病史重要性和影像特征价值等针对性建议。

本文的研究不仅可以为医疗决策提供新的见解，还可以为临床实践和未来的出血性脑卒中研究提供宝贵的数据和方法学范例。同时也将有助于改善出血性脑卒中患者的治疗和管理方案，提高他们的康复前景和生活质量。

关键词：XGBoost，随机森林，分段多项式拟合，主成分分析，K-Means 聚类，灰色关联，多元线性回归，递归特征消除法，支持向量机，网格搜索，**Fisher 精确检验**，**Levene 检验**，单因素方差分析

目录

一、 问题重述	5
1.1 问题背景	5
1.2 问题提出	5
二、 问题分析	6
三、 模型假设与符号说明	7
3.1 模型基本假设	7
3.2 符号说明	7
四、 数据预处理	8
4.1 数据清洗	8
4.2 数据转换	8
五、 问题一模型建立与求解	10
5.1 第一小问模型建立与求解	10
5.1.1 第一小问求解思路	10
5.1.2 第一小问模型建立	10
5.1.3 第一小问模型的求解与分析	11
5.2 第二小问模型建立与求解	12
5.2.1 第二小问求解思路	12
5.2.2 第二小问模型建立	12
5.2.3 第二小问模型求解与分析	15
六、 问题二模型建立与求解	19
6.1 第一小问模型建立与求解	19
6.1.1 第一小问求解思路	19
6.1.2 第一小问模型建立	19
6.1.3 第一小问模型求解与分析	21
6.2 第二小问模型建立与求解	25
6.2.1 第二小问求解思路	25
6.2.2 第二小问模型建立	25
6.2.3 第二小问模型求解与分析	27
6.3 第三小问模型建立与求解	33
6.3.1 第三小问求解思路	33
6.3.2 第三小问模型建立	33
6.3.3 第三小问模型求解与分析	34
6.4 第四小问模型建立与求解	39
6.4.1 第四小问求解思路	39
6.4.2 第四小问模型建立	40

6.4.3 第四小问模型求解与分析	43
七、 问题三模型建立与求解	46
7.1 第一、二小问模型建立与求解	46
7.1.1 第一、二小问求解思路	46
7.1.2 第一、二小问模型建立	47
7.1.3 第一、二小问模型求解与分析	52
7.2 第三小问模型建立与求解	58
7.2.1 第三小问求解思路	58
7.2.2 第三小问模型建立	59
7.2.3 第三小问模型求解与分析	62
7.2.4 对临床决策的建议	70
八、 模型评价与推广	72
8.1 模型的优点	72
8.2 模型的不足	72
8.3 模型的推广	73
参考文献	74
附录	75

一、 问题重述

1.1 问题背景

脑卒中，又叫做脑中风，是一种常见的脑血管疾病，通常是因为脑部血液供应中断或者血管破裂造成的，我国是脑卒中高发国家之一。脑卒中中的出血性脑卒中占据了病例的 10%-15%，其致死率和致残率非常高。出血性脑卒中通常是由脑动脉瘤破裂或脑动脉异常等因素引起，患者通常会有发病急、病情危重、免疫力特别低，以及肢体活动功能上的障碍等症状。不仅对患者的健康造成了巨大的威胁，而且对患者家庭的生活和经济造成了很大的负担。^[1]

为了帮助医生更好的制定治疗方案，减轻患者的痛苦，提高患者的生活质量，精准预测出血性脑卒中患者的病情发展至关重要。得益于医学影像技术和人工智能的快速发展，为实现预测出血性脑卒中病情预测提供了新的可能。^[2]

本文的研究目的是基于提供的真实临床数据，探索出血性脑卒中患者的血肿扩展风险，水肿发生和进展规律以及治疗干预水肿进展的关联因素，出血性脑卒中患者的预后预测以及关键因素。

本文将利用了一些人工智能的方法，对真实的临床数据进行了分析预测，旨在帮助医生更精确地评估出血性脑卒中患者的风险以及预后，继而改善患者的治疗和康复。我们希望本文的工作能够为未来的临床决策提供重要的决策支持，并为出血性脑卒中的研究以及治疗方案提供有用的科学依据。

1.2 问题提出

问题一：血肿扩张风险建模

a) 根据所提供的相关数据，在前 100 例出血性脑卒中患者（sub001 至 sub100）中，判断是否发生了血肿扩张，定义标准为后续检查相对体积增加 $\geq 33\%$ 或绝对体积增加 $\geq 6\text{ mL}$ ，如果发生血肿扩张事件的话，记录下血肿扩张的时间。

b) 基于前 100 例患者的个人史、疾病史、发病治疗相关信息以及首次影像检查结果，构建模型以预测所有患者是否会发生血肿扩张事件。

问题二：血肿周围水肿发生及进展建模

a) 利用前 100 例患者的水肿体积和重复检查时间点数据，构建全体患者水肿体积随时间的进展曲线，最后需要计算前 100 例患者的真实值与拟合曲线之间的残差。

b) 探索不同患者群体（3-5 个亚组）的水肿体积随时间的个体差异，构建每个亚组的水肿体积进展曲线，并计算前 100 例患者的真实值与各曲线之间的残差，同时提供亚组的标识。

c) 从所给定的数据中，分析不同治疗方法对水肿体积进展模式的影响。

d) 建模分析出血性脑卒中血肿体积、水肿体积和治疗方法之间的关系。

问题三：出血性脑卒中患者预后预测及因素分析

a) 基于前 100 例患者的个人史、疾病史、发病相关信息以及首次影像结果，建立模型来预测所有患者的 90 天 mRS 评分。

b) 利用前 100 例患者的临床、治疗、首次影像以及随访影像结果，预测所有含有随访数据的患者（sub001 至 sub100, sub131 至 sub160）的 90 天 mRS 评分。

c) 分析出血性脑卒中患者的预后（90 天 mRS 评分）与个人史、疾病史、治疗方法和影像特征之间的关联，并提出一些临床决策的建议。

二、 问题分析

问题一：

问题一主要是根据已知的数据计算是否发生血肿扩展，再根据是否发生血肿扩展的计算结果以及相关影响因素建立数学模型预测数据表中的所有患者发生血肿扩张的概率。由题目可知，血肿扩张是根据血肿体积的变化量决定的。如果后续的血肿体积比首次检查的血肿体积相对体积不小于 33% 或者体积增加的量不小于 6 毫升，则认定为发生血肿扩张。血肿扩张判定公式如下：

$$\text{血肿扩张} = \begin{cases} 1 & HM_{\text{volume},48h} - HM_{\text{volume},\text{初始}} \gg 6ml \text{ or } \frac{\Delta HM_{\text{volume}}}{HM_{\text{volume},\text{初始}}} \gg 33\% \\ 0 & HM_{\text{volume},48h} - HM_{\text{volume},\text{初始}} < 6ml \text{ or } \frac{\Delta HM_{\text{volume}}}{HM_{\text{volume},\text{初始}}} < 33\% \end{cases}$$

其中 1 代表患者发生了血肿扩张，反之，0 代表患者没有发生血肿扩张。 $HM_{\text{volume},48h}$ 表示发病后 48 小时内的一次 HM_{volume} 值， $HM_{\text{volume},\text{初始}}$ 表示首次影像检查的 HM_{volume} 值。 $\Delta HM_{\text{volume}}$ 表示发病后 48 小时内的一次 HM_{volume} 值比首次影像检查的 HM_{volume} 值的增量。

针对第一个小问，首先需要进行数据表的数据合并，再根据题目所给的血肿扩展的定义计算判断患者是否发生了血肿扩张，由附件表格 1 中对应的时间计算确定当前的检查是在发病 48 小时内，最后输出是否发生了血肿扩张以及发生的时间。针对第二个小问。

针对第二个小问，需要根据 3 个表的相关数据建立一个预测模型，预测出所有患者发生血肿扩张的概率。首先需要进行数据清洗，处理好缺失值和异常值，对分类变量进行独热编码，对连续变量进行归一化处理。在处理好数据之后，我们发现样本非常不平衡，所以用 SMOTE 进行过采样在进行模型的训练。预测模型非常多，选择 LGBM、XGB、SVC、MLP 和 LR 常见的预测模型进行实验对比，以准确率、召回率、精确度、F1 分数、AUC 作为评价指标。选择表现最好的模型进行训练，最后预测出结果。

问题二：

针对第一小问，整合所有患者水肿体积与时间进展的散点图之后，发现数据较为复杂，采用分段多项式拟合的方法，对散点图进行拟合，并且调整不同的分段点数和多项式拟合阶数来测试不同的参数带来的拟合效果，然后挑选出最好的一个来进行预测，并计算出所需的残差值。

针对第二小问，使用主成分分析法来进一步处理数据之后，然后通过手肘法来确定聚类个数，知晓聚类个数之后，使用 K-Means 算法对所有患者进行聚类分析，然后分成不同的亚组，利用第一小问得到的模型来分别对每个亚组进行预测计算残差值。

针对第三小问，计算出当前检查出的水肿体积与前一次检查水肿体积的差异值，可以得到患者水肿体积变化的情况，然后根据患者水肿体积的变化情况来对其身体状况进行统计，然后根据结果与患者接受的治疗方法进行灰色关联分析得到最后的治疗方式的显著性作用排序。

针对第四小问，需要参照表 1 和表 2 的数据建立相关性分析模型，探究不同的治疗方法对于患者脑部血肿体积与水肿体积之间的关系。对此，可以在先对数据分布做出统计后，绘制出相应的图表，得到数据探索的初步方向。再结合多元线性回归模型，更严谨地得到治疗方法对于血肿体积和水肿体积的回归方程以及各治疗方法自变量的显著性结果。

问题三：

针对第一、二小问，可以看出这是一个高维的复杂分类预测问题，使用传统的机器学习算法需要我们进一步对数据优化，首先采用不同的特征筛选方法对数据集进行处理，然后将划分出的主要特征作为变量，采用经典的随机森林算法、支持向量机算法、XGBoost 算法、LightGBM 算法来作为主要算法，然后使用随机搜索和网格搜索作为超参数调优的策略，最终训练出主要的几个模型，预测并对分类结果进行投票，从而得到最终的结果。

针对第三小问，需要划分为多个子问题，以便于高效准确地解决。对于各子问题，需要做大量的数据处理，包括变量的提取、二元数据类型的转换以及对血压、年龄等连续变量的分类。之后采用多种检验方法对特征不同的数据进行检验，包括对大量的分布统计数据进行单因素方差分析。最后根据 90mRs 的有序性建立有序 Logistics 回归模型，分析出患者预后与个人史、疾病史、治疗方法和影像特征之间的相关性，参照相关临床的诊疗信息为今后医生在脑卒中临床决策提出针对性建议。

三、模型假设与符号说明

3.1 模型基本假设

- (1) 提供的所有数据均为真实有效，符合实际情况。
- (2) 所有患者的 mRS 评分等级只受到出血性脑卒中的影响。
- (3) 默认除了提供的数据之外的因素不会影响患者病情。
- (4) 在使用有序 Logistics 回归模型中，预后是具有从低到高层次的有序性。
- (5) 在使用梯度提升树模型（XGBoost 和 LightGBM）假设特征是独立同分布的，这是它们能够逐步构建树的基础。

3.2 符号说明

表 1 符号说明

序号	符号	含义
1	NP	正常血压
2	NH	正常高压
3	H1	轻度高血压
4	H2	中度高血压
5	X	归一化矩阵
6	ACC	准确率
7	R	召回率
8	P	精确率
9	$y_i(x)$	多项式拟合第 i 段
10	RSS	最小化残差平方和
11	Yr	降维数据点
12	DBI	簇间距离之比
13	CH	类内的紧密度
14	$I(X;Y)$	互信息
15	$H(X Y)$	条件熵

四、 数据预处理

4.1 数据清洗

数据采集之后，往往会出现各种问题，比如缺失值、异常值等。这时候就需要进行数据清洗，将不符合要求的数据剔除或者进行填充。具体来说，数据清洗包括以下几个方面：

在给定的数据集中，往往会出现很多数据上的问题，比如缺失值、异常值等。所有，我们首先需要做的就是数据清洗，将错误的数据去除或者就行填充。数据清洗包括如下两个方面：

（1）处理缺失值

使用 python 中的 pandas 库对所有数据进行分析，并没有发现缺失值，因此不用处理缺失值。

（2）处理异常值

附件 1 中患者 sub074 的首次影像的流水号为 20180719000630，但是在表 2 中的首次影像流水号则为 20180719000020，而在表 2 中 sub074 的随访 1 流水号才为 20180719000630，所以最正确的做法是将件 1 中患者 sub074 的首次影像的流水号改正为 20180719000020

4.2 数据转换

（1）独热编码

独热编码，又被称为一位有效编码，是一种用来对一组 N 个状态进行编码的方法。它利用 N 位状态寄存器，每个状态都有自己独立的寄存器位，并且在任何给定时刻，只有一位是有效的，即只有一位的值为 1，而其他位都为 0。这种编码方法通常用于表示参数，使用 0 和 1 来表示不同的状态。在机器学习领域，也引入了"one-hot 向量"的概念，它是一个在任意维度的向量中，只有一个维度的值为 1，而其他维度的值都为 0。将分类数据转换成 one-hot 向量的过程通常被称为 one-hot 编码。在统计学中，虚拟变量也代表了类似的概念。

我们可以将性别这一指标进行独热编码。在机器学习中，特征之间的距离或相似度计算对于回归、分类、聚类等算法至关重要。通常，我们使用欧式空间的相似度计算或余弦相似性来衡量特征之间的相似程度。然而，对于离散特征，常规的计算方法存在一些限制，因为它们基于欧式空间的假设。为了解决这个问题，我们可以使用独热编码将离散特征的取值映射到欧式空间中的点。这种编码方式使得离散特征之间的距离计算更加合理，从而有助于更准确地进行分类和其他机器学习任务。160 例患者男女比例饼图如下所示：

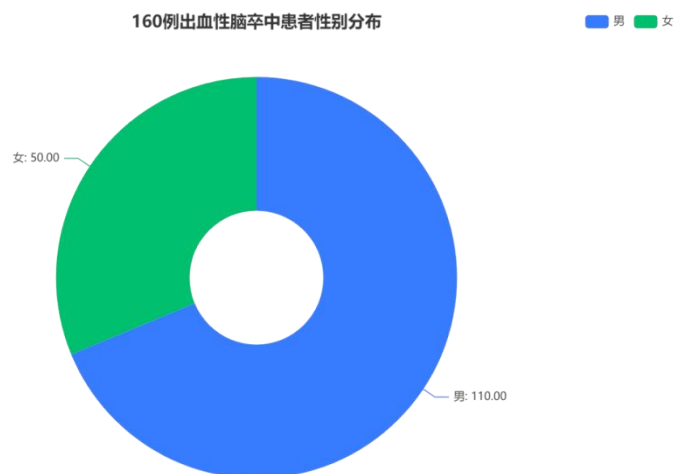


图 4.1 160 例患者的性别分布

(2) 数据归一化

在机器学习中，归一化处理是必要的，因为它有助于消除不同特征之间的量纲差异，确保它们在相同的数值范围内。这样做的主要目的是避免量纲对模型的影响。不同特征可能具有广泛不同的取值范围，有些可能非常大，而其他则相对较小。如果不进行归一化处理，模型可能会过于关注那些具有更大取值范围的特征，而忽略那些取值范围较小的特征，从而降低了模型性能。

此外，归一化还有助于确保各个特征之间的变化幅度相似，这可以减少梯度下降算法的迭代次数，从而提高模型的收敛速度。并且归一化还可以减小异常值或极端值的影响。如果某个特征存在异常值，其取值范围可能远远超过其他特征，这可能导致模型对该特征过于敏感。通过归一化，可以减少异常值的影响，提高模型的稳定性。

在一些机器学习算法中，如逻辑回归和支持向量机，我们需要为特征进行权重计算。然而，当特征的取值范围存在较大差异时，可能会引发问题。这种差异可能导致模型对取值范围较大的特征赋予过高的权重，而对取值范围较小的特征赋予过低的权重，从而对模型性能产生负面影响。计算过程如下，假设有 n 个要评价的对象， m 个评价指标构成的矩阵如下：

$$X = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{pmatrix}$$

若评价指标为正向指标，则对进行归一化处理为：

$$x'_{ij} = \frac{x_{ij} - \min(x_{1j}, x_{2j}, \cdots, x_{nj})}{\max(x_{1j}, x_{2j}, \cdots, x_{nj}) - \min(x_{1j}, x_{2j}, \cdots, x_{nj})}$$

若评价指标为负向指标，则对进行归一化处理为：

$$x'_{ij} = \frac{\max(x_{1j}, x_{2j}, \cdots, x_{nj}) - x_{ij}}{\max(x_{1j}, x_{2j}, \cdots, x_{nj}) - \min(x_{1j}, x_{2j}, \cdots, x_{nj})}$$

五、 问题一模型建立与求解

5.1 第一小问模型建立与求解

5.1.1 第一小问求解思路

根据题目所给出的判断是否发生血肿扩张的判别条件，我们首先提取出表格 1 中的入院首次影像检查流水号和发病到首次影像检查时间间隔两列，根据表 1 中的入院首次影像检查流水号在附件 1 中找到对应的时间，让这两个字段形成映射。根据字段发病到首次影像检查时间间隔和首次检查时间计算出首次发病的时间。之后我们提取出表格 2 字段中的各时间点流水号及对应的 HM_volume 值。在发病时间的之后 48 小时内，依次计算后续检查比首次检查的相对体积增加值和绝对体积增加值是否不小于 33%以及 6ml，则认定该患者发生了血肿扩张，并记录下发生血肿扩张的时间。

5.1.2 第一小问模型建立

根据上述的思路，按照如下所示的流程图计算所要求的结果：

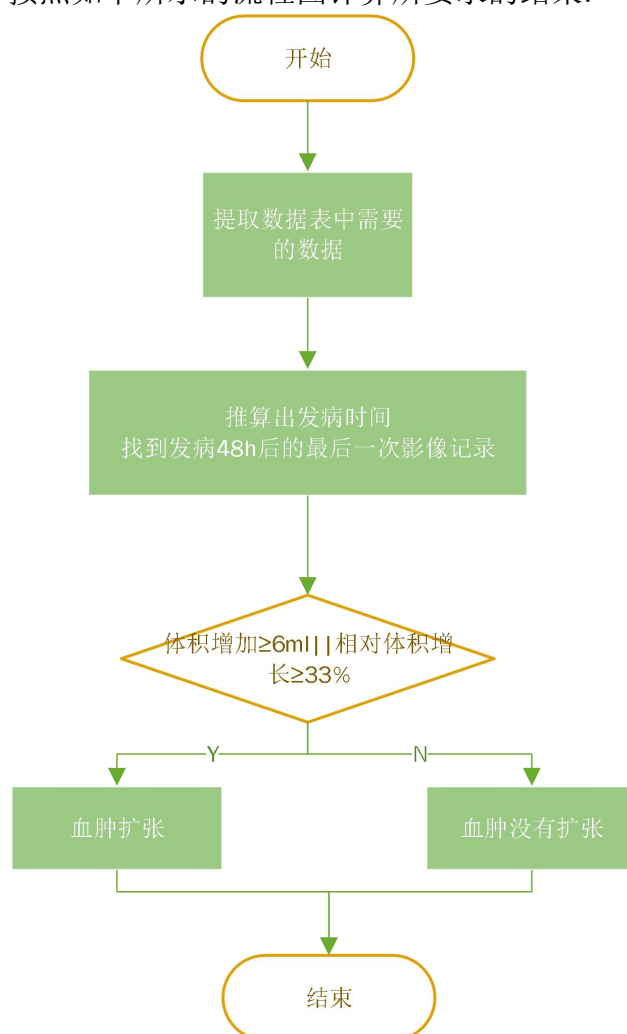


图 5.1 计算流程图

5.1.3 第一小问模型的求解与分析

本小问属于简单的计算题，需要根据所提供的几个数据文件提取出我们需要的字段，然后计算得出最终的结果。最终计算出前 100 名患者中，共有 22 位患者在发病 48 小时内发生了。发生了血肿扩张的患者信息如下表所示：

表 5.1 患者血肿扩张的信息

ID	是否扩张	发生时间
sub003	1	9.5225
sub005	1	26.4675
sub009	1	40.056111
sub017	1	14.870278
sub033	1	30.813056
sub036	1	39.501111
sub038	1	15.809167
sub039	1	29.176111
sub048	1	12.860833
sub054	1	16.225
sub057	1	14.615833
sub060	1	23.726111
sub061	1	6.5416667
sub070	1	9.6533333
sub076	1	15.993611
sub077	1	14.119444
sub079	1	27.847222
sub080	1	20.573333
sub081	1	27.421667
sub085	1	15.613333
sub092	1	11.924722
sub095	1	7.4316667
sub098	1	42.756667
sub099	1	17.672778

5.2 第二小问模型建立与求解

5.2.1 第二小问求解思路

根据上一问得到的患者发生血肿扩张的情况，我们首先对样本使用 SMOTE 进行过采样。之后使用了五个机器学习算法分别是 XGBoost 、MLP 、LightGBM、SVC、LR 训练出相应模型之后，使用准确率、召回率、精确度、F1 以及 AUC 指标对模型进行了评估，然后利用效果最好的模型对全部血肿扩张概率进行预测就可以得到最后答案。

5.2.2 第二小问模型建立

在测试数据集中，由于自变量的数量差别差异较大，会使模型产生偏差，对预测分析产生一些问题，所有在进行预测之前首先选择 SMOTE 进行过采样处理。SMOTE (Synthetic Minority Oversampling Technique)，是一个合成少数类过采样技术。它的基本思想是对少数类样本进行分析并根据少数类样本人工合成新样本添加到数据集中。^[3]SMOTE 算法步骤如下图所示：

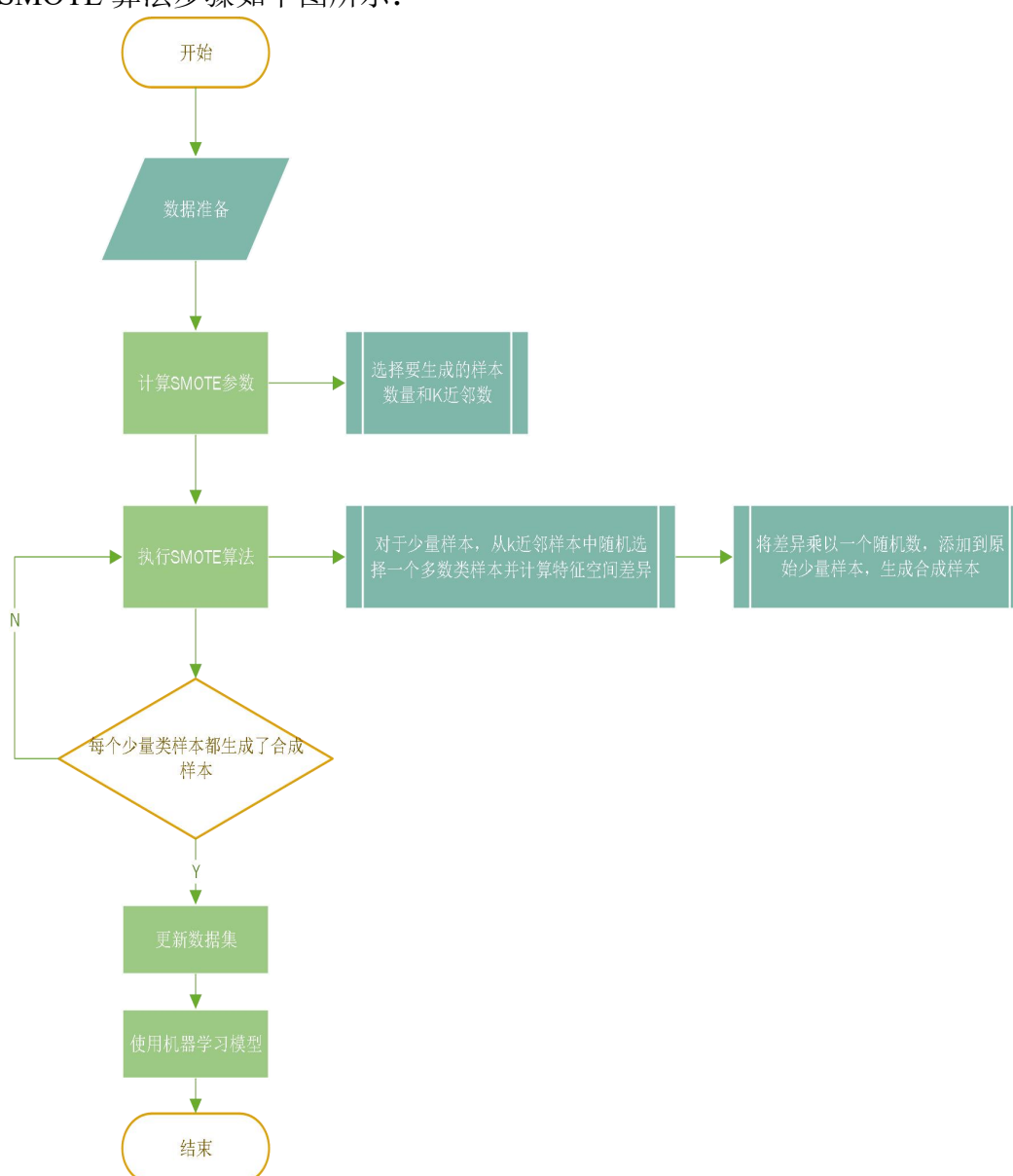


图 5.2 SMOTE 算法步骤

我们对如下 5 个模型进行对比实验分析：

XGBoost (eXtreme Gradient Boosting): 一种梯度提升树模型，使用正则化技术来避免过拟合，支持自定义损失函数。^[4]XGBoost 模型进行分类预测流程图如下所示：

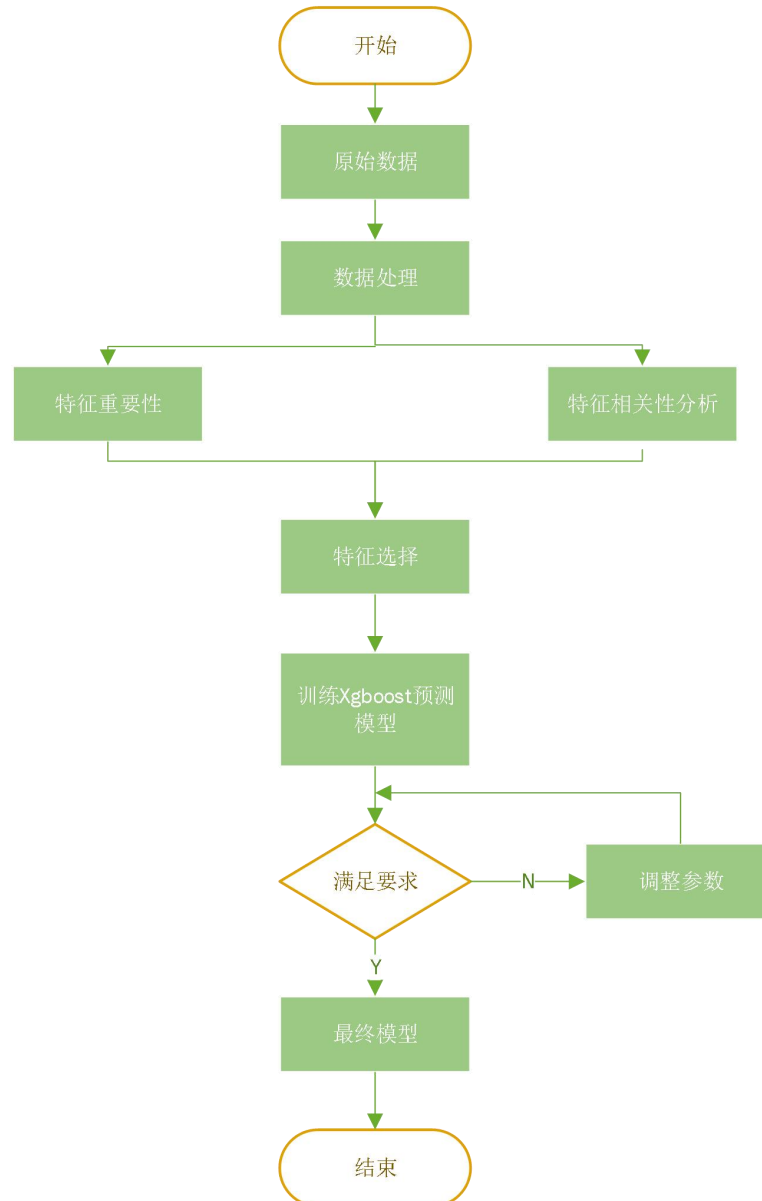


图 5.3 XGBoost 分类预测流程

MLP (Multilayer Perceptron): 是一种人工神经网络 (Artificial Neural Network, ANN) 的变体，也叫做前馈神经网络。它有由多个神经元层组成，每一层都与前一层全连接。MLP 的每个神经元都有一个激活函数，用于处理输入信号和产生输出。常用的激活函数有 sigmoid 函数、ReLU 函数等。每个神经元接收来自上一层神经元的输出，并根据权重和偏置进行加权求和，然后通过激活函数进行非线性转换。MLP 的每层都有若干个神经元，并且每个神经元都与下一层的所有神经元相连。

在训练阶段，MLP 通过反向传播算法来调整神经元之间的连接权重，以最小化训练样本的损失函数。反向传播算法使用梯度下降的思想，通过计算损失函数对权重的偏导数来更新权重。训练完成后，MLP 可以用于对未知样本的分类、回归等任务。^[5]

LGBM (Light Gradient Boosting Machine): 是一种梯度提升树模型，使用决策树作为基础的学习器，通过不断训练决策树来提升模型的性能。^[6]

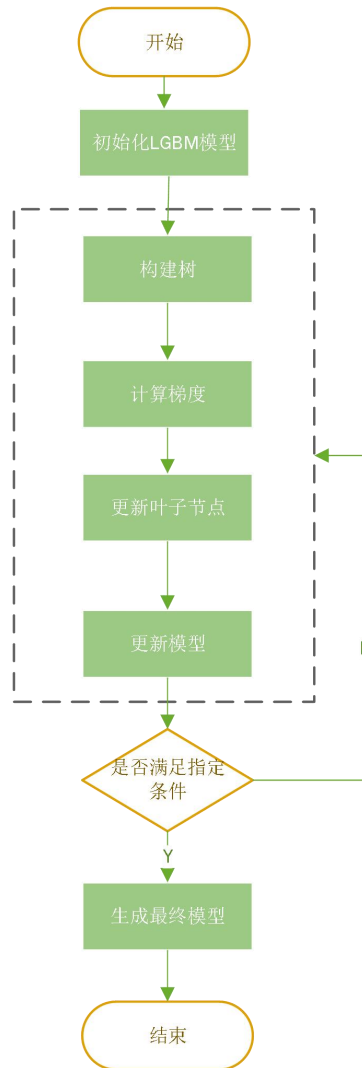


图 5.4 LGBM 模型分类预测流程

SVC (Support Vector Classifier):是一种支持向量机的变体模型,最终目的是找到一个最佳平面,将不同类别的数据分开,实现最优化分类问题。[7]

LR (Logistic Regression):是一种常见的二分类问题的线性模型,需要假设输入和输出是具有线性关系的。Logistic 回归分析是一种强大的统计工具,用于处理二元分类问题。Logistic 回归的 y 是分类变量, Logistic 回归分析的自变量 x 可以是定量或者变量。根据输入特征的线性组合来预测一个二元目标变量的概率。[8]Logistic 回归的数学表达式如下:

$$P(Y = 1|X) = \frac{1}{1 + e^{-(b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n)}}$$

其中, $P(Y = 1|X)$ 是目标变量为 1 的概率, $X_1, X_2, \dots, X_n$ 是输入特征 $b_0, b_1, b_2, \dots, b_n$ 是模型的系数。

在 Logistic 回归中,通过最大似然估计来估计回归系数,使得模型对观测数据的概率最大化。我们使用梯度下降算法来最小化损失函数,最终得到最优的回归系数。

为了评估以上模型的性能,我们选择了以下常用的指标进行模型性能评价:

准确率 (Accuracy): 用来衡量模型的整体性能,指模型正确预测的样本数量占总样本数量的比例。计算公式如下所示:

$$ACC = \frac{TP + TN}{TP + TN + FP + FN}$$

其中，TP(True Positive)意思是真阳性，表示模型将正类别样本正确地预测为正类别的数量。TN (True Negative)意思是真阴性，表示模型将负类别样本正确地预测为负类别的数量。FP (False Positive)意思是真阴性，指的是模型将负类别样本错误地预测为正类别的数量。FN (False Negative)意思是真阴性，指的是模型将正类别样本错误地预测为负类别的数量。

召回率 (Recall) :也称作真阳性率，指的是，在所有的实际正例样本中，模型成功预测为正例的比例。计算公式如下所示：

$$R = \frac{TP}{TP + FN}$$

精确度 (Precision)：指的是在模型预测的所有正例样本中为正例的比例。公式如下所示：

$$P = \frac{TP}{TP + FP}$$

F1 分数 (F1-Score)：综合考虑了精确度和召回率，帮助平衡精确度和召回率。计算公式如下所示：

$$\frac{2}{F_1} = \frac{1}{P} + \frac{1}{R}$$

AUC (曲线下面积)：指的是 Receiver Operating Characteristic (ROC) 曲线下的面积。AUC 取值范围从 0 到 1，AUC 值越接近 1，模型性能越好。

以上几个评价指标通常一起使用，可以更全面的评估模型性能。

5.2.3 第二小问模型求解与分析

以发生血肿扩张的概率作为预测结果，在对数据进行了 SMOTE 过采样操作之后，几个模型的指标如下表所示：

表 5.2 模型各指标数

模型	XGBOOST	MLP	LGBM	SVC	LR
ACC	0.78	0.49	0.76	0.45	0.49
RECALL	0.83	0.4	0.81	0.81	0.58
PRECISION	0.75	0.49	0.73	0.47	0.49
F1	0.78	0.57	0.76	0.4	0.5
AUC	0.85	0.48	0.83	0.46	0.56

这几个模型采样后的训练集和测试集 Receiver Operating Characteristic(ROC)曲线以及对比如下几个图所示：

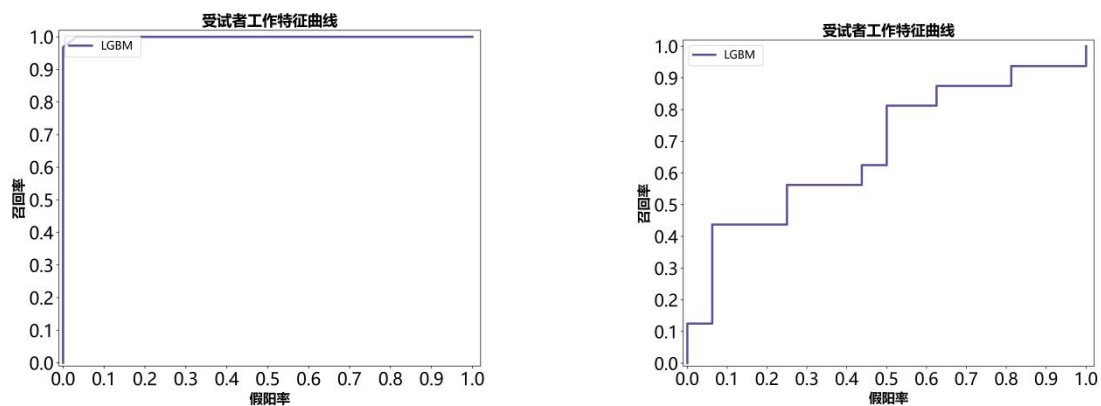


图 5.5 LGBM 模型的训练集和测试集的 ROC 曲线

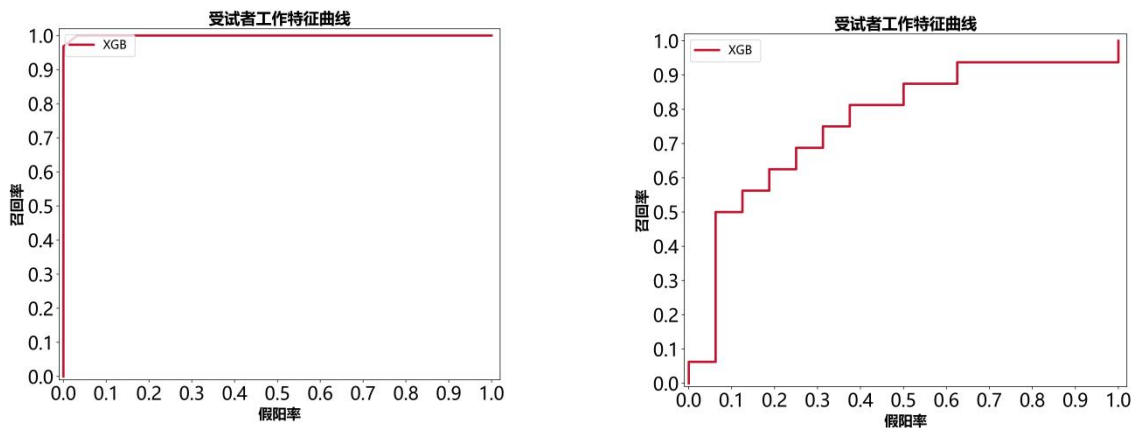


图 5.6 XGBoost 模型的训练集和测试集的 ROC 曲线

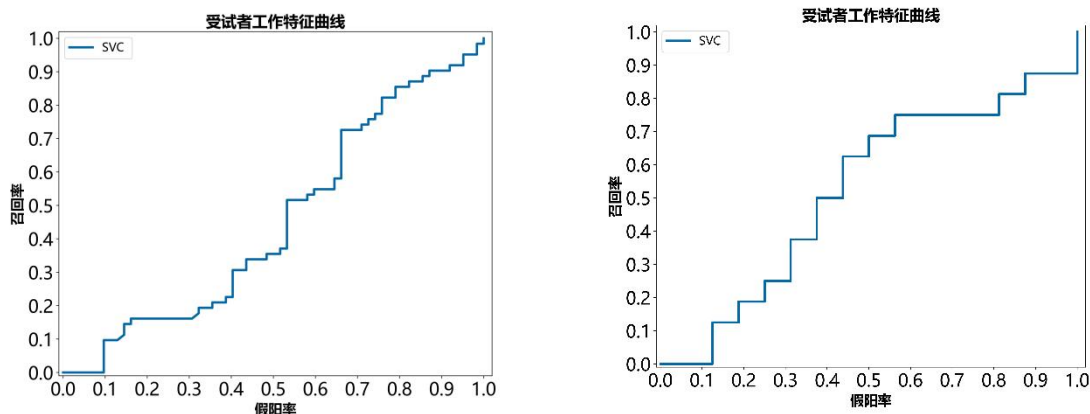


图 5.7 SVC 模型的训练集和测试集的 ROC 曲线

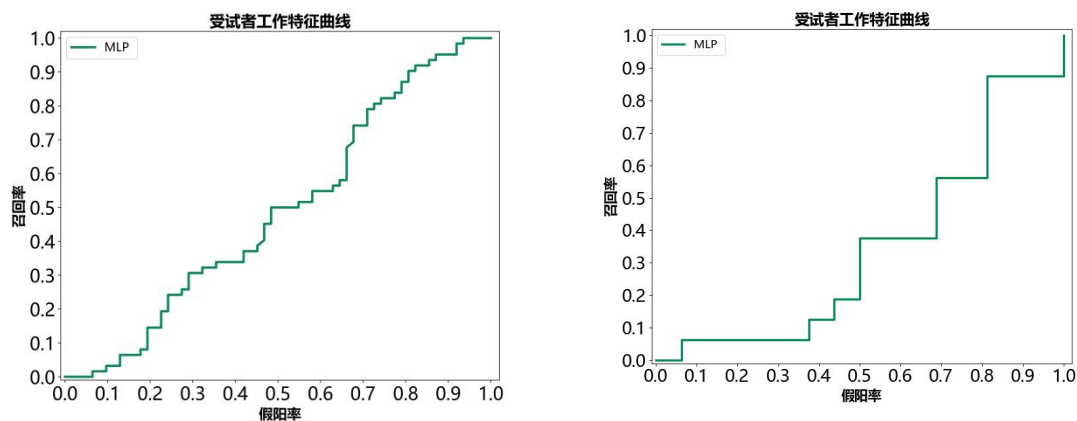


图 5.8 MLP 模型的训练集和测试集的 ROC 曲线

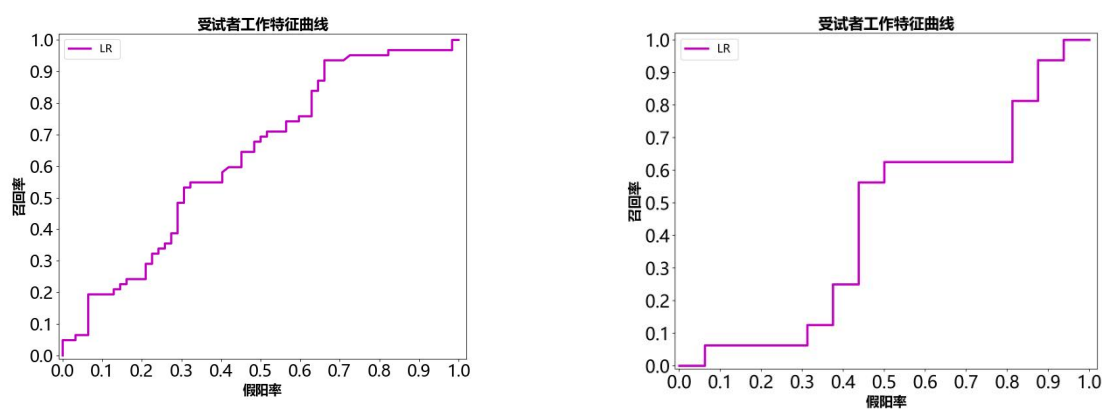


图 5.9 LR 模型的训练集和测试集的 ROC 曲线

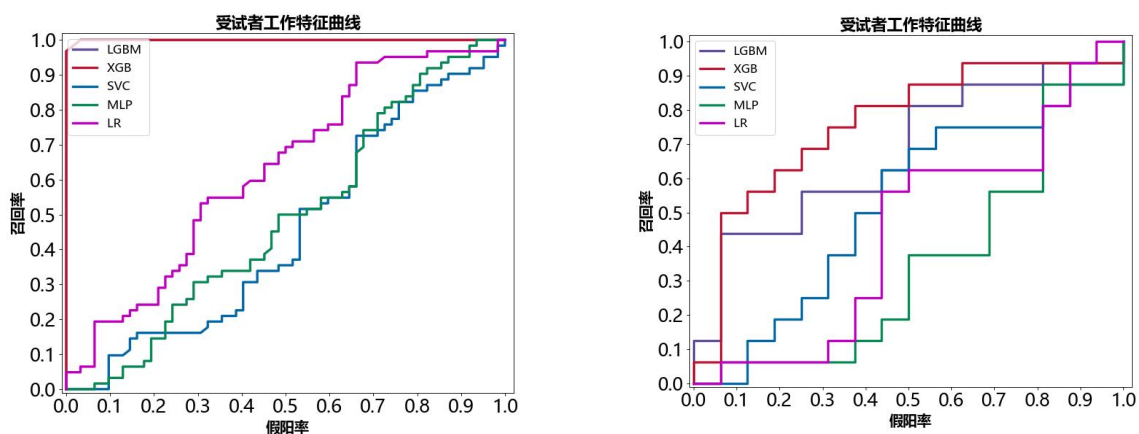


图 5.10 五个模型的训练集和测试集的 ROC 曲线对比图

由上述可知在比较的几个方法中，XGBoost 模型表现是最好的，所以我们选择 XGBoost 模型训练数据集，预测最终的结果。

发生血肿扩张以及发生时间的分布图如下所示：



图 5.11 血肿扩张分布图

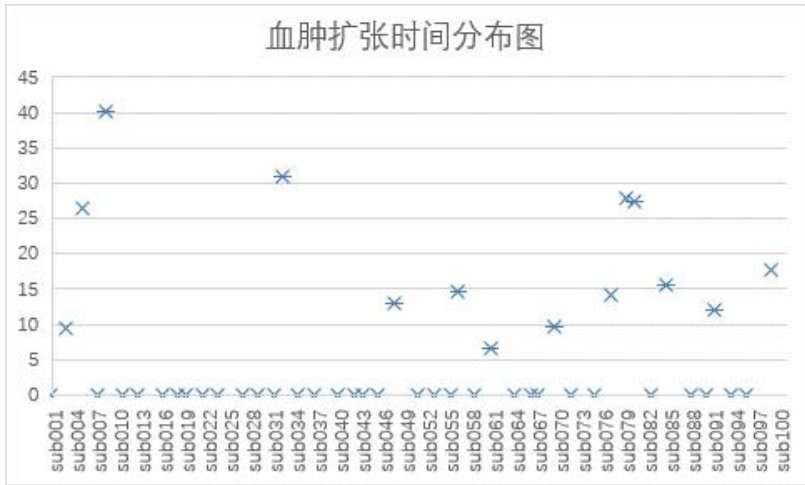


图 5.12 血肿扩张时间分布图

应用 XGBoost 模型对本小问的数据进行分析预测。预测所有患者的血肿扩张的概率见答案文件所示。所有患者发生血肿扩张的概率分布如下图所示：

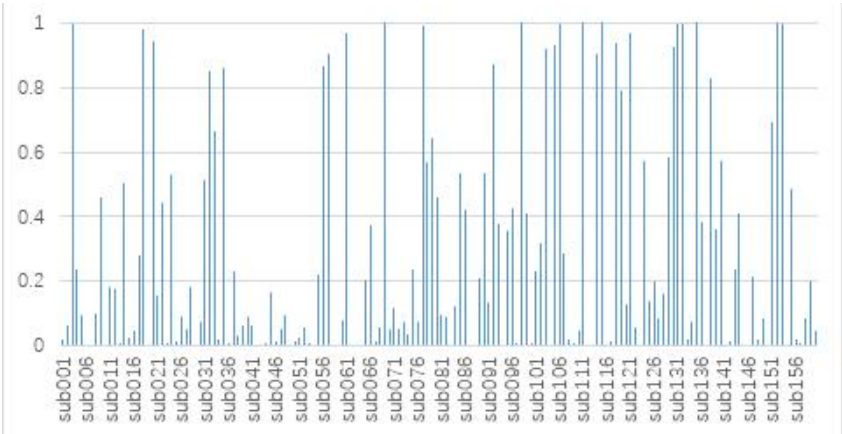


图 5.13 血肿扩张概率分布图

从分析结果可知，通过使用 XGBoost 模型，我们可以根据患者的个人史、疾病史、发病及治疗相关史和影像检查结果预测出患者发生血肿扩张的概率。对 XGBoost 模型的准确率进行了检测，该模型准确率高，可以应用于临床实践，为医者和患者提供准确的血肿扩张预测，从而可以在发生血肿扩张前进行干预，减轻医生的工作量以及患者的痛苦。

六、 问题二模型建立与求解

6.1 第一小问模型建立与求解

6.1.1 第一小问求解思路

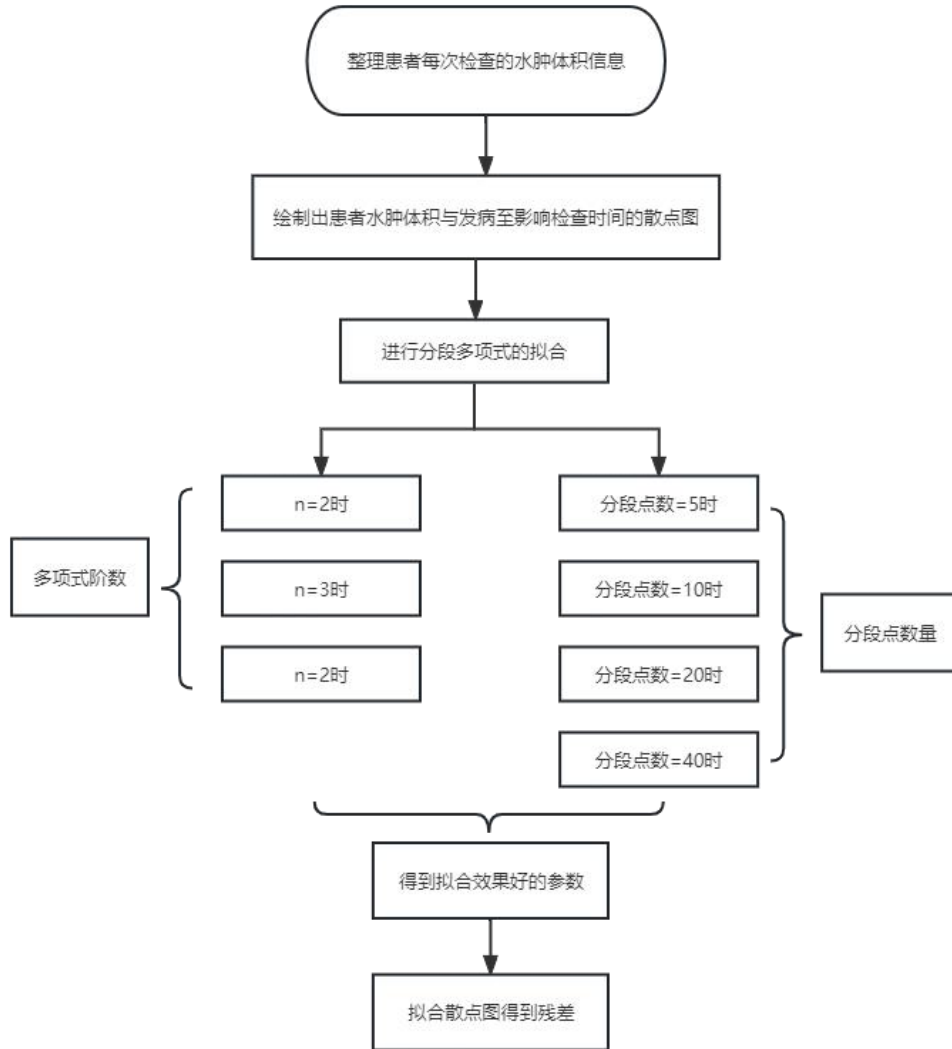


图 6.1 分析患者水肿体积和时间进展的关系主要思路

6.1.2 第一小问模型建立

(1) 分段多项式拟合

分段多项式拟合是一种用于拟合数据的方法，其中数据被划分为多个段，然后在每个段内使用多项式函数来适应数据的特征。这种方法的目标是找到最佳的多项式函数，以最小化实际数据点与拟合曲线之间的差异。分段多项式拟合适用于处理具有非线性特征的数据，并允许在不同的段内使用不同的多项式函数来拟合数据。^[9]

拟合的主要步骤包括以下几个方面：确定分割点：首先，需要确定如何将数据集划分为不同的段。这些分割点可以是预先定义的，也可以通过某种算法（如插值、聚类等）来自动确定。在每个段内进行拟合：

对于每个段，需要选择一个适当的多项式函数，以便在该段中拟合数据。通常可选择的多项式包括线性、二次、三次等，具体选择取决于数据的性质。

求解多项式系数：在每个段内使用的多项式函数需要求解对应的多项式系数。这可以通过拟合算法，如最小二乘法，来实现。系数的选择将影响拟合曲线的形状。

组合段：最后，将每个段内的拟合曲线组合在一起，形成整体的分段多项式拟合曲线。这涉及将各个段之间的拟合曲线平滑连接，以确保整体拟合的连续性和一致性。

分段多项式拟合的公式通常表示为：

在第 i 个段内：

$$y_i(x) = a_i \cdot x^n + a_{i-1} \cdot x^{n-1} + \dots + a_2 \cdot x^2 + a_1 \cdot x + a_0$$

其中， $y_i(x)$ 是在第 i 个段内的拟合函数， x 是自变量， a_i 是第 i 个段的多项式系数， n 是多项式的阶数。

分段多项式拟合的关键是通过求解每个段的多项式系数来获得整个数据集的分段多项式拟合曲线。这种方法相对于分段线性拟合更能够适应数据中的非线性特征，但也需要注意可能出现的过度拟合问题。

在实际应用中，需要谨慎选择分割点和多项式的阶数，以平衡拟合的准确性和过度拟合的风险。选择不合适的分割点或使用过高阶的多项式可能导致模型在训练数据上表现良好，但在新数据上表现不佳。因此，调整分割点和多项式的阶数是分段多项式拟合中的重要任务，通常需要通过交叉验证等方法来进行模型选择和调优。

(2) 非线性最小二乘法拟合

非线性最小二乘法拟合是一种用于拟合非线性函数模型与实际数据之间关系的方法。与线性最小二乘法不同，非线性最小二乘法拟合的模型函数可以是非线性的。这种方法通常用于处理数据中包含复杂非线性关系的情况，以找到最佳拟合曲线或曲面来描述数据的行为。^[10]

假设我们有一个非线性函数模型，表示为：

$$y = f(x, \beta)$$

其中， y 是因变量， x 是自变量， β 是待估计的参数向量。

非线性最小二乘法拟合的目标是找到最佳的参数向量 β ，以最小化实际数据点与拟合曲线之间的差异。这可以通过最小化残差平方和 (RSS) 来实现，即最小化以下目标函数：

$$RSS(\beta) = \sum (y_i - f(x_i, \beta))^2$$

其中， y_i 是第 i 个数据点的实际值， x_i 是对应的自变量值。

为了最小化目标函数，可以使用迭代的优化算法，例如高斯-牛顿法、Levenberg-Marquardt 算法等。这些算法通过不断调整参数向量 β 的值，直到达到最小化残差平方和的效果。

非线性最小二乘法拟合的基本步骤包括：

选择初始参数：首先，需要确定适当的初始参数向量 β 的值。

迭代优化：使用迭代优化算法，如高斯-牛顿法或 Levenberg-Marquardt 算法，反复调整参数向量 β 的值，以最小化残差平方和。

停止准则：在迭代过程中，可以设定停止准则，如达到最大迭代次数或残差平方和的变化小于某个阈值等，以确定何时终止优化。

解释结果：最终，获得最佳的参数向量 β 的估计值。这些估计值可以用来解释数据与模型之间的关系。

需要注意的是，选择初始参数向量的值非常关键，不同的初始值可能导致不同的拟合结果。因此，通常需要通过启发式方法或经验来选择适当的初始值。此外，由于非线

性最小二乘法可能存在多个局部最小值，因此有时需要尝试不同的初始参数向量值，以寻找全局最小值。^[11]

(3) 残差

残差值是拟合曲线与实际观测数据之间的误差，用于衡量拟合的准确程度。每个数据点的残差值可以通过实际观测值与拟合曲线上对应点的差异计算得到。具体而言，对于数据点 (X_i, Y_i) ，拟合曲线上对应的预测值为 Y_{fit_i} ，因此该数据点的残差值为 $残差 = Y_i - Y_{fit_i}$ 。

残差值在拟合分析中用于评估拟合模型的质量和适用性。当拟合曲线与实际观测数据高度吻合时，残差值接近零。相反，较大的残差值可能表明拟合模型不够准确，或者该模型不适合描述实际数据。

6.1.3 第一小问模型求解与分析

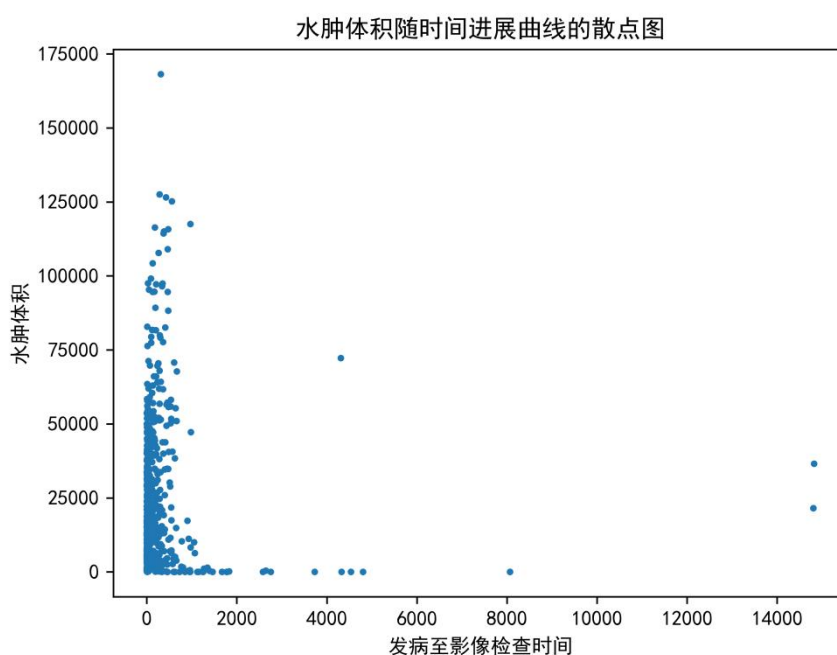


图 6.2 水肿体积随时间进展曲线的散点图

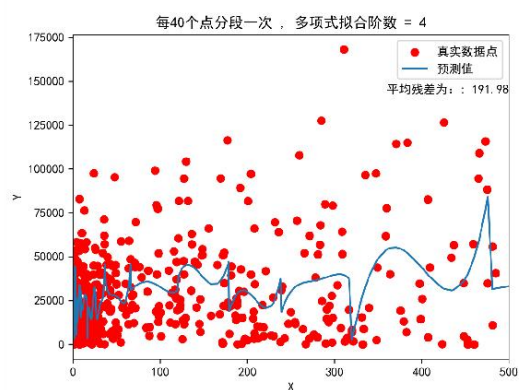
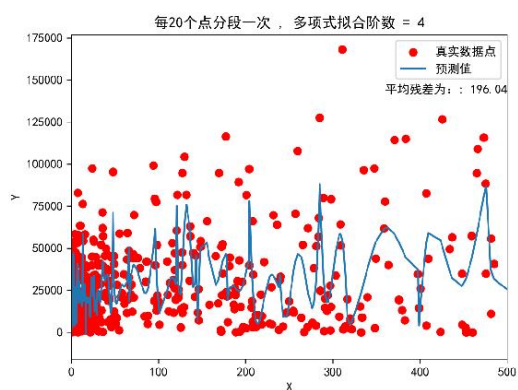
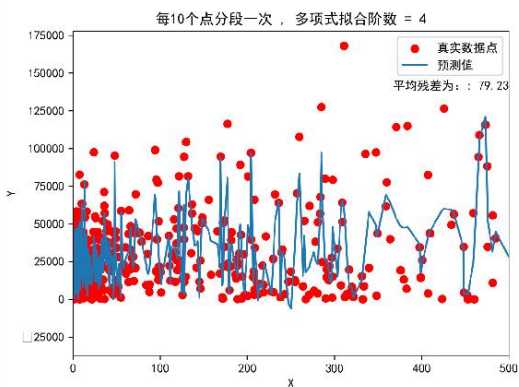
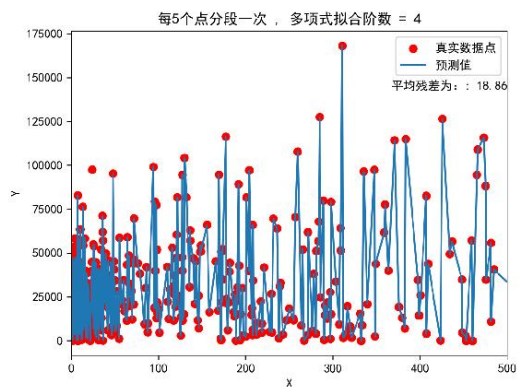


图 6.3 多项式拟合阶数为 4，不同分段点结果

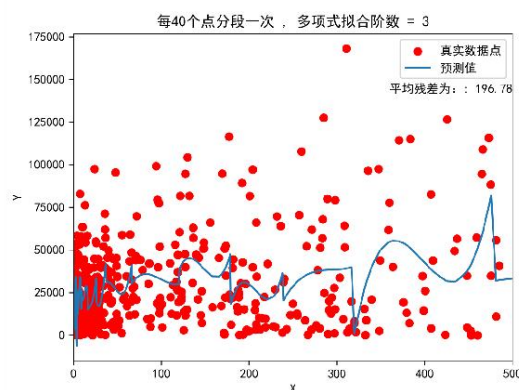
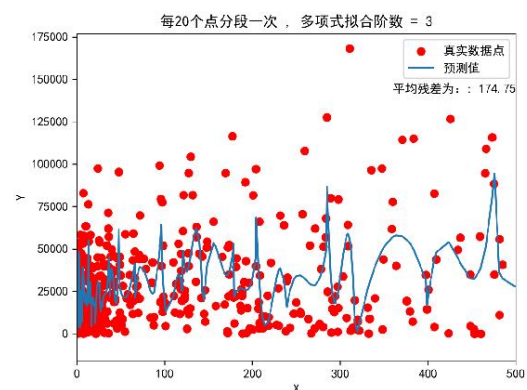
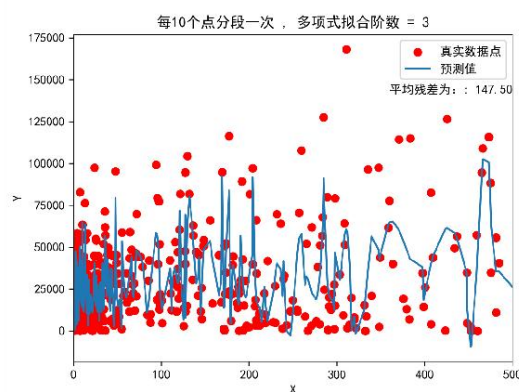
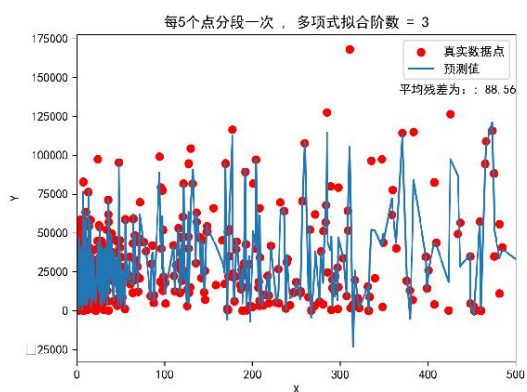


图 6.4 多项式拟合阶数为 3，不同分段点结果

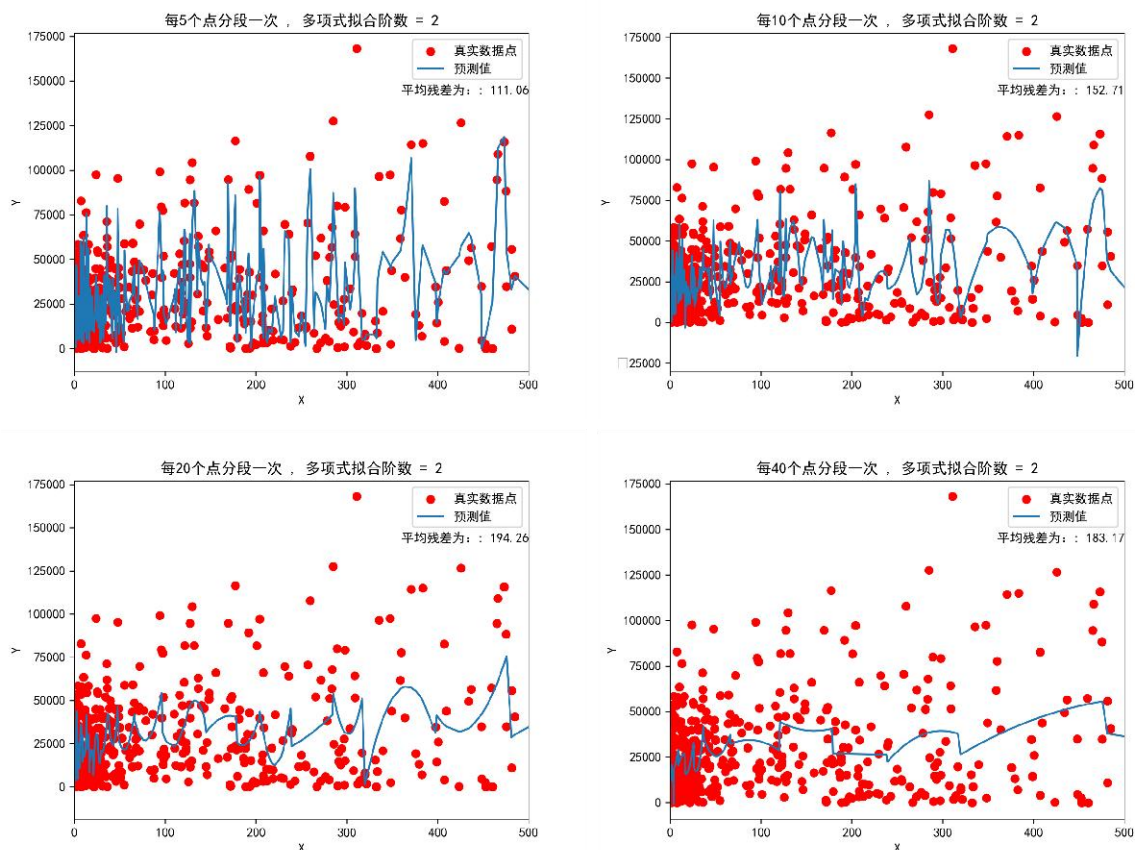


图 6.5 多项式拟合阶数为 2 时，不同分段点结果

从以上 12 个曲线拟合图中我们可以总结所有的平均残差值如下：

表 6.1 多项式拟合结果总结

	多项式拟合阶数=2	多项式拟合阶数=3	多项式拟合阶数=4
分段点数=5	111.06	88.56	18.86
分段点数=10	152.71	147.50	79.23
分段点数=20	194.26	174.75	196.04
分段点数=40	183.17	196.78	191.98

可以发现当分段点数为 5，同时多项式拟合阶数为 4 的时候平均残差值最小，平均残差值仅为 18.86。

然后我们再将所有人的信息输入进行拟合预测后与真实值的结果残差如下可得结果为：（这里展示部分预测得分）

表 6.2 残差结果

	残差（全体）		残差（全体）
sub001	-20551.4	sub051	9702.6
sub002	-3705.8	sub052	-11394.16667
sub003	-14211.33333	sub053	-3483
sub004	177.5	sub054	-1055
sub005	3672.666667	sub055	-6558.666667
sub006	-13939.2	sub056	2574.75
sub007	4108.571429	sub057	15982.66667
sub008	-92.5	sub058	1836.333333
sub009	-1539.333333	sub059	3853.666667

sub010	3754. 4	sub060	-2722. 125
sub011	12436	sub061	-25735. 33333
sub012	1170. 285714	sub062	4518
sub013	5994	sub063	-11435. 25
sub014	2544. 666667	sub064	19493. 5
sub015	-23317. 25	sub065	-14864
sub016	-18329	sub066	-7145. 2
sub017	5161. 25	sub067	8826. 666667
sub018	-1724. 75	sub068	-132. 375
sub019	-62. 5	sub069	-2080. 6
sub020	10172. 4	sub070	12671
sub021	11299	sub071	-4285
sub022	8721. 333333	sub072	7923. 666667
sub023	-3087	sub073	13069. 66667
sub024	12024	sub074	-14789. 5
sub025	14240	sub075	-6401. 8
sub026	-9615. 75	sub076	10788. 6
sub027	1546. 75	sub077	-959. 8
sub028	-10437. 6	sub078	21378. 66667
sub029	-4628. 75	sub079	6392. 8
sub030	-12308. 6	sub080	11274. 66667
sub031	-8872. 333333	sub081	-5271. 8
sub032	7548. 333333	sub082	-1092. 333333
sub033	9363. 666667	sub083	-65
sub034	-6170. 5	sub084	237. 8
sub035	-3041. 666667	sub085	-12753. 16667
sub036	7610. 6	sub086	13074. 25
sub037	-6284. 2	sub087	6575. 6
sub038	5333. 4	sub088	2229. 333333
sub039	-2157. 142857	sub089	757. 75
sub040	-10637. 16667	sub090	9477. 5
sub041	12554. 25	sub091	7543. 25
sub042	10616	sub092	7161. 5
sub043	-3716	sub093	15645
sub044	-10104	sub094	-794. 4
sub045	2452. 75	sub095	-47828. 8
sub046	-7996. 333333	sub096	24327. 16667
sub047	10525. 5	sub097	2656
sub048	-4364. 25	sub098	-1983. 4
sub049	9276. 5	sub099	6898
sub050	13648	sub100	7922. 666667

6.2 第二小问模型建立与求解

6.2.1 第二小问求解思路

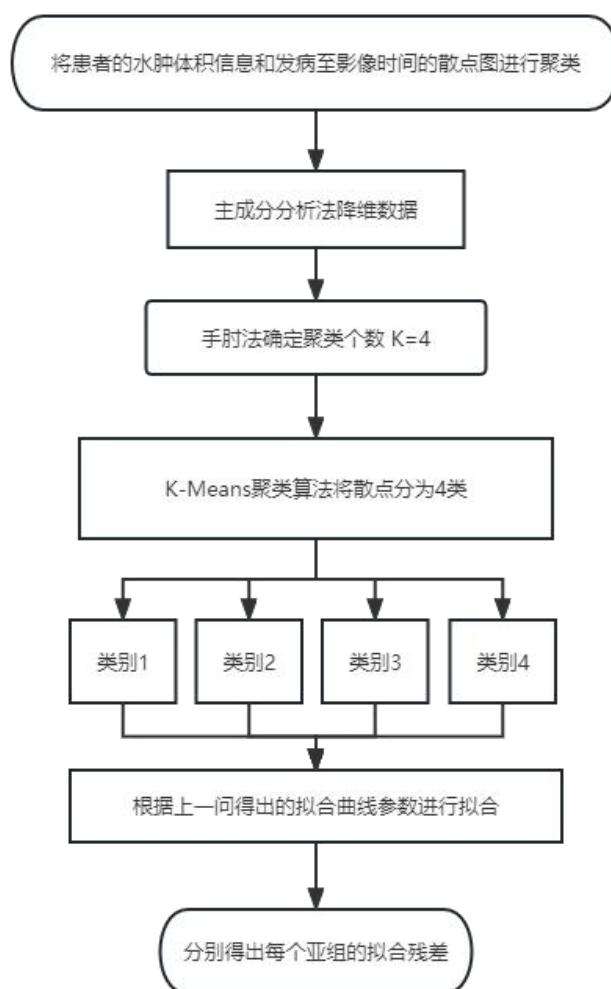


图 6.6 将患者分类并分析各类别中水肿体积和时间进展的关系主要思路

6.2.2 第二小问模型建立

(1) 主成分分析法

主成分分析（PCA）是一种常用的数据分析方法，通过线性变换将原始数据变换为一组线性无关的表示，用于提取数据的主要特征分量并降低数据的维度。PCA 将高维数据降维到较低维度，以保留尽可能多的数据信息。^[12]

PCA 的核心思想是通过线性变换，将原始数据映射到一个新的坐标系下，使得在新坐标系下数据的方差最大化。这相当于对原始数据进行了一种旋转和投影，以便在新坐标系中，第一个主成分（主轴）上的方差最大，第二个主成分上的方差次之，以此类推。通过选择保留的主成分数量，可以控制降维后的维度。这里使用 SVD 方法进行 PCA 降维，假定有 $p \times n$ 维数据样本 X ，共有 p 个样本，每行是 n 维， $p \times n$ 实矩阵可以分解为：

$$X = U \Sigma V^T$$

这里，正交阵 U 的维数是 $p \times n$ ，正交阵 V 的维数是 $n \times n$ （正交阵满足： $UU^T=VV^T=I$ ）， Σ 是 $n \times n$ 的对角阵。接下来，将 Σ 分割成 r 列，记作 Σ_r ；利用 U 和 V 便能够得到降维数据点 Y_r ：

$$Y_r = U\Sigma_r$$

(2) 手肘法

手肘法是一种用于确定 K-Means 聚类算法中最佳簇数量 K 的常用方法。它基于簇内平方和损失函数（ $inertia_$ ）随着 K 值的增加而减小的趋势来帮助选择 K 的值。

手肘法的步骤如下：

对于不同的 K 值（通常从 2 开始增加），分别运行 K-Means 算法，计算每个 K 值对应的簇内平方和损失函数。将每个 K 值对应的簇内平方和损失函数绘制成折线图。

观察折线图，找到一个“拐点”，即损失函数开始下降速度明显变缓的地方。

这个拐点所对应的 K 值即为最佳的簇数量。

手肘法的核心思想是，随着簇的数量增加，簇内平方和损失函数逐渐减小，因为每个数据点会更接近其所属的聚类中心。然而，随着 K 值的继续增加，簇内数据点之间的距离减小的速度会变得较慢，导致损失函数下降幅度减小，形成一个拐点。

需要注意的是，手肘法是一种启发式方法，不能保证一定能找到最佳的 K 值。在实际应用中，可能需要结合领域知识和实际需求来综合考虑选择最佳的 K 值。如果折线图上没有明显的手肘点，还可以考虑使用其他方法来辅助选择 K 值，如轮廓系数等。^[13]

(3) K-Means 算法

K-Means 算法是一种常用的聚类算法，用于将数据集划分为 K 个不同的簇。算法的主要步骤包括：

- 1) 随机选择 K 个数据点作为初始的聚类中心。
- 2) 将数据集中的每个数据点分配给离其最近的聚类中心，形成 K 个簇。
- 3) 对每个簇，计算其中所有数据点的均值，得到新的聚类中心。
- 4) 重复步骤 2 和 3，直到聚类中心不再发生变化或达到最大迭代次数。

K-Means 算法的优化目标是 최소화每个数据点与其所属聚类中心的距离之和，也即簇内的平方和损失函数（ $inertia_$ ）。在每次迭代中，算法尽可能地减小损失函数的值，以获得更好的聚类效果。K-Means 算法具有简单易懂和计算效率高的优点。然而，它也有一些限制，如对初始聚类中心的敏感性和对异常值的敏感性。此外， K 值的选择也是一个需要仔细考虑的问题，可以使用手肘法、轮廓系数等方法来辅助选择最佳的 K 值。需要注意的是，K-Means 算法适用于数值型数据，对于具有非数值型特征的数据，需要进行预处理或者考虑使用其他适合的聚类算法。^[14]

6.2.3 第二小问模型求解与分析

我们挑选年龄、高血压病史、卒中病史、糖尿病史、房颤史、冠心病史、吸烟史、饮酒史、脑室引流、止血治疗、降颅压治疗、降压治疗、镇静和镇痛治疗、止吐护胃、营养神经等十五个特征，先使用主成分分析法降维挑选出用来聚类的特征。

表 6.3 主成分分析检验结果

KMO 检验和 Bartlett 的检验		
KMO 值		0.526
Bartlett 球形度检验	近似卡方	294.864
	df	105
	P	0.000***

注：***、**、*分别代表 1%、5%、10%的显著性水平

上表展示了 KMO 检验和 Bartlett 球形检验的结果，用来分析是否可以进行主成分分析。若通过 KMO 检验 ($KMO > 0.6$)，说明了题项变量之间是存在相关性的，符合主成分分析要求。若通过 Bartlett 检验： $P < 0.05$ ，呈显著性，则可以进行主成分分析。KMO 检验的结果显示，KMO 的值为 0.526，同时，Bartlett 球形检验的结果显示，显著性 P 值为 0.000***，水平上呈现显著性，拒绝原假设，各变量间具有相关性，主成分分析有效。

表 6.4 主成分分析的总方差解释

总方差解释			
成分	特征根		
	特征根	方差解释率(%)	累积方差解释率(%)
1	2.071	13.81	13.81
2	1.793	11.956	25.766
3	1.62	10.803	36.569
4	1.316	8.774	45.343
5	1.262	8.416	53.759
6	1.076	7.175	60.934
7	1.015	6.764	67.698
8	0.836	5.571	73.269
9	0.758	5.052	78.321
10	0.724	4.827	83.148
11	0.636	4.238	87.386
12	0.573	3.818	91.204
13	0.57	3.802	95.005
14	0.379	2.524	97.53
15	0.371	2.47	100

表 6.5 聚类指标的差异性分析

	聚类类别（平均值±标准差）				F	P
	类别 3(n=47)	类别 2(n=45)	类别 4(n=43)	类别 1(n=25)		
年龄	55.213±3.106	67.622±3.406	81.953±4.928	43.76±5.109	566.19	0.000***
高血压病史	0.936±0.247	0.911±0.288	0.791±0.412	0.88±0.332	1.715	0.166
卒中病史	0.106±0.312	0.467±0.505	0.233±0.427	0.0±0.0	10.153	0.000***
糖尿病史	0.106±0.312	0.089±0.288	0.233±0.427	0.2±0.408	1.611	0.189
房颤史	0.0±0.0	0.044±0.208	0.116±0.324	0.0±0.0	2.991	0.033**
冠心病史	0.0±0.0	0.067±0.252	0.233±0.427	0.04±0.2	6.097	0.001***
吸烟史	0.213±0.414	0.156±0.367	0.14±0.351	0.36±0.49	1.881	0.135
饮酒史	0.128±0.337	0.089±0.288	0.07±0.258	0.32±0.476	3.448	0.018**
脑室引流	0.128±0.337	0.022±0.149	0.047±0.213	0.04±0.2	1.697	0.170
止血治疗	0.787±0.414	0.622±0.49	0.605±0.495	0.84±0.374	2.448	0.066*
降颅压治疗	0.766±0.428	0.6±0.495	0.698±0.465	0.92±0.277	3.045	0.031**
降压治疗	0.851±0.36	0.844±0.367	0.907±0.294	0.96±0.2	0.908	0.439
镇静、镇痛治疗	0.66±0.479	0.533±0.505	0.721±0.454	0.68±0.476	1.232	0.300
止吐护胃	0.957±0.204	0.889±0.318	1.0±0.0	0.96±0.2	1.994	0.117
营养神经	0.979±0.146	0.889±0.318	0.953±0.213	1.0±0.0	1.914	0.129

注：***、**、*分别代表 1%、5%、10%的显著性水平

上表展示了定量字段差异性分析的结果，包括均值±标准差的结果、F 检验结果、显著性 P 值。分析每个分析项的 P 值是否显著($P < 0.05$)。若呈显著性，拒绝原假设，说明两组数据之间存在显著性差异，可以根据均值±标准差的方式对差异进行分析，反之则表明数据不呈现差异性。

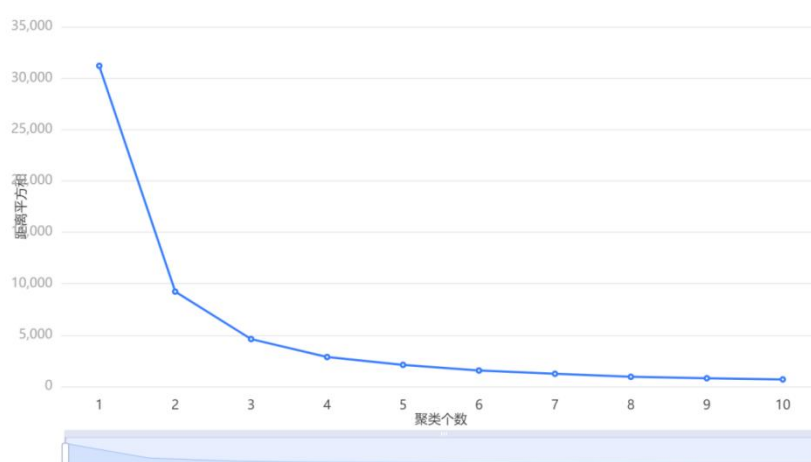


图 6.7 手肘法分析聚类个数

然后我们使用手肘法进行聚类个数的确定，可以看出我们分成 4 类亚组或者 5 类亚组最为合适，这里我们取较小的 4，更能代表分类的一个代表性，然后我们根据我们选取的 4 来使用 K-Means 算法对我们的数据进行分类。

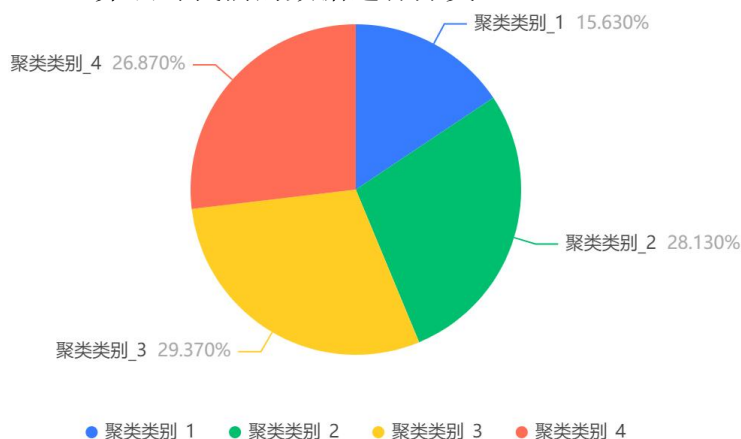


图 6.8 K-Means 聚类结果占比描述性统计 (1)

表 6.6 K-Means 聚类结果频数描述性统计 (2)

聚类类别	频数	百分比%
类别 1	25	15.625
类别 2	45	28.125
类别 3	47	29.375
类别 4	43	26.875
合计	160	100.0

聚类分析的结果显示，聚类结果共分为 4 类，类别 1 的频数为 25，所占百分比为 15.625%；类别 2 的频数为 45，所占百分比为 28.125%；类别 3 的频数为 47，所占百分比为 29.375%；类别 4 的频数为 43，所占百分比为 26.875%。

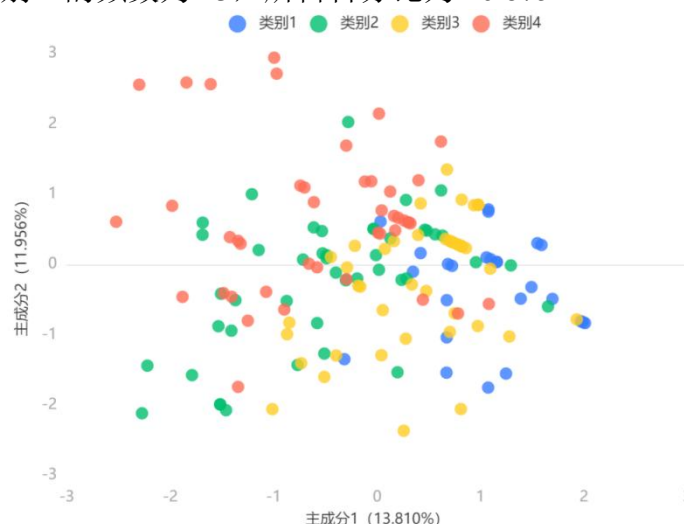


图 6.9 K-Means 聚类结果二维展示图

表 6.7 K-Means 指标

轮廓系数	DBI	CH
0.53	0.582	515.511

轮廓系数：对于一个样本集合，它的轮廓系数是所有样本轮廓系数的平均值。轮廓系数的取值范围是 $[-1,1]$ ，同类别样本距离越相近不同类别样本距离越远，分数越高，聚类效果越好。DBI (Davies-bouldin)：该指标用来衡量任意两个簇的簇内距离之后与簇间距离之比。该指标越小表示聚类效果越好。CH(Calinski-Harbasz Score)：通过计算类内各点与类中心的距离平方和来度量类内的紧密度（分母），通过计算类间中心点与数据集中心点距离平方和来度量数据集的分离度（分子），CH 指标由分离度与紧密度的比值得到，CH 越大表示聚类效果越好。

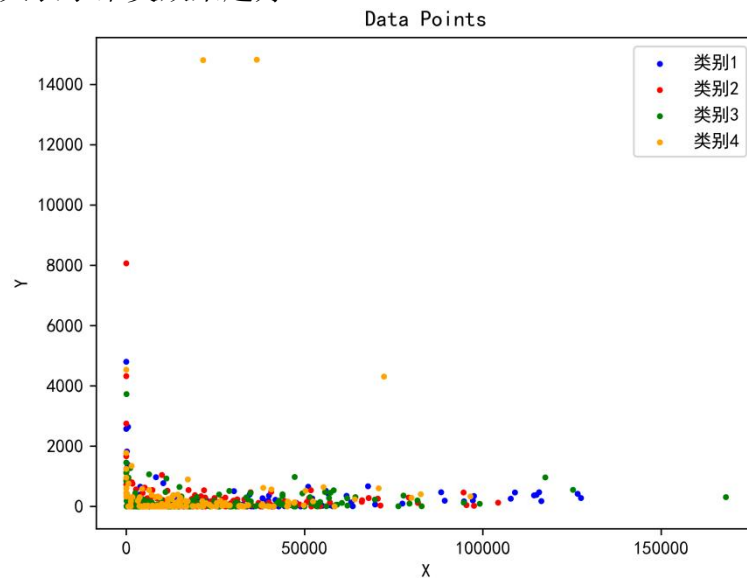


图 6.10 所有类别的散点图

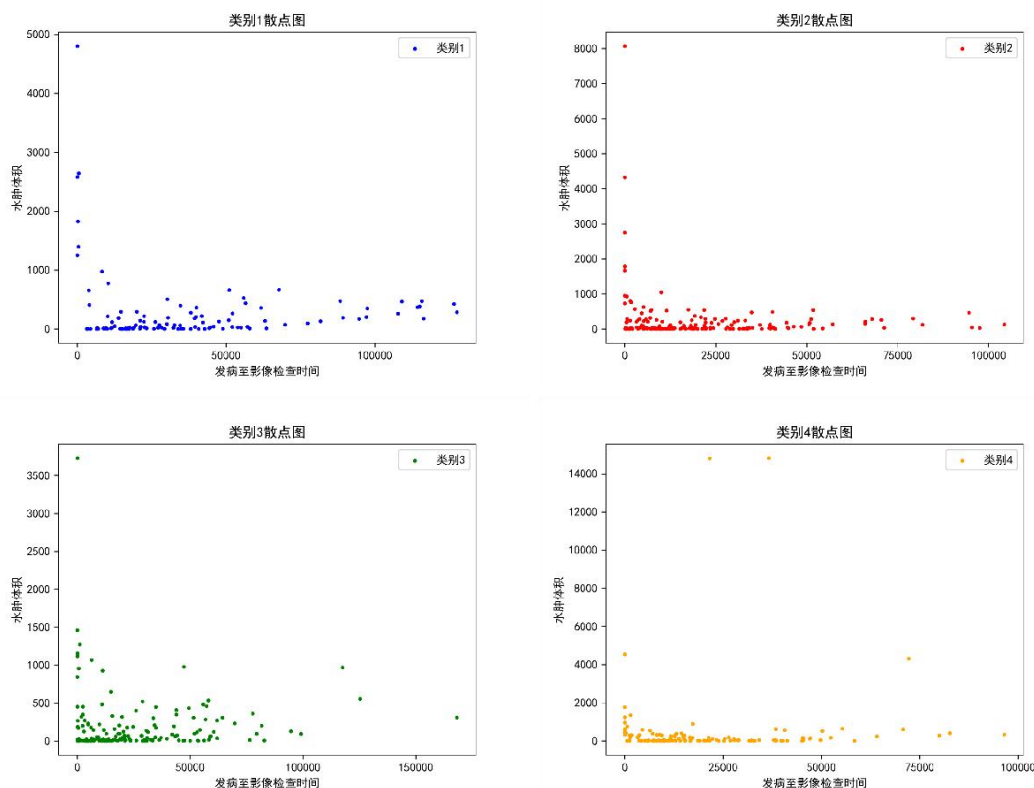


图 6.11 类别 1、类别 2、类别 3、类别 4 各自散点图

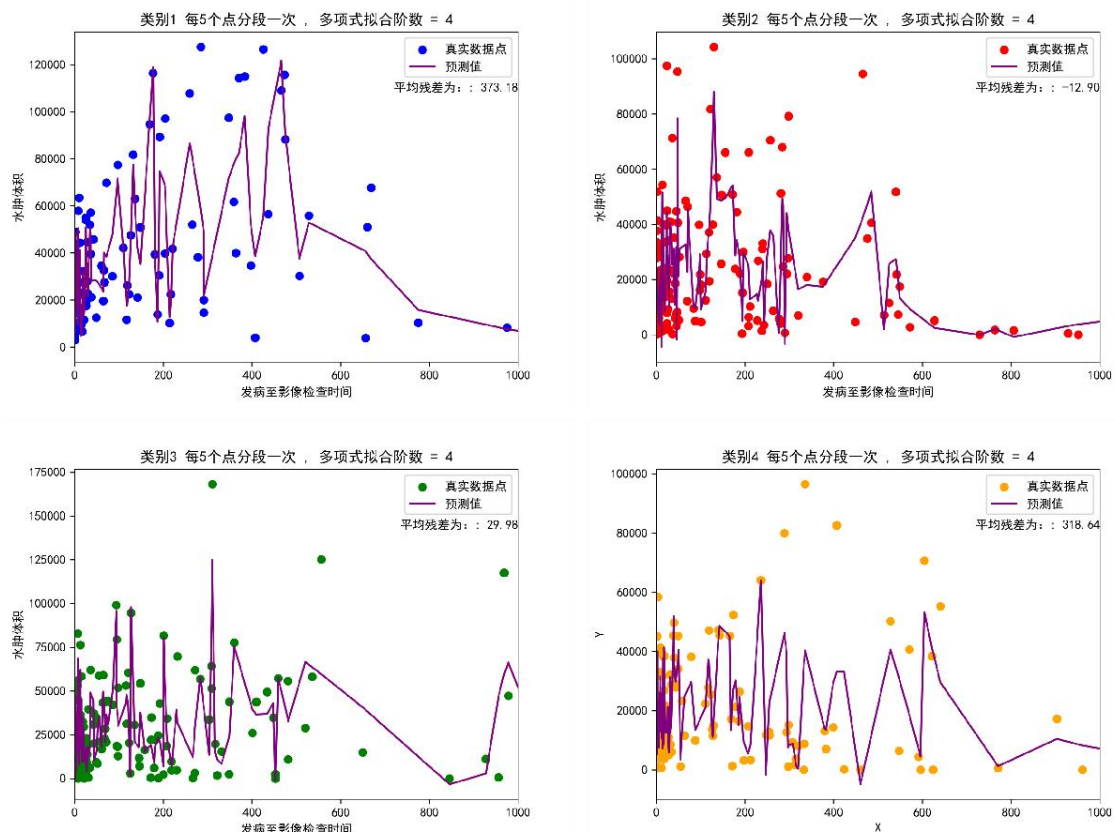


图 6.12 类别 1、类别 2、类别 3、类别 4 各自拟合图（取 X 轴前 1000）

然后我们再将所有人的信息输入进行拟合预测后与真实值的结果残差如下可得结果为：（这里展示部分预测得分）

表 6.8 分组之后分别拟合的残差结果

	残差（亚组）	亚组		残差（亚组）	亚组
sub001	-20802	类别 1	sub051	4977.6	类别 4
sub002	-1372	类别 3	sub052	-7109.666667	类别 4
sub003	-7912.333333	类别 4	sub053	8861.25	类别 1
sub004	104.75	类别 2	sub054	2940.75	类别 3
sub005	-2887.333333	类别 3	sub055	-3803	类别 4
sub006	-18529	类别 2	sub056	-649	类别 2
sub007	3284.142857	类别 3	sub057	-529	类别 4
sub008	-14411.25	类别 3	sub058	-3564.666667	类别 3
sub009	-9291.333333	类别 2	sub059	9667.666667	类别 2
sub010	2992.2	类别 2	sub060	-631.25	类别 2
sub011	13349.66667	类别 4	sub061	-13421.16667	类别 1
sub012	-1152.714286	类别 3	sub062	-1410	类别 3
sub013	180.6	类别 2	sub063	-10124.25	类别 3
sub014	2871.666667	类别 2	sub064	10716.33333	类别 4
sub015	-20949	类别 4	sub065	-9584.25	类别 4
sub016	1394.333333	类别 3	sub066	-3529.8	类别 4

sub017	4714.5	类别 2	sub067	6422.333333	类别 4
sub018	-3315.75	类别 4	sub068	2937.125	类别 1
sub019	3264.666667	类别 2	sub069	-7214.2	类别 2
sub020	7585.4	类别 2	sub070	1345.666667	类别 4
sub021	4574	类别 3	sub071	-10490.16667	类别 2
sub022	10613	类别 4	sub072	3808	类别 4
sub023	6036.428571	类别 1	sub073	6724.666667	类别 3
sub024	523.25	类别 4	sub074	-9115.666667	类别 1
sub025	8477.666667	类别 1	sub075	276.8	类别 3
sub026	-5262.75	类别 3	sub076	3122	类别 4
sub027	4026.25	类别 2	sub077	965.8	类别 4
sub028	-9932.4	类别 4	sub078	14811	类别 3
sub029	-13932.75	类别 3	sub079	4088.4	类别 3
sub030	-22340.4	类别 2	sub080	-386	类别 2
sub031	-2874.666667	类别 1	sub081	-14938.8	类别 4
sub032	9059	类别 3	sub082	1505.666667	类别 1
sub033	5909.5	类别 3	sub083	5365.666667	类别 1
sub034	-9503	类别 3	sub084	10240.2	类别 1
sub035	1775.166667	类别 4	sub085	-9038.166667	类别 3
sub036	-5286.6	类别 2	sub086	11738	类别 3
sub037	-7952.4	类别 2	sub087	15829.2	类别 2
sub038	8158.4	类别 3	sub088	5793	类别 2
sub039	-14251.71429	类别 1	sub089	-2694.75	类别 2
sub040	-1562.666667	类别 1	sub090	2323.5	类别 3
sub041	9668.5	类别 2	sub091	6516.75	类别 3
sub042	2578.25	类别 4	sub092	-3010.75	类别 4
sub043	-1320.25	类别 4	sub093	9076.6	类别 3
sub044	-8809.333333	类别 3	sub094	3041.8	类别 2
sub045	13319	类别 1	sub095	-24261.4	类别 3
sub046	-3219.666667	类别 2	sub096	12563.16667	类别 4
sub047	17263	类别 3	sub097	8024	类别 4
sub048	-5095.25	类别 3	sub098	4977.6	类别 4
sub049	5713.5	类别 3	sub099	-7109.666667	类别 4
sub050	14741	类别 1	sub100	8861.25	类别 1

6.3 第三小问模型建立与求解

6.3.1 第三小问求解思路

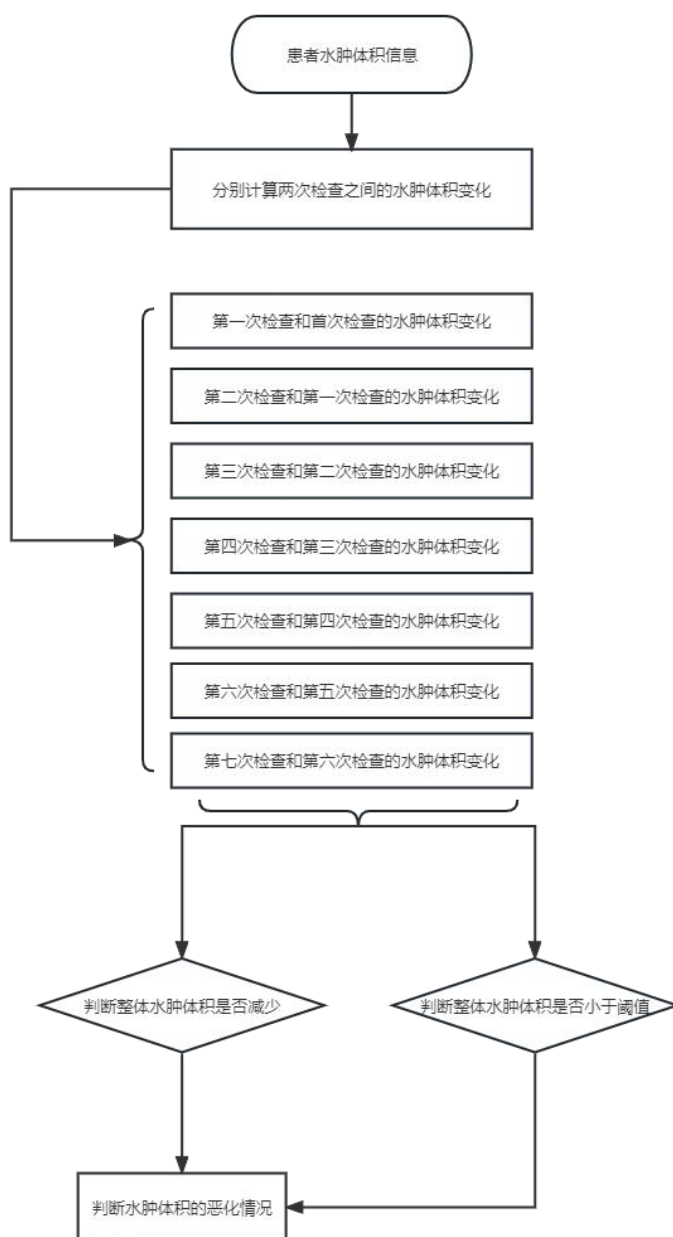


图 6.13 分析不同治疗方法对水肿体积进展模式的影响思路

6.3.2 第三小问模型建立

(1) 灰色关联算法

灰色关联算法（GRA）是一种用于分析多个变量之间关联程度的方法。该方法最初由中国学者韩国权于 1988 年提出，其基本思想是通过计算变量之间的灰色关联度来评估它们之间的关联性。

灰色关联度是一个介于 0 和 1 之间的数值，表示变量之间的相似程度。数值越接近 1，表示变量之间的关联程度越高；数值越接近 0，表示变量之间的关联程度越低。

灰色关联算法的步骤如下：数据预处理：将原始数据标准化处理，使得各个变量具有相同的量纲和量级。构建关联度矩阵：将标准化后的数据转换为关联度矩阵，其中每

个元素代表第 i 个变量与第 j 个变量之间的关联度。确定参考序列：选择一个参考序列，该序列作为其他变量与之比较的标准。可以根据实际问题选择最为重要或具有特殊意义的变量作为参考序列。计算灰色关联度：对于每个变量，将其与参考序列的关联度进行计算，并得到灰色关联度序列。排序和评价：根据计算得到的灰色关联度序列，对各个变量进行排序和评价，以确定它们之间的关联程度。灰色关联算法适用于多个变量之间的关联性分析，并可用于辅助决策、预测和优化等问题。它具有计算简单、不受数据分布、尺度和数量限制等优点，

6.3.3 第三小问模型求解与分析

我们分别统计了患者每次检查时，水肿体积相对于上次检查时水肿体积的大小变化，可以看的出来绝大部分的患者随着时间的推移，虽然水肿体积整体都在不断地增加，但是我们可以看的出来，随着检查次数的不断增加，对患者的病情更加掌握，患者水肿体积的减少程度在不断增加，说明目前有的治疗方法在一定程度上对于水肿体积的影响是存在的，但是这还是要靠我们的具体分析才能下定论。在条形图中，绿色代表两次检查中水肿体积增加，红色代表两次检查中水肿体积减少，患者脑部的水肿体积我们可以感受到，随着检查次数的增加，很明显红色区域的面积在不断地增大，说明随着时间流逝，大部分患者脑内的水肿体积扩张是得到了及时的控制的，证明了目前对水肿体积的治疗方法是行之有效的。检查次数不断增加的同时，患者数量也随着时间的增加而不断变少，这其中大部分的患者都是因为水肿体积降低至零才停止了继续检查。

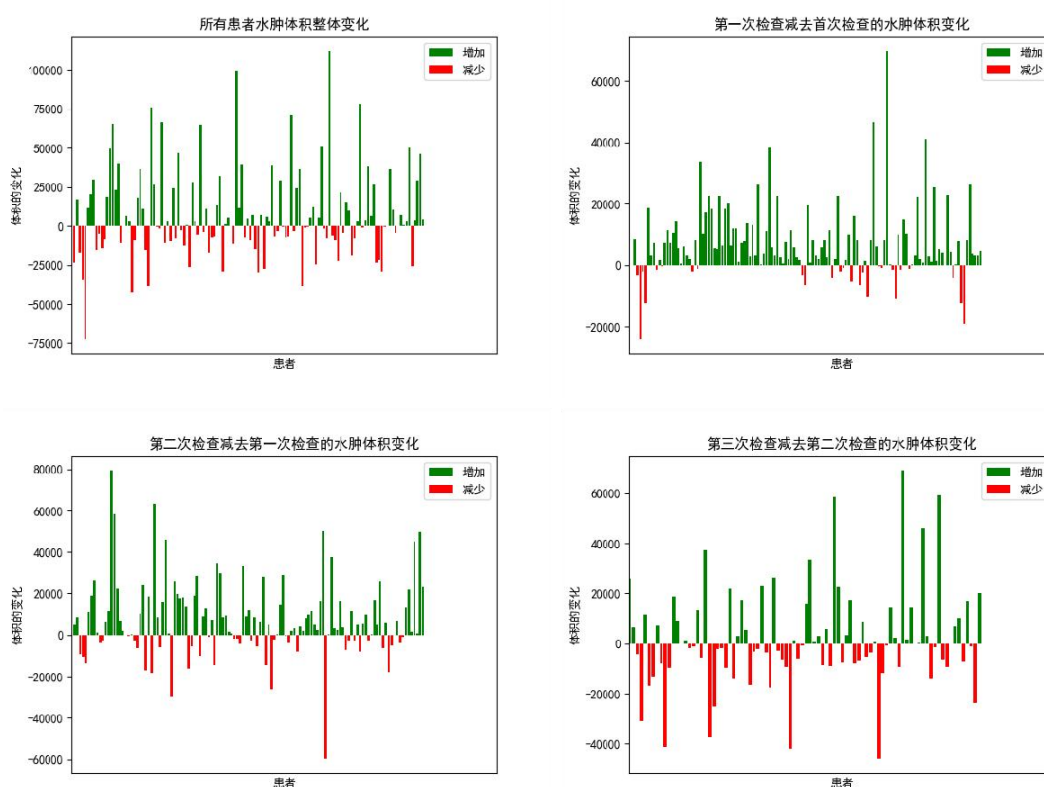


图 6.14 水肿体积变化图（1）

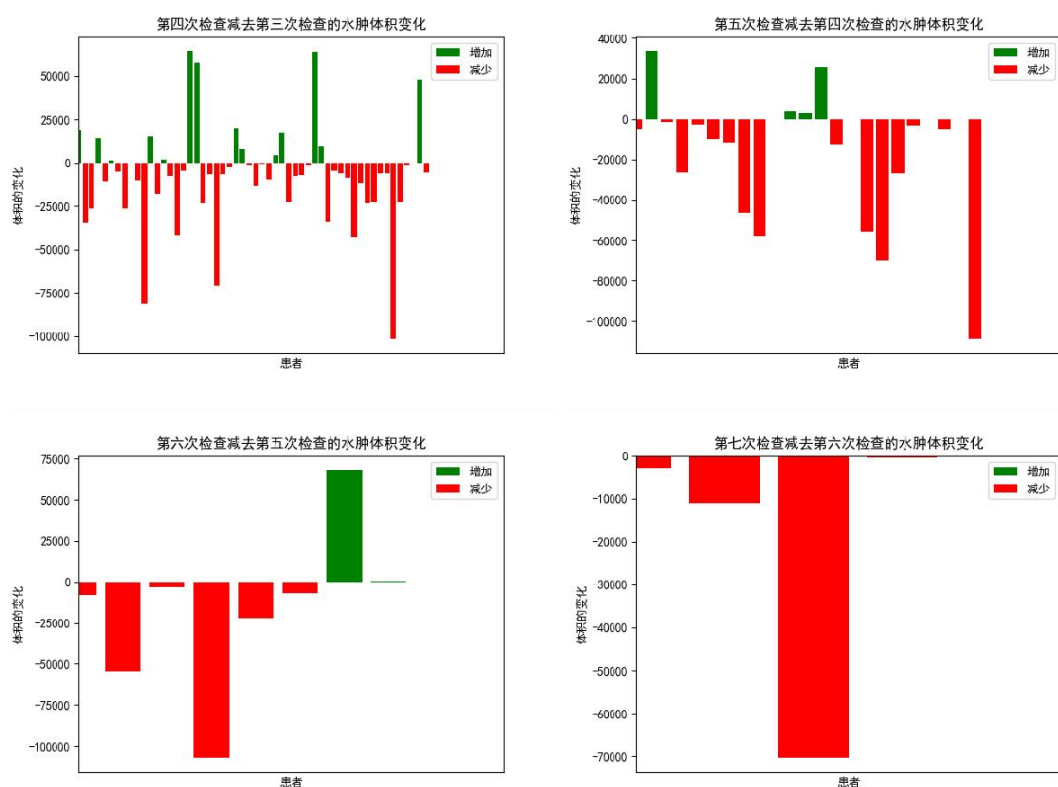


图 6.15 水肿体积变化图（2）

表 6.9 前 10 项灰色关联度展示

患者	脑室引流	止血治疗	降颅压治疗	降压治疗	镇静、镇痛治疗	止吐护胃	营养神经
1	1	0.86	0.86	0.88	0.85	0.88	0.88
2	0.84	0.97	0.97	0.95	0.84	0.94	0.94
3	1	0.86	0.86	0.88	0.85	0.88	0.88
4	0.84	0.97	0.97	0.95	0.84	0.84	0.84
5	0.84	0.97	0.97	0.84	0.84	0.94	0.94
6	0.36	0.84	0.84	0.95	0.84	0.94	0.94
7	1	1	0.86	0.88	1	0.88	0.88
8	1.00	0.86	1.00	0.88	1.00	1.00	0.88
9	1	0.86	0.86	0.88	1	0.88	0.88
10	0.84	0.97	0.84	0.95	0.98	0.94	0.94

列举了所有患者信息中的前 10 项关联度，从上表可知，针对 7 个评价项（脑室引流、止血治疗、降颅压治疗、降压治疗、镇静、镇痛治疗、止吐护胃、营养神经）以及 129 项数据进行灰色关联度分析，并且以恶化作为“参考值”(母序列)，研究 7 个评价项(脑室引流、止血治疗、降颅压治疗、降压治疗、镇静、镇痛治疗、止吐护胃、营养神经与恶化的关联关系（关联度），并基于关联度提供分析参考，使用灰色关联度分析时，分辨系数取 0.5，结合关联系数计算公式计算出关联系数值，并根据关联系数值，然后计算出关联度值用于评价判断。

分辨系数 $\rho \in (0, \infty)$, ρ 越小, 分辨力越大, 一般 ρ 的取值区间为 $(0, 1)$, 具体取值可视情况而定。当 $\rho \leq 0.5463$ 时, 分辨力最好, 通常取 $\rho = 0.5$ 。

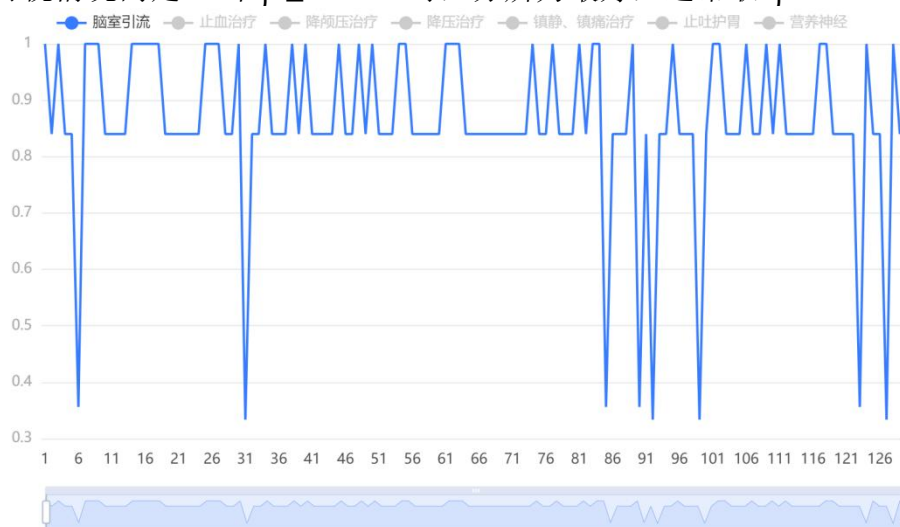


图 6.16 脑室引流关联图

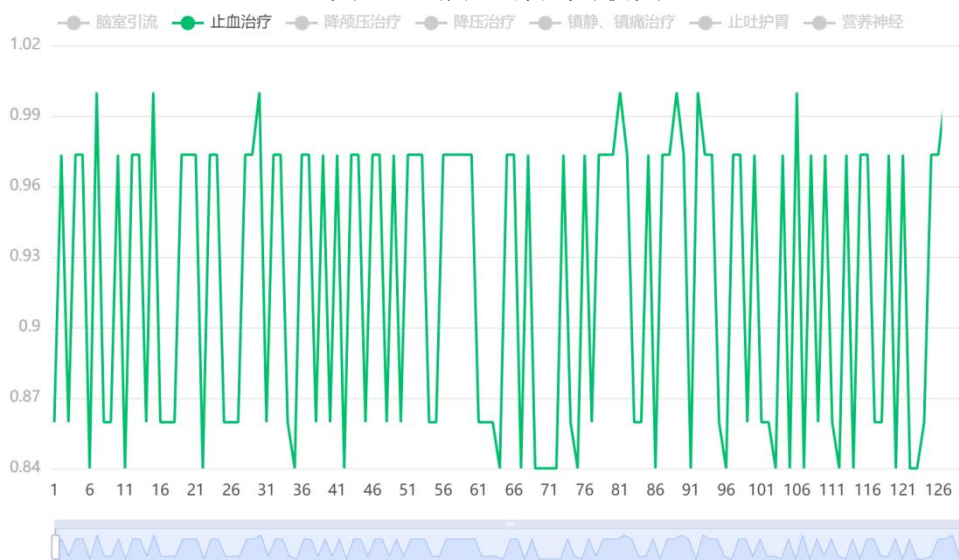


图 6.17 止血治疗关联图

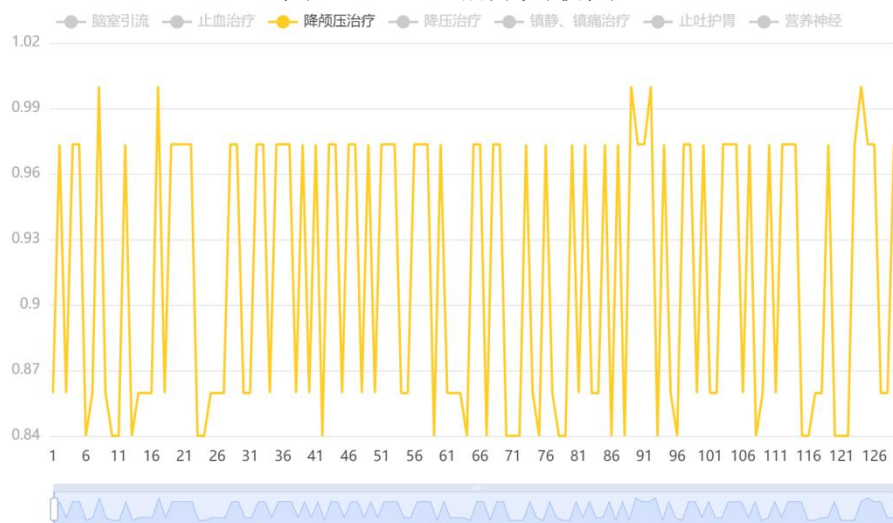


图 6.18 降颅压关联图



图 6.19 降压治疗关联图

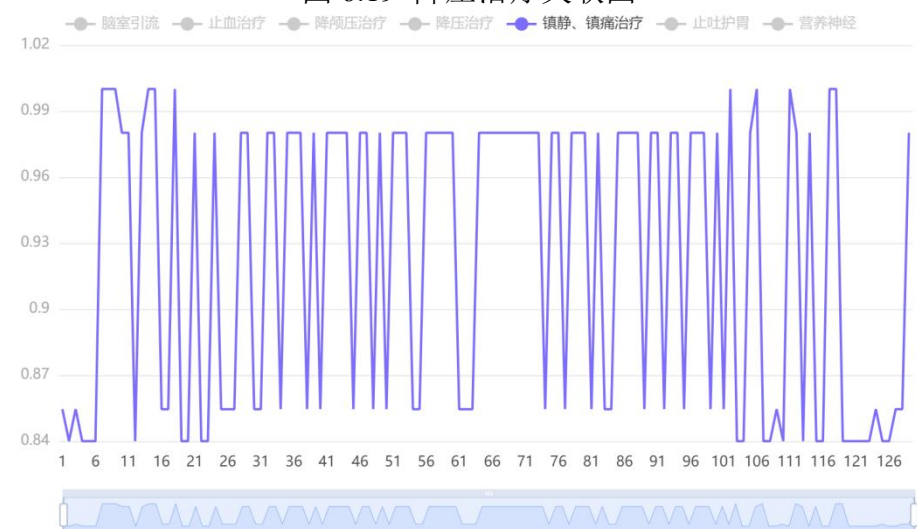


图 6.20 镇静、镇痛关联图

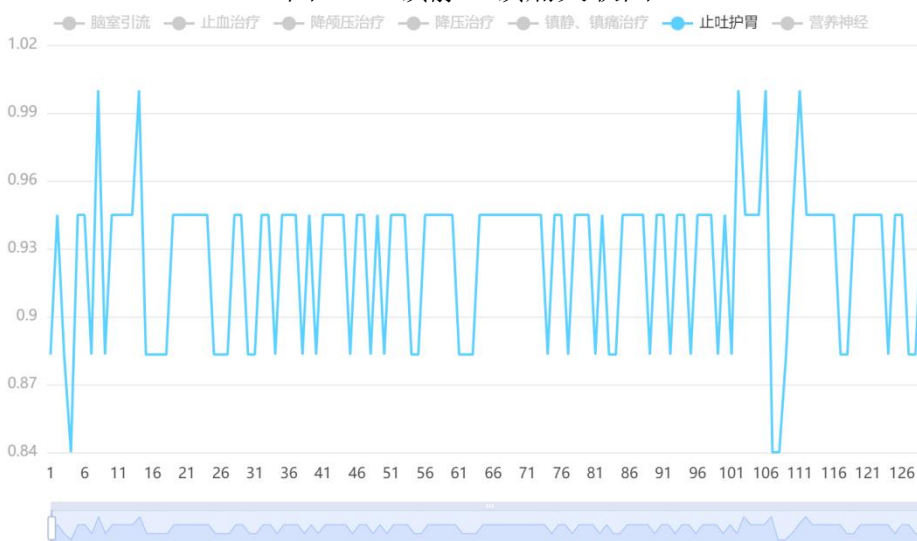


图 6.21 止吐护胃关联图

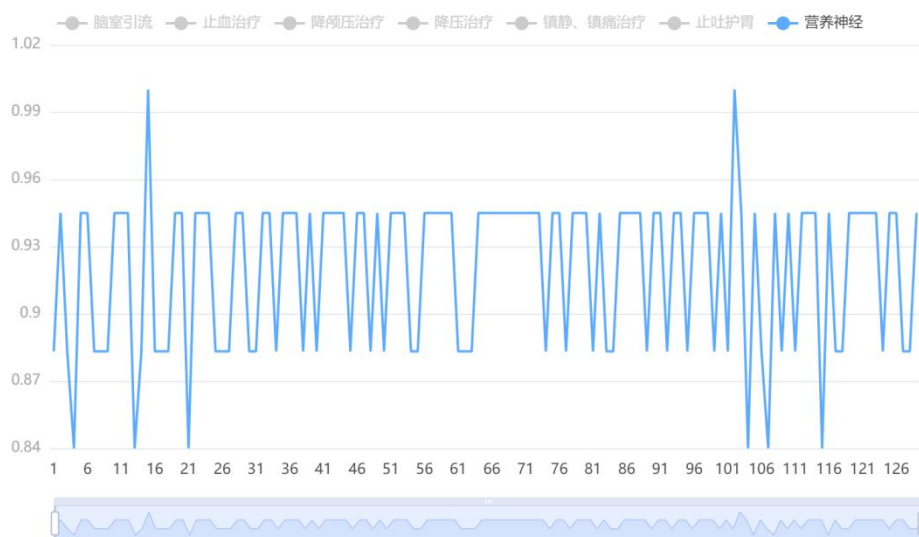


图 6.22 营养神经关联图

表 6.10 灰色关联分析治疗方式关联系数

关联度结果		
评价项	关联度	排名
止吐护胃	0.926	1
镇静、镇痛治疗	0.923	2
营养神经	0.92	3
止血治疗	0.919	4
降压治疗	0.916	5
降颅压治疗	0.912	6
脑室引流	0.86	7

结合上述关联系数结果进行加权处理，最终得出关联度值，使用关联度值针对 7 个评价对象进行评价排序；关联度值介于 0~1 之间，该值越大代表其与“参考值”（母序列）之间的相关性越强，也即意味着其评价越高。从上表可以看出：针对本次 7 个评价项，止血治疗评价最高(关联度为：0.926)，其次是镇静、镇痛治疗(关联度为：0.92)。

不同治疗对水肿进展模式的影响大小为：

止吐护胃 > 镇静、镇痛治疗 > 营养神经 > 止血治疗 > 降压治疗 > 镇静、降颅压治疗 > 脑室引流

6.4 第四小问模型建立与求解

6.4.1 第四小问求解思路

根据任务要求，需要根据表 1 “患者列表及临床信息”的 100 名患者数据中的治疗方法以及表 2 “患者影像信息血肿及水肿的体积及位置”中的血肿体积和水肿体积，建立模型分析这些变量间的关联关系。本问题求解思路流程图如图 6.23 所示。

首先，需要对数据进行预处理。处理数据中存在的缺失值和异常之后，计算每名患者的平均血肿与水肿体积。

其次，根据处理好的数据进行分类，完成数据探索，通过绘制是否进行过不同治疗方法的患者的血肿与水肿大小条形图，来初步探索自变量和因变量之间的关系。

之后，根据初步探索明确的方向，建立多元线性回归模型，以便更好地发掘自变量（是否完成某种治疗方法）对于因变量（平均血肿与水肿体积）之间的相关关系。

最后，根据建立的模型进行诊断和显著性分析，探究各变量间的关联关系，最终得到结论。

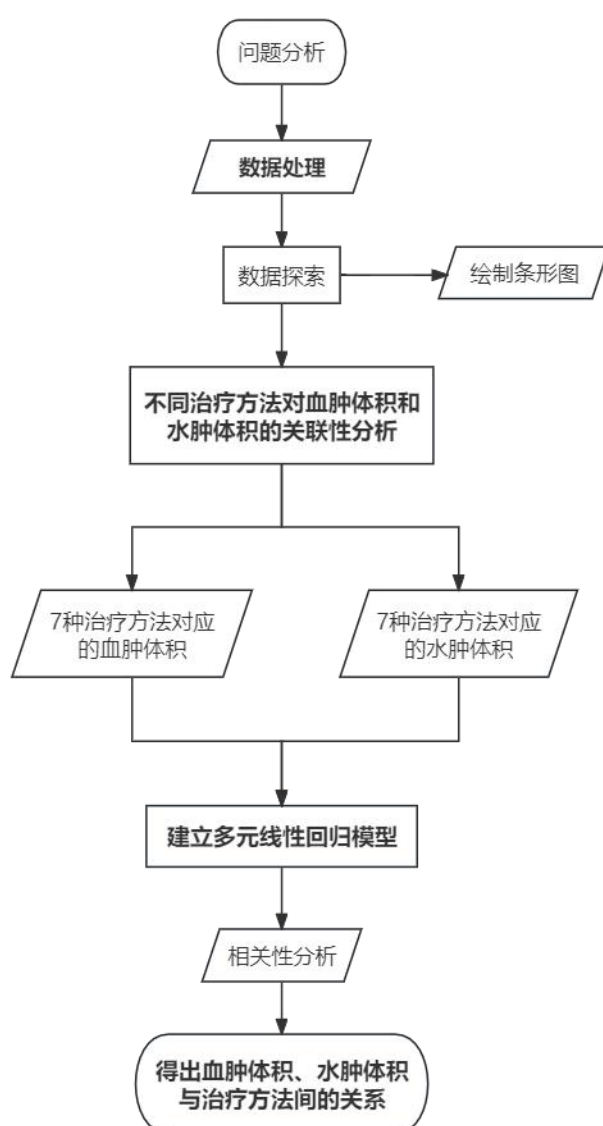


图 6.23 第四小问问题分析思路流程图

6.4.2 第四小问模型建立

1) 问题分类

根据题目所述任务与现实意义，结合患者数据可以将问题分解为两个子问题;①治疗方法对于患者血肿体积的关联性分析；②治疗方法对于患者水肿体积的关联性分析。

2) 治疗方法与血肿体积的关系模型

①数据说明

治疗方法定义为：脑室引流、止血治疗、降颅压治疗、降压治疗、镇静镇痛治疗、止吐养胃和营养神经

血肿体积定义为：每一名患者多次随访时由 CT 扫描获取的脑内血肿平均体积

②数据探索

Step 1: 合并两张表 1 和表 2 数据表，计算每一名患者多次随访时的脑内血肿的平均体积。

Step 2: 选择因变量为血肿体积，自变量为治疗方法。

Step 3: 计算不同治疗方法的血肿体积，并绘制条形图。

Step 4: 拟合多元线性回归模型并进行检验。

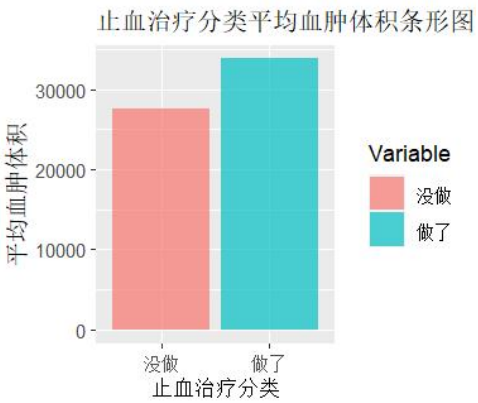
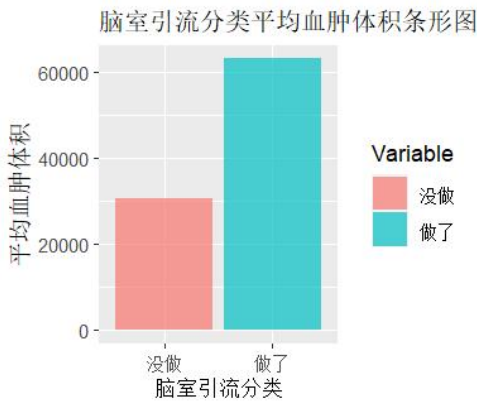
Step 5: 根据模型的结果解释治疗方法与血肿体积之间的关系。

③分析方法

首先，在计算每一名患者多次随访时的脑内血肿的平均体积后，计算不同治疗方法的血肿体积，得到平均血肿体积分布表 6.12。并绘制对应的条形图，如图 6.24。

表 6.12 不同治疗方法的平均血肿体积分布表

治疗方法	未完成患者的平均血肿体积	已完成患者的平均血肿体积
脑室引流	30551.90	63061.04
止血治疗	27589.60	33808.39
降颅压治疗	21728.72	35904.67
降压治疗	25772.10	33087.69
镇静、镇痛治疗	32954.08	32422.74
止吐养胃	29854.08	32584.35
营养神经	22251.29	32929.58



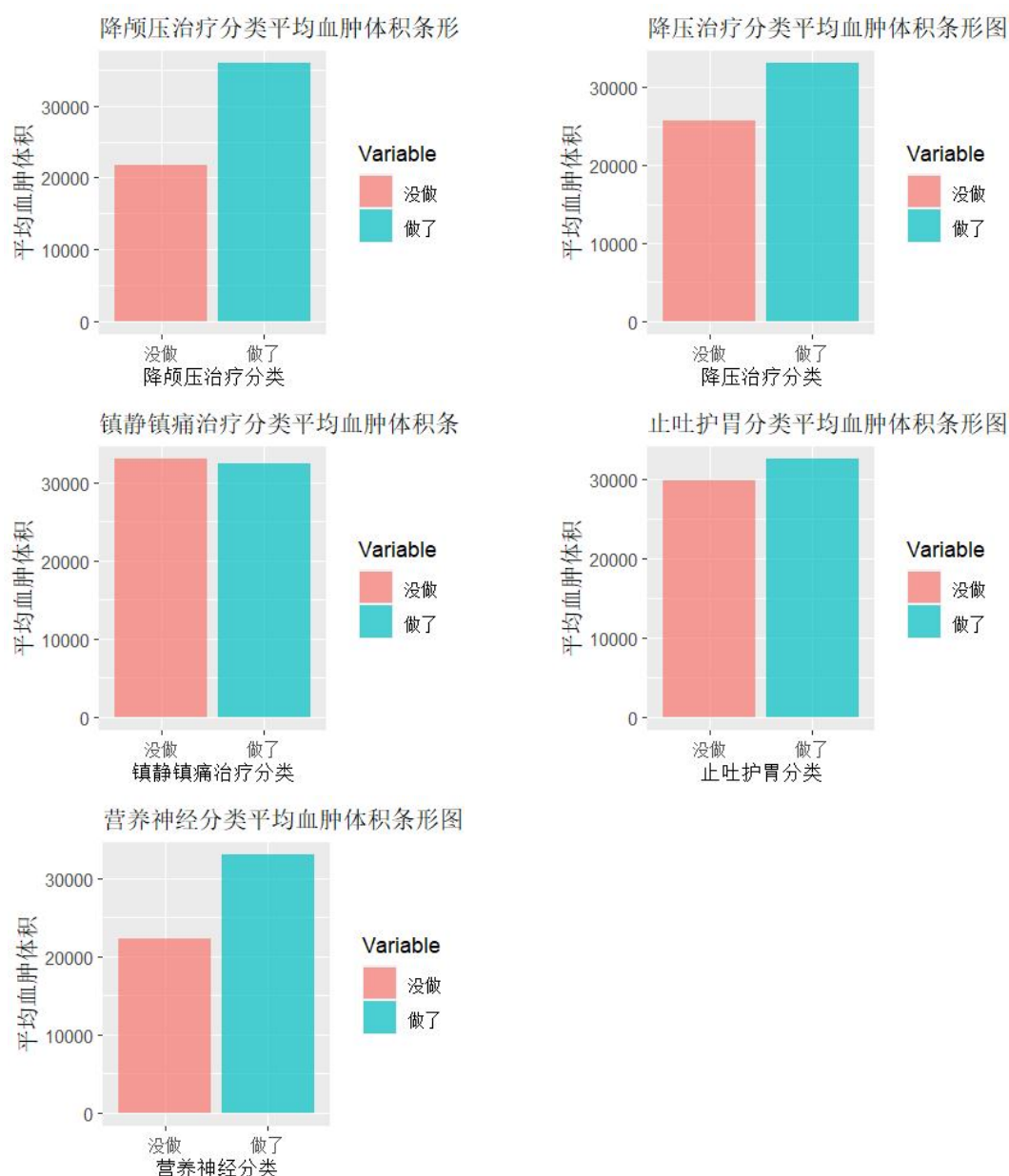


图 6.24 不同治疗方法的平均血肿体积分布图

根据不同治疗方法的平均血肿体积分布表和分布图，可以发现对于绝大部分治疗方法而言，完成了治疗的患者的血肿体积更大，故可进一步推断出完成治疗方法对患者脑内血肿体积增加呈正相关关系。

为了精确地确定这种猜想，使用多元线性回归模型来拟合这些变量，其中自变量为 7 种治疗方法，响应变量为血肿体积。

在多元线性回归模型中，假设有一个因变量（也称为响应变量） Y 和多个自变量（也称为解释变量或预测变量） X_1 、 X_2 、 X_3 、...、 X_p 。模型的基本形式如下：

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \dots + \beta_p X_p + \varepsilon$$

其中： Y 是因变量（待预测或解释的变量）， X_1 、 X_2 、 X_3 、...、 X_p 是自变量（用于解释 Y 的变量）。 β_0 、 β_1 、 β_2 、 β_3 、...、 β_p 是回归系数，表示自变量对因变量的影响程度。 ε 是误差项，表示模型的随机误差。

使用 R 包中的函数 `lm()`，可以建立多元线性回归模型。最后根据回归模型的结果和方差分析确定显著水平即可。

2) 治疗方法与水肿体积的关系模型

①数据说明

治疗方法定义为：脑室引流、止血治疗、降颅压治疗、降压治疗、镇静镇痛治疗、止吐养胃和营养神经

水肿体积定义为：每一名患者多次随访时由 CT 扫描获取的脑内水肿平均体积

②数据探索

Step 1: 合并两张表 1 和表 2 数据表，计算每一名患者多次随访时的脑内水的平均体积。

Step 2: 选择因变量为水肿体积，自变量为治疗方法。

Step 3: 计算不同治疗方法的血肿体积，并绘制条形图。

Step 4: 拟合多元线性回归模型并进行检验。

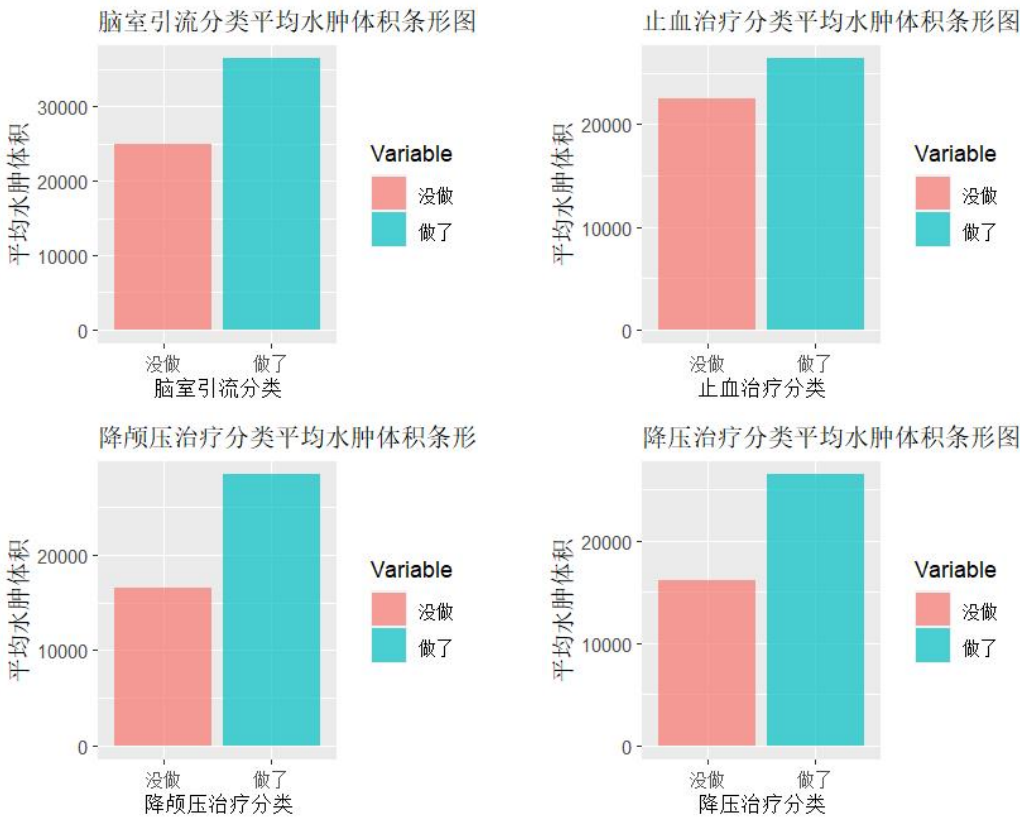
Step 5: 根据模型的结果解释治疗方法与血肿体积之间的关系。

③分析方法

首先，在计算每一名患者多次随访时的脑内水肿的平均体积后，计算不同治疗方法的水肿体积，得到平均水肿体积分布表 6.13。并绘制对应的条形图，如图 6.25。

表 6.13 不同治疗方法的平均水肿体积分布表

治疗方法	未完成患者的平均水肿体积	已完成患者的平均水肿体积
脑室引流	24909.71	36405.34
止血治疗	22565.91	26405.83
降颅压治疗	16617.42	28435.88
降压治疗	16092.48	26426.14
镇静、镇痛治疗	28207.12	25139.27
止吐养胃	22807.50	25685.80
营养神经	20961.69	25792.69



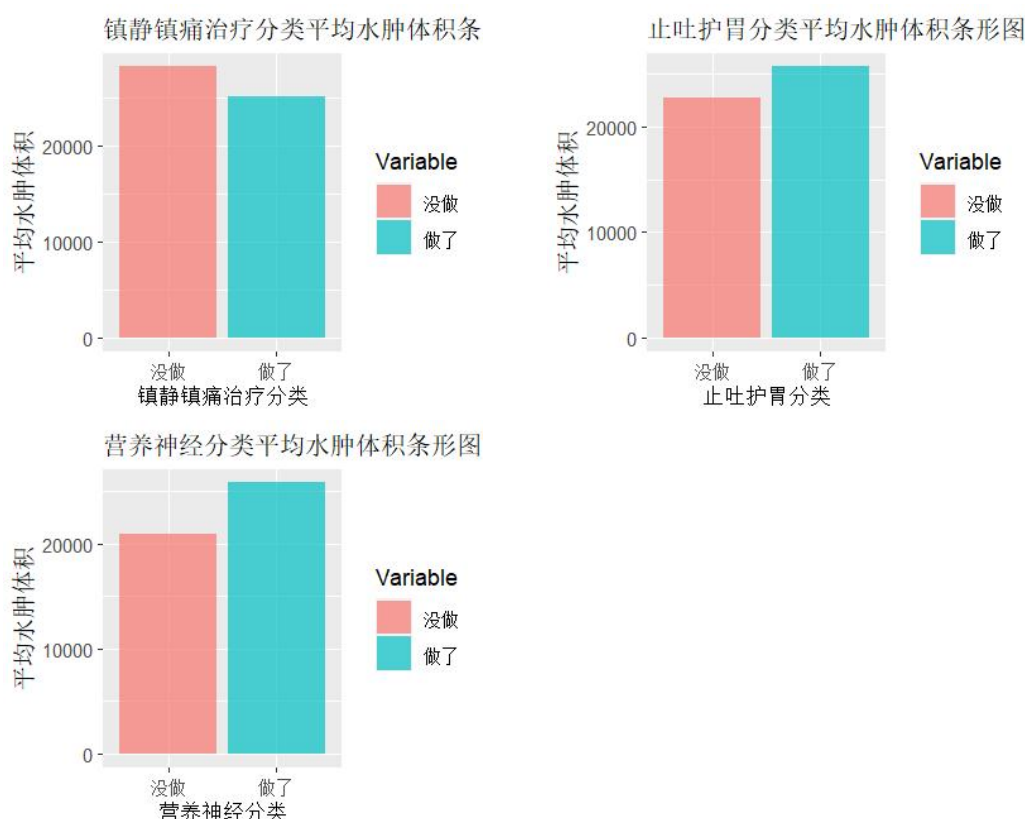


图 6.25 不同治疗方法的平均水肿体积分布图

根据不同治疗方法的平均水肿体积分布表和分布图，可以发现对于大部分治疗方法而言，和血肿一样，完成了治疗的患者的水肿体积更大，故可进一步推断出完成治疗方法对患者脑内水肿体积增加呈正相关关系。

为了精确地确定这种猜想，使用多元线性回归模型来拟合这些变量，其中自变量为 7 种治疗方法，响应变量为水肿体积。

使用 R 包中的函数 `lm()`，可以建立多元线性回归模型。最后根据回归模型的结果和方差分析确定显著水平即可。

6.4.3 第四小问模型求解与分析

1) 血肿体积与治疗方法相关性分析

首先将 7 种治疗方法数据类型转换为因子型：

```
p_m_imfo$脑室引流 <- factor(p_m_imfo$脑室引流,labels = c("haven't","have"))
p_m_imfo$止血治疗 <- factor(p_m_imfo$止血治疗,labels = c("haven't","have"))
p_m_imfo$降颅压治疗 <- factor(p_m_imfo$降颅压治疗,labels = c("haven't","have"))
p_m_imfo$降压治疗 <- factor(p_m_imfo$降压治疗,labels = c("haven't","have"))
p_m_imfo$镇静镇痛治疗 <- factor(p_m_imfo$镇静镇痛治疗,labels = c("haven't","have"))
p_m_imfo$止吐护胃 <- factor(p_m_imfo$止吐护胃,labels = c("haven't","have"))
p_m_imfo$营养神经 <- factor(p_m_imfo$营养神经,labels = c("haven't","have"))
```

使用 R 中的 `lm()` 函数建立 Logistics 回归模型：

```
fit <- lm(p_m_imfo$H_mean ~ p_m_imfo$脑室引流+p_m_imfo$止血治疗+
p_m_imfo$降颅压治疗+p_m_imfo$降压治疗+p_m_imfo$镇静镇痛治疗+
```

```
p_m_imfo$止吐护胃+p_m_imfo$营养神经)
```

得到输出信息为:

Call:

```
lm(formula = p_m_imfo$H_mean ~ p_m_imfo$脑室引流 + p_m_imfo$止血治疗 +  
    p_m_imfo$降颅压治疗 + p_m_imfo$降压治疗 + p_m_imfo$镇静镇痛治疗 +  
    p_m_imfo$止吐护胃 + p_m_imfo$营养神经)
```

Residuals:

Min	1Q	Median	3Q	Max
-30573	-14481	-4513	6908	119151

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	16942.0	17690.0	0.958	0.340715
p_m_imfo\$脑室引流 have	34410.5	9824.9	3.502	0.000714 ***
p_m_imfo\$止血治疗 have	752.4	6570.1	0.115	0.909078
p_m_imfo\$降颅压治疗 have	16188.6	6404.2	2.528	0.013183 *
p_m_imfo\$降压治疗 have	-5411.4	9451.0	-0.573	0.568330
p_m_imfo\$镇静镇痛治疗 have	650.0	7294.1	0.089	0.929185
p_m_imfo\$止吐护胃 have	-4883.8	15621.1	-0.313	0.755262
p_m_imfo\$营养神经 have	10168.1	12401.6	0.820	0.414390

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 23050 on 92 degrees of freedom

Multiple R-squared: 0.1789, Adjusted R-squared: 0.1164

F-statistic: 2.864 on 7 and 92 DF, p-value: 0.009475

结果与分析: 通过模型中各治疗方法的系数,可以看出脑室引流和降颅压治疗对平均血肿体积的影响是显著的,因为它们系数的 $p\text{-value} < 0.05$,特别是脑室引流的影响非常显著。这意味着脑室引流和降颅压治疗对于患者的平均血肿体积有深刻的相关性。F-statistic 用于测试整个模型的显著性,即治疗方法对平均血肿体积的影响是否显著。在这个模型中, F-statistic 的 $p\text{-value} = 0.009475 < 0.05$,表明整个模型具有显著性。这意味着治疗方法作为一组自变量对平均血肿体积的影响是显著的。

2) 水肿体积与治疗方法相关性分析

首先将 7 种治疗方法数据类型转换为因子型:

```
p_m_imfo$脑室引流 <- factor(p_m_imfo$脑室引流,labels = c("haven't","have"))  
p_m_imfo$止血治疗 <- factor(p_m_imfo$止血治疗,labels = c("haven't","have"))  
p_m_imfo$降颅压治疗 <- factor(p_m_imfo$降颅压治疗,labels = c("haven't","have"))  
p_m_imfo$降压治疗 <- factor(p_m_imfo$降压治疗,labels = c("haven't","have"))  
p_m_imfo$镇静镇痛治疗 <- factor(p_m_imfo$镇静镇痛治疗,labels = c("haven't","have"))  
p_m_imfo$止吐护胃 <- factor(p_m_imfo$止吐护胃,labels = c("haven't","have"))  
p_m_imfo$营养神经 <- factor(p_m_imfo$营养神经,labels = c("haven't","have"))
```

使用 R 中的 `lm()` 函数建立 Logistics 回归模型:

```
fit <- lm(p_m_imfo$ED_mean ~ p_m_imfo$脑室引流+p_m_imfo$止血治疗+
  p_m_imfo$降颅压治疗+p_m_imfo$降压治疗+p_m_imfo$镇静镇痛治疗+
  p_m_imfo$止吐护胃+p_m_imfo$营养神经)
```

得到输出信息为：

```
Call:
lm(formula = p_m_imfo$ED_mean ~ p_m_imfo$脑室引流 + p_m_imfo$止血治疗 +
  p_m_imfo$降颅压治疗 + p_m_imfo$降压治疗 + p_m_imfo$镇静镇痛治疗 +
  p_m_imfo$止吐护胃 + p_m_imfo$营养神经)
Residuals:
    Min       1Q   Median       3Q      Max
-30610 -11680  -4420    10165   70321
Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)      10171      14032   0.725   0.4704
p_m_imfo$脑室引流 have    11781       7794   1.512   0.1341
p_m_imfo$止血治疗 have   -1912       5212  -0.367   0.7145
p_m_imfo$降颅压治疗 have   11977       5080   2.358   0.0205 *
p_m_imfo$降压治疗 have    3892       7497   0.519   0.6049
p_m_imfo$镇静镇痛治疗 have -3525       5786  -0.609   0.5439
p_m_imfo$止吐护胃 have    2781      12391   0.224   0.8229
p_m_imfo$营养神经 have    4008       9838   0.407   0.6846

Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Residual standard error: 18290 on 92 degrees of freedom
Multiple R-squared:  0.1088,    Adjusted R-squared:  0.04097
F-statistic: 1.604 on 7 and 92 DF,  p-value: 0.144
```

结果与分析：通过观察模型中各治疗方法的系数，可以看出降颅压治疗对平均水肿体积的影响是显著的，因为它的系数的 $p\text{-value} < 0.05$ 。这说明降颅压治疗对于患者的平均水肿体积有明显的相关性。在此模型中，F-statistic 的 $p\text{-value} = 0.144$ ，表明整个模型在统计上不具有显著性。这意味着治疗方法作为一组自变量对平均水肿体积的影响是不够显著的。

综上所述，治疗方法中的脑室引流和降颅压治疗对平均水肿体积具有显著的正相关关系，而降颅压治疗对平均水肿体积也有显著的正相关关系。然而，在水肿体积方面，整个模型的显著性不高，可能要考虑其他因素以更好的解释治疗方法与水肿体积的关系。

七、 问题三模型建立与求解

7.1 第一、二小问模型建立与求解

7.1.1 第一、二小问求解思路

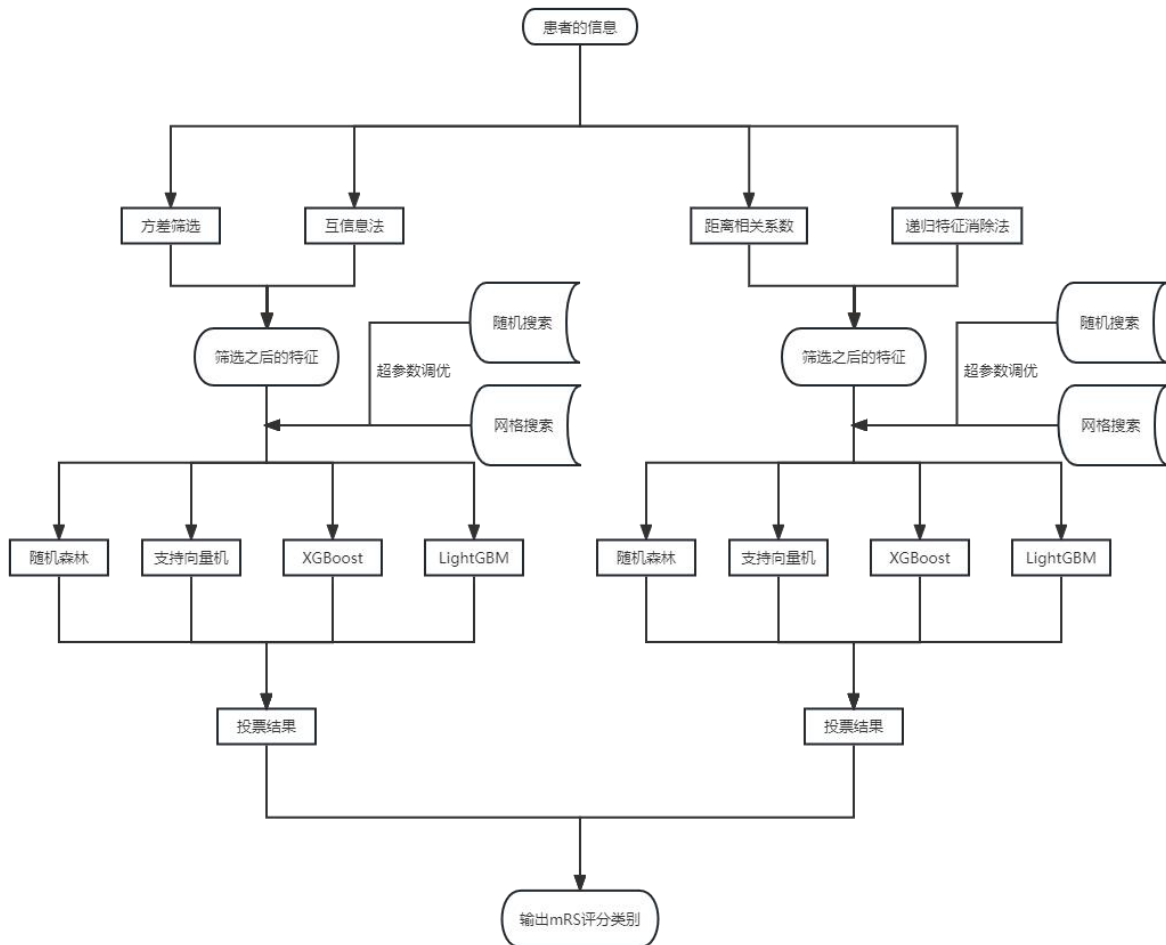


图 7.1 问题三第一、二小问求解思路

我们希望能够通过前 100 个患者个人史、疾病史、发病相关及影像结果构建对出血性脑卒中患者 90 天 mRS 的定类预测模型。我们为了确定用于这个分类预测模型的主要特征，首先要对这些字段进行处理与筛选。为了选择出具有独立性的模型特征，我们分别提出了第一种方法，利用方差筛选法和互信息法来筛选特征，第二种方法是使用距离相关系数法和递归特征消除法。使用这两种方法从前 100 个患者个人史、疾病史、发病相关及影像结果等字段中选出具有代表性的十个特征，再通过这十个特征对预测他们的 90mRS 评分。

第一种方法中，方差筛选法和互信息法都是基于特征与目标变量之间的相关性来进行特征选择的方法。方差筛选法通过计算特征的方差来评估特征的差异性，互信息法则通过计算特征与目标变量之间的互信息量来评估特征的相关性。通过选择合适的阈值，可以根据特征与目标变量之间的相关性程度来判断是否保留或剔除某个特征。

第二种方法中，距离相关系数法是一种通过计算距离相关系数来衡量特征与目标变量之间相关性的方法，我们同样利用此方法将一些相关性低的重要特征挑选出，然后距离相关系数旨在对分子描述符进行重要性排序，递归特征消除法可以基于任何机器学习模型进行特征选择，常用的有线性回归、逻辑回归和支持向量机等。通过逐步剔除不重

要的特征，模型在每一轮迭代中都会重新训练，最后得到一个特征数量较少但性能仍然较好的模型。

最后我们再将筛选出来的重要特征，利用随机森林、支持向量机、XGBoost、LightGBM 四种模型通过网格搜索和随机搜索进行超参数优化，然后得到综合两种方法调参、四种机器学习算法的八个模型，同时这八个模型又结合了分两种特征筛选的方法，所以总共我们能有十六种得到的最终模型。

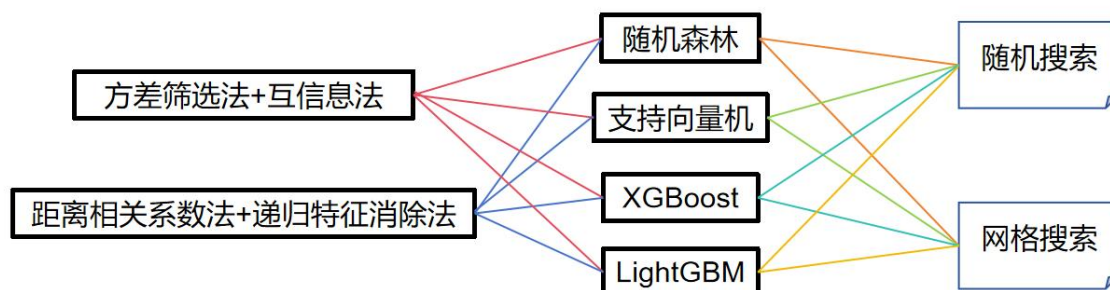


图 7.2 模型来源图

但是我们不能选取所有的模型，我们需要通过特征筛选方法、超参数调整方法来对四种机器学习算法也进行一次筛选。

7.1.2 第一、二小问模型建立

(4) 方差筛选

低方差滤波，是一种常用的特征选择方法，主要用于识别那些方差较小的特征，并将其从数据集中移除。这一方法的核心目标是减少数据集中的噪音和冗余信息，从而提高模型的性能和泛化能力。通常，低方差滤波会对每个特征的方差进行计算，并设定一个阈值。只有当某个特征的方差低于设定的阈值时，该特征才被认为是低方差特征，随后会被移除。

方差筛选是低方差滤波的一种具体实现方法，它依赖于计算每个特征的方差，并根据设定的阈值来进行特征选择。通过简单地设置一个阈值，方差筛选可以轻松识别出那些方差较低的特征，并将它们舍弃。这个过程非常高效，因为它只需要考虑特征的方差情况，而不需要复杂的计算或参数调整。^[15]

(5) 互信息

互信息法是一种常用的特征选择方法，其主要目的是评估特征与目标变量之间的相关性。这个方法建立在信息论的基础上，通过计算特征和目标变量之间的互信息来度量它们之间的依赖关系。

互信息衡量了两个随机变量之间信息的共享程度，也就是一个随机变量的观察结果对于另一个随机变量的预测能力。在特征选择中，互信息的应用主要是衡量每个特征与目标变量之间的相关性程度。互信息的值越大，表示特征与目标变量之间的关联性越强。这一方法可以帮助筛选出与目标变量高度相关的特征，有助于提高模型性能和泛化能力。

对于两个随机变量 X 和 Y ，它们的互信息 $I(X;Y)$ 表示 X 和 Y 之间的信息共享程度，可以通过以下公式计算：

$$I(X;Y) = H(X) - H(X|Y)$$

其中， $H(X)$ 是随机变量 X 的熵， $H(X|Y)$ 是在给定 Y 的条件下随机变量 X 的条件熵。熵 $H(X)$ 可以通过以下公式计算：

$$H(X) = -\sum P(x) \log(P(x))$$

其中， $P(x)$ 是随机变量 X 取值为 x 的概率。

条件熵 $H(X|Y)$ 可以通过以下公式计算：

$$H(X|Y) = -\sum P(x,y) \log(P(x|y))$$

其中， $P(x,y)$ 是随机变量 X 和 Y 同时取值为 (x,y) 的概率， $P(x|y)$ 是在给定 Y 的条件下随机变量 X 取值为 x 的概率。

在特征选择中，通常会评估每个特征与目标变量之间的互信息，并据此对特征进行排序。互信息值较高的特征通常被选为最终特征，以便构建与目标变量的预测模型。这一过程有助于减少特征空间的维度，提高模型效率和预测性能。

(6) 距离相关系数法

距离相关系数（Distance Correlation）是一种用于度量两个随机变量之间相关性的方法，它具有捕捉非线性相关性的优势。与传统的相关系数（例如皮尔逊相关系数）不同，距离相关系数不受线性关系假设的制约，因此更适用于描述复杂的关联关系。这一方法的应用有助于降低特征空间的维度，从而提高模型的效率和预测性能。

距离相关系数的计算步骤如下：

- 1) 计算样本数据的距离矩阵：对于每个样本，通过欧氏距离或其他距离度量方法（如曼哈顿距离、马氏距离等），计算其与其他样本之间的距离，从而得到一个距离矩阵。对于两个样本向量 x 和 y ，可以使用欧氏距离公式来计算它们之间的距离：

$$d(x,y) = \sqrt{\sum (x_i - y_i)^2}$$

其中， x_i 和 y_i 分别表示向量 x 和 y 的第 i 个元素。

- 2) 计算样本数据的中心化距离矩阵：对距离矩阵进行中心化操作，即将每个元素减去其所在行的均值、所在列的均值以及总体的均值。这一步骤有助于消除数据的整体偏移，使得距离相关系数更准确地反映出变量之间的相关性

$$\begin{aligned} M_{ij} = & d(x_i, y_j) - \text{mean}(d(x_i, y_l), \dots, d(x_i, y_n)) \\ & - \text{mean}(d(x_l, y_j), \dots, d(x_n, y_j)) \\ & + \text{mean}(d(x_l, y_l), \dots, d(x_n, y_n)) \end{aligned}$$

其中， M_{ij} 表示中心化的距离矩阵的第 i 行第 j 列元素， x_i 和 y_j 分别表示样本数据的第 i 个和第 j 个向量。

- 3) 计算距离矩阵的相关系数：对中心化距离矩阵进行相关系数的计算，得到最终的距离相关系数。距离相关系数的取值范围在 0 到 1 之间，数值越接近 1 表示变量之间的相关性越强。这一数值反映了变量之间的非线性关系程度。

$$\begin{aligned} dCor(X,Y) = & \sqrt{\sum (M_{ij} * M_{ij})} \\ & / \sqrt{(\sum (M_{ii} * M_{ii}) \\ & * \sum (M_{jj} * M_{jj}))} \end{aligned}$$

其中， X 和 Y 分别表示两个样本数据的矩阵， $dCor(X,Y)$ 表示它们之间的距离相关系数。

(7) 递归特征消除法

递归特征消除法（Recursive Feature Elimination, RFE）是一种特征选择方法，其核心思想是逐步排除对预测模型影响较小的特征，从给定的特征集合中精炼出最重要的特征。它的操作步骤包括以下几个关键点：首先，利用原始特征集合构建预测模型，并计算每个特征的重要性得分。然后，根据得分，逐步移除重要性较低的特征，更新特征集合。接着，使用更新后的特征集合重新训练预测模型，并计算新的特征重要性得分。这一过程重复进行，直到达到预设的特征数量或满足停止条件为止。在递归特征消除法中，特征重要性的计算方法通常包括基于模型的特征重要性评估和基于模型性能变化的特

征重要性评估。通过这种逐步剔除的策略，最终得到的特征集合将包含对模型预测性能贡献最大的特征。这一过程有助于提高模型的效率和泛化能力。

计算步骤如下所示：

1) 计算特征重要性得分

2) 根据得分选择重要性较低的特征： $\text{least_important_feature} = \text{argmin}(\text{importance_score})$

3) 更新特征集合：特征集合 = 特征集合 - $\{\text{least_important_feature}\}$

重复上述步骤，直到达到预设的特征数量或达到停止条件。

(8) 随机森林模型

随机森林算法是一种集成学习方法，它基于决策树算法的原理。决策树是一种树状结构，其中每个内部节点代表一个特征或属性，每个分支表示这个特征或属性的一个取值，而每个叶子节点表示一个分类或回归的结果。借助决策树，我们可以将数据集划分为多个子集，每个子集包含了相同特征或属性的数据。然后，我们可以对每个子集进行分析，进行分类或回归任务。这一过程有助于将数据集的复杂问题分解为更简单的子问题，从而提高了算法的效率和准确性。随机森林示意图如下所示：

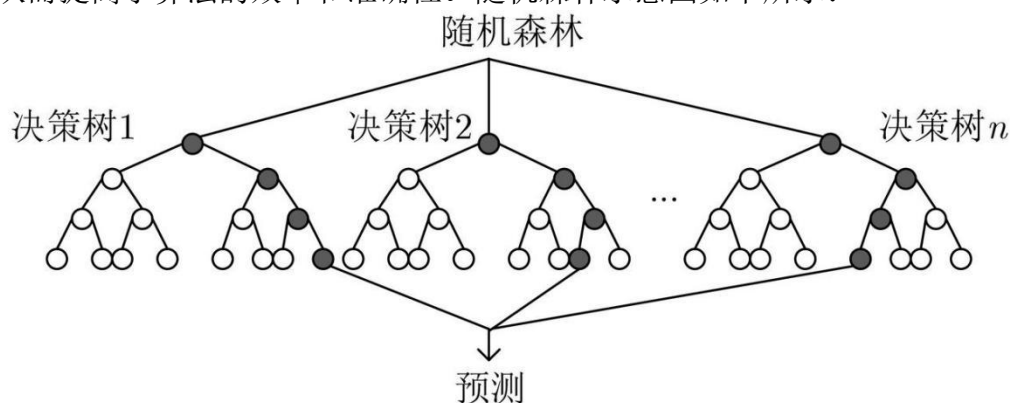


图 7.3 随机森林原理

随机森林算法引入了两种关键的随机性：一是数据的随机性，二是特征的随机性。对于数据的随机性，它采用自助采样法从原始数据集中随机选取了 n 个样本（通常 n 小于原始数据集的样本数），构建一个新的训练数据子集，然后使用这个子集来训练一个新的决策树。至于特征的随机性，随机森林在每个决策树的节点上，从总特征数中随机选择了 m 个特征（通常 m 远小于总特征数），再从这 m 个特征中挑选出最佳的用于节点分裂。^[16]

随机森林算法是一种改进的 Bagging 集成学习方法，它的核心思想仍然是 Bagging，但采用了 CART 决策树作为基学习器，具有更好的性能和鲁棒性。具体过程如下：

1) 输入为样本集： $D = \{(x, y_{(1)}), (x_2, y_{(2)}), \dots, (x_m, y_{(m)})\}$

2) 对于 $t = 1, 2, \dots, T$:

对训练集进行第 t 次随机采样，共采集 m 次，得到包含 m 个样本的采样集 D_T 。

用采样集 D_T 训练第 T 个决策树模型 $G_T(x)$ ，在训练决策树模型的节点的时候，在节点上所有的样本特征中选择一部分样本特征，在这些随机选择的部分样本特征中选择一个最优的特征来做决策树的左右子树划分。

(9) 支持向量机模型

支持向量机 (Support Vector Machine, SVM) 通常简称为 SVM，是一种二分类模型。其基本思想是在特征空间中找到一个间隔最大的线性分类器。这个分类器的特点在于它能够同时最小化经验误差并最大化几何边缘，因此 SVM 也被称为最大边缘分类器。学习策略的核心目标是最大化分类器的间隔，这最终可以转化为一个凸二次规划问题的

求解过程。SVM 在文本分类、图像分类、生物序列分析、生物数据挖掘、手写字符识别等多个领域都有广泛的应用。[17]

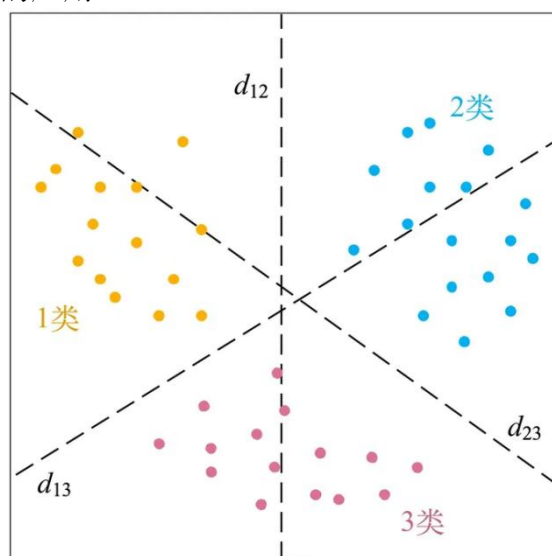


图 7.4 SVM 分类原理

SVM 的核心思想是将数据点映射到一个高维空间，然后在这个高维空间中找到一个最大间隔的超平面。在这个超平面的两侧，建立两个互相平行的超平面，以最大化它们之间的距离或差距。这个距离的最大化目标有助于减小分类器的总误差。在具体描述中，我们有一些数据点，每个数据点都属于不同的类别。我们用 x 表示数据点，用 y 表示类别，通常使用 1 和 -1 来表示两个不同的类别。线性分类器的任务是在一个 n 维的数据空间中找到一个或多个超平面，这些超平面用来将不同类别的数据点分开。这个过程旨在最大化超平面之间的间隔。

(10) XGBoost 模型

XGBoost 是一种基于梯度提升树 (Gradient Boosting Decision Tree, GBDT) 的机器学习算法，它通过不断迭代来训练树模型，以逼近真实分布。XGBoost 的目标是提高速度和效率，因此被称为 "Extreme Gradient Boosting" (极端梯度提升)。

XGBoost 的核心思想是在每次迭代中添加一棵树，并不断进行特征分裂，以生成新的树，每棵树的生成都是为了拟合上次预测的残差。当训练完成后，有多棵树组成的模型，每个树对应一个叶子节点，每个叶子节点有一个分数。对于要预测的样本，它会在每棵树中落到一个叶子节点上，最后将每棵树对应的分数相加，得到该样本的最终预测值。

XGBoost 的目标是使多棵树的预测值 y_i' 尽量接近真实值 y_i ，并具有良好的泛化能力。这一目标通过多轮迭代实现，每一轮迭代都在现有树的基础上增加一棵新树，以拟合前面树的预测结果与真实值之间的残差。

(11) LightGBM

LightGBM 是一种基于梯度提升决策树 (Gradient Boosting Decision Tree) 的高效机器学习算法。它通过一系列优化技术来提高模型的训练速度和性能，核心思想包括梯度提升和决策树。梯度提升是一种迭代方法，LightGBM 使用负梯度来更新模型的预测值，以最小化损失函数。每一步都引入一个新的模型来减少残差，决策树在这里被用作近似函数。决策树是一种树状分类器或回归器，将数据逐步划分为不同子集，以使每个子集内的数据具有相似的目标变量值。在 LightGBM 中，每个决策树表示模型中的一个简单模型，通过特征值的划分和叶节点的预测值来完成。为提高性能，LightGBM 采用了直方图算法、互斥特征捆绑和 GOSS 等创新技术，以减少内存占用、提升训练速度，并提

高模型准确性。综上所述，LightGBM 以梯度提升和决策树为核心，通过一系列优化策略实现高效的机器学习。

（12）网格搜索

超参数调优在机器学习中至关重要，能够显著提升模型性能。一种常用的方法是网格搜索，它通过穷举搜索超参数的不同组合来找到最佳配置。具体而言，网格搜索首先为每个超参数指定一组备选值，然后将这些备选值的组合构成一个超参数网格。接下来，对每个超参数组合进行模型训练和评估，以找到性能最佳的配置。尽管网格搜索是一种有效的方法，但当超参数数量和备选值较多时，搜索空间急剧扩大，计算开销较大，导致效率下降。

（13）随机搜索

随机搜索是一种随机化的超参数调优方法，它不同于网格搜索那种穷举搜索所有可能组合的方式。相反，随机搜索会在超参数空间中随机抽样一定数量的组合进行评估，以找到最佳的超参数配置。这种方法的优势在于它更加高效，尤其当超参数空间很大时，能够在相对短的时间内找到性能良好的超参数组合。

（14）分类问题投票法

集成学习是一种强大的机器学习方法，它通过结合多个模型的预测结果来提高整体性能。在集成学习中，我们通常会使用一组弱学习器（也称为基学习器），这些模型可以是相同的或不同的，用于解决同一个问题。关键的思想是通过协同工作，这些弱学习器能够产生更准确和鲁棒的整体模型。

通常，基本模型的性能可能有限，可能因高偏差或高方差而表现不佳。集成学习的目标是通过合理组合这些基本模型，以减少偏差或方差，最终获得一个强大的模型。投票是一种常见的集成学习方法。集成学习是一种强大的技术，它通过协同多个模型的力量来提高机器学习任务的性能。投票是其中一种常见的集成策略，可用于分类问题，通过多数决策或赋予权重的方式来获得更好的分类结果。

（15）交叉验证

在交叉验证中，我们采用一种有效的技术来评估和调整模型性能。这个技术的核心思想是将训练数据划分为多个小的训练-测试子集，然后利用这些子集来训练和验证模型。在标准的 k 折交叉验证中，我们首先将整个数据集分为 k 个相等大小的子集，每个子集称为一个“折叠”。然后，我们进行 k 轮迭代，每轮中选择一个不同的折叠作为验证集，而其余的 $k-1$ 个折叠作为训练集。这意味着每个折叠都有机会充当验证集，从而使模型在未见过的数据上进行验证。每一轮迭代中，我们使用训练集来训练模型，然后使用验证集来评估模型的性能。最终，我们将 k 轮迭代中的性能评估结果取平均值，得到一个综合的性能指标，用于衡量模型的整体性能。



图 7.5 5 折交叉验证示例图

7.1.3 第一、二小问模型求解与分析

特征的选取:

使用方差筛选法和互信息法选出重要特征如下:

["脑出血前 mRS 评分", "饮酒史", "止吐护胃", '营养神经', "HM_ACA_R_Ratio", "HM_PCA_L_Ratio", "ED_Cerebellum_R_Ratio", "ED_MCA_L_Ratio", "ED_Cerebellum_L_Ratio", "original_shape_Elongation"]

然后再使用距离相关系数法和递归特征消除法选出重要特征如下:

['HM_volume', '糖尿病史', 'NCCT_original_firstorder_Range', 'HM_ACA_L_Ratio', '冠心病史', 'NCCT_original_firstorder_Entropy', '饮酒史', 'original_shape_Maximum2DDiameterColumn', 'NCCT_original_firstorder_Uniformity', 'ED_volume']

先使用方差筛选法和互信息法得到的重要特征进行模型建立:

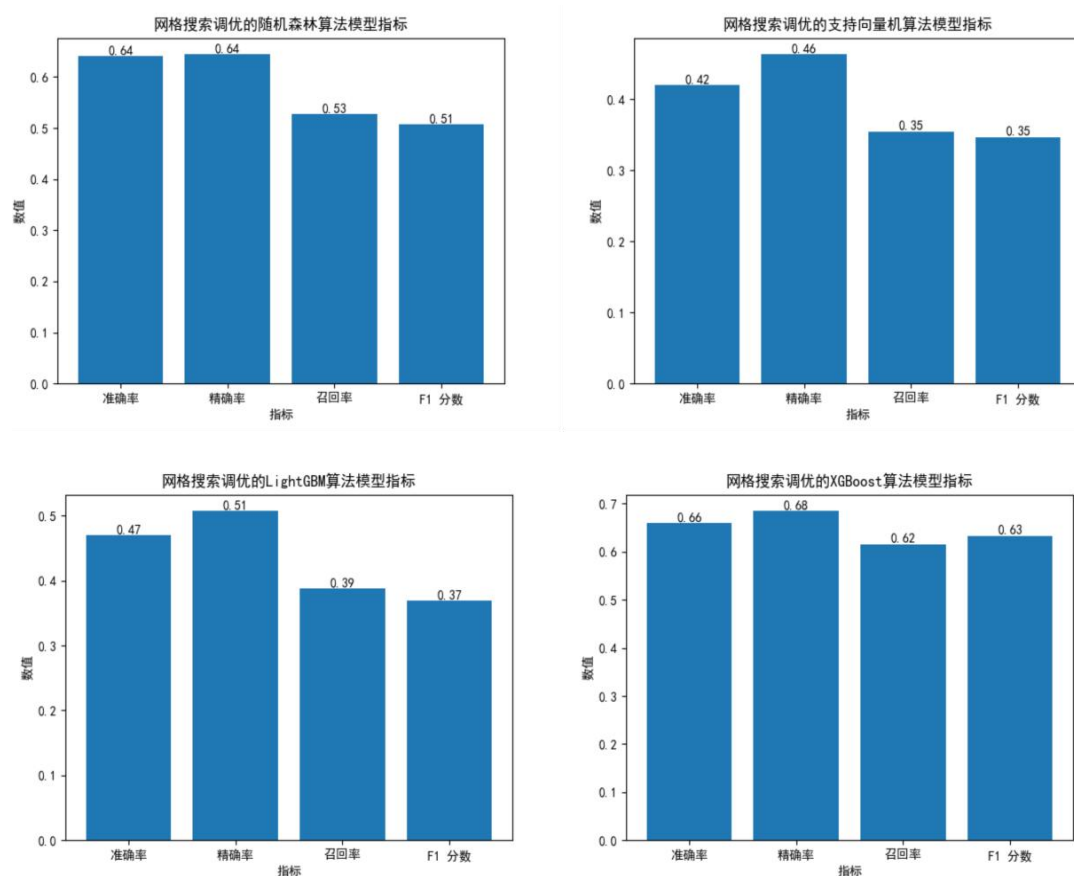


图 7.6 方差筛选法、互信息法特征筛选后用网格搜索调参后的模型指标图

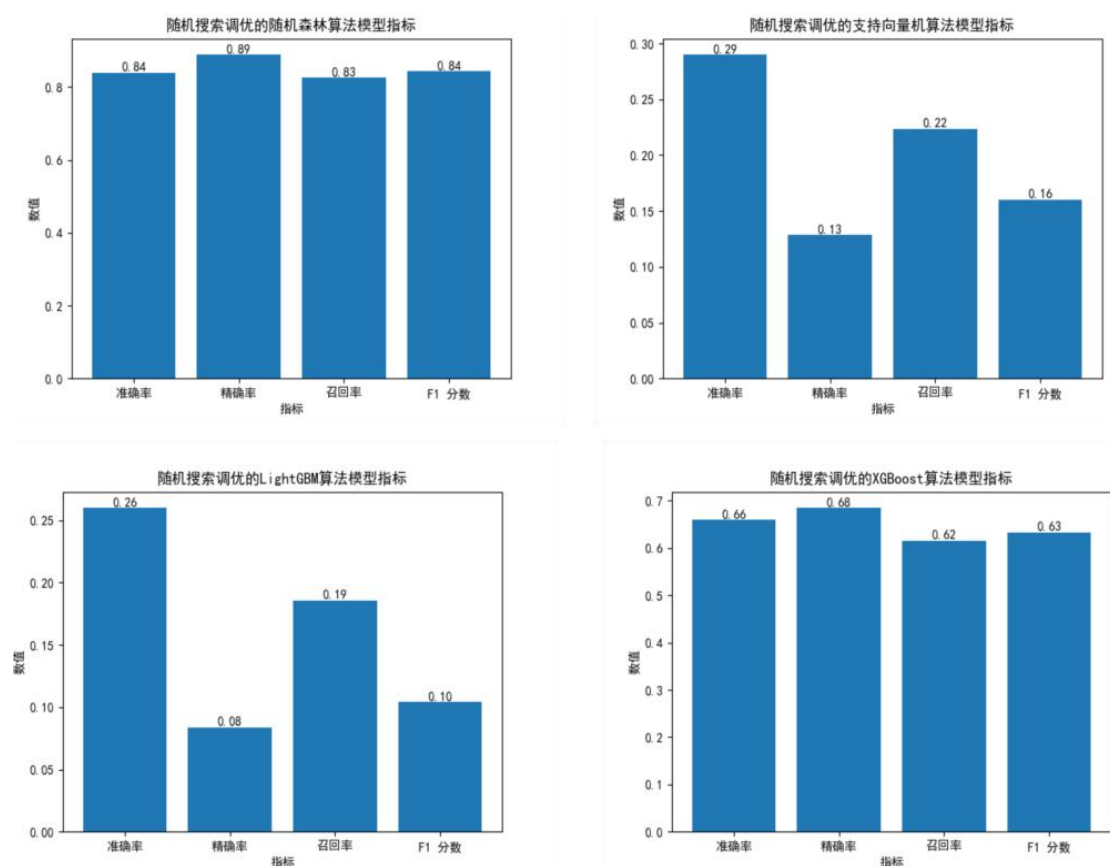


图 7.7 方差筛选法、互信息法特征筛选后用随机搜索调参后的模型指标图

根据以上原理可以得到对应的模型，然后用模型去预测前 100 名患者的 mRS 评分等级，并且分析准确率。在用方差筛选法和互信息法先筛选特征之后，再利用网格搜索和随机搜索进行超参数优化之后得到的模型，其中可以看出：

在网格搜索中，由于遍历了大部分的超参数，所以可以比较全面的来训练模型，但是耗时也相应的提升了许多。可以看出网格搜索调优的 XGBoost 和随机森林算法的准确率有比较好的效果。

而随机搜索中，随机森林算法达到了好的效果，可以在前 100 名患者的 mRS 评分等级中处于 0.89 的准确率，其次是 XGBoost 算法，也可以有较好的预测能力，但是随机搜索由于自身的随机性，其他两个算法 LightGBM 和支持向量机的整体效果并不好，所以我们在后续的预测工作中会让这些表现不好的模型不参与后续的 mRS 评分等级投票。

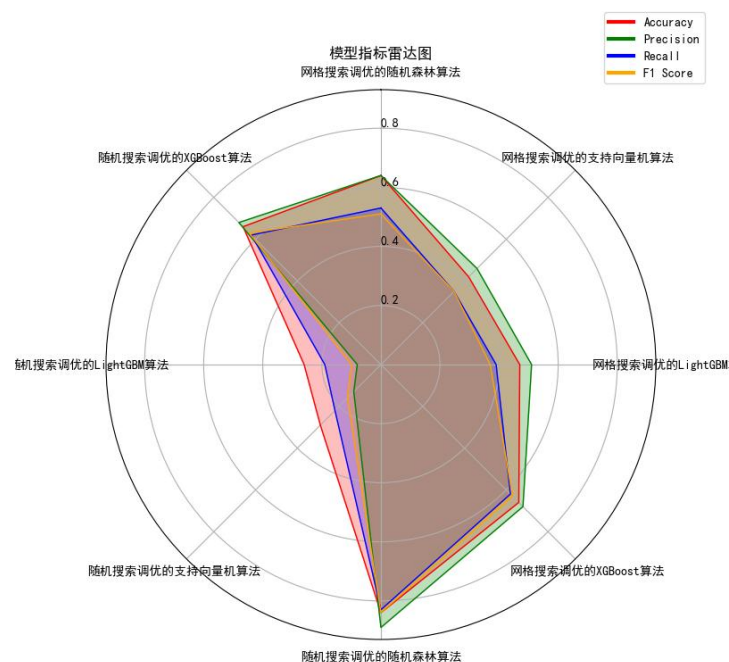


图 7.8 八个模型的指标雷达图

可以用同样的分析方法再看用距离相关系数法和递归特征消除法筛选完之后几种算法的效果：

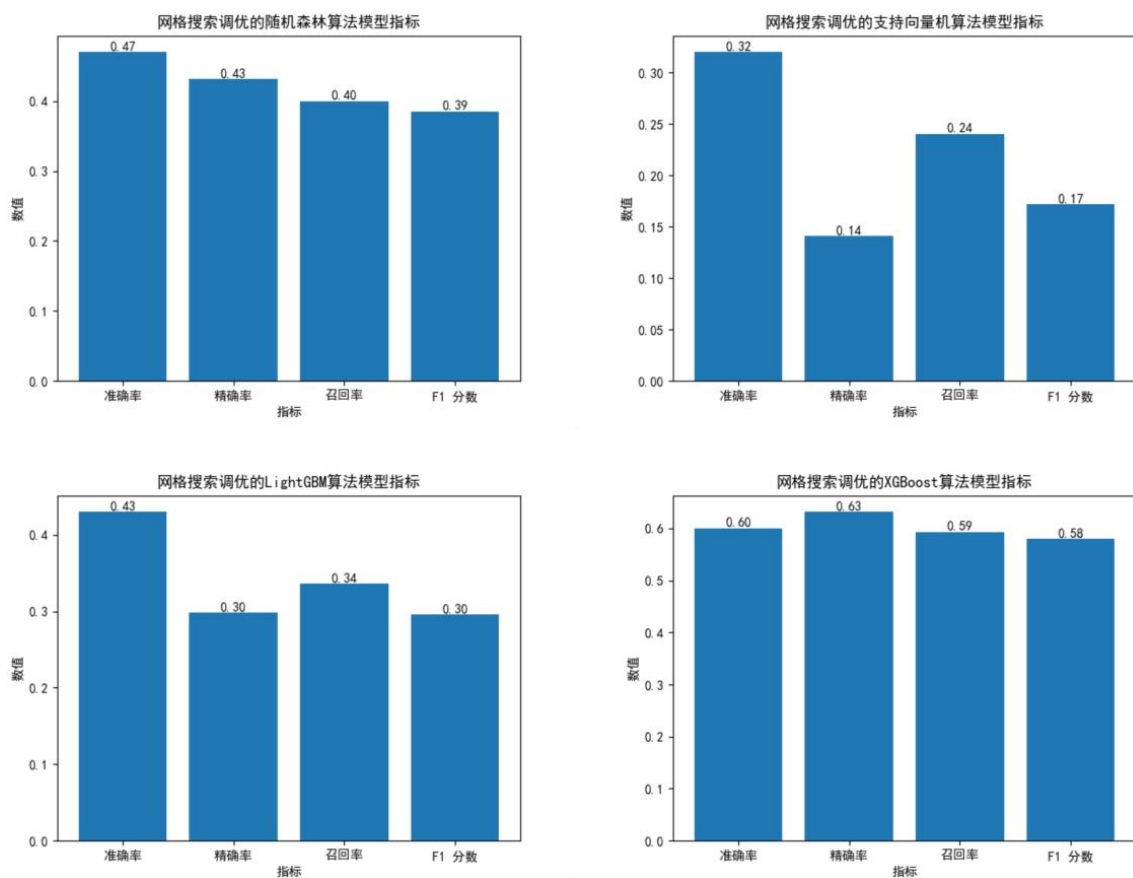


图 7.9 距离相关系数法和递归特征消除法特征筛选后用网格搜索调参后的模型指标

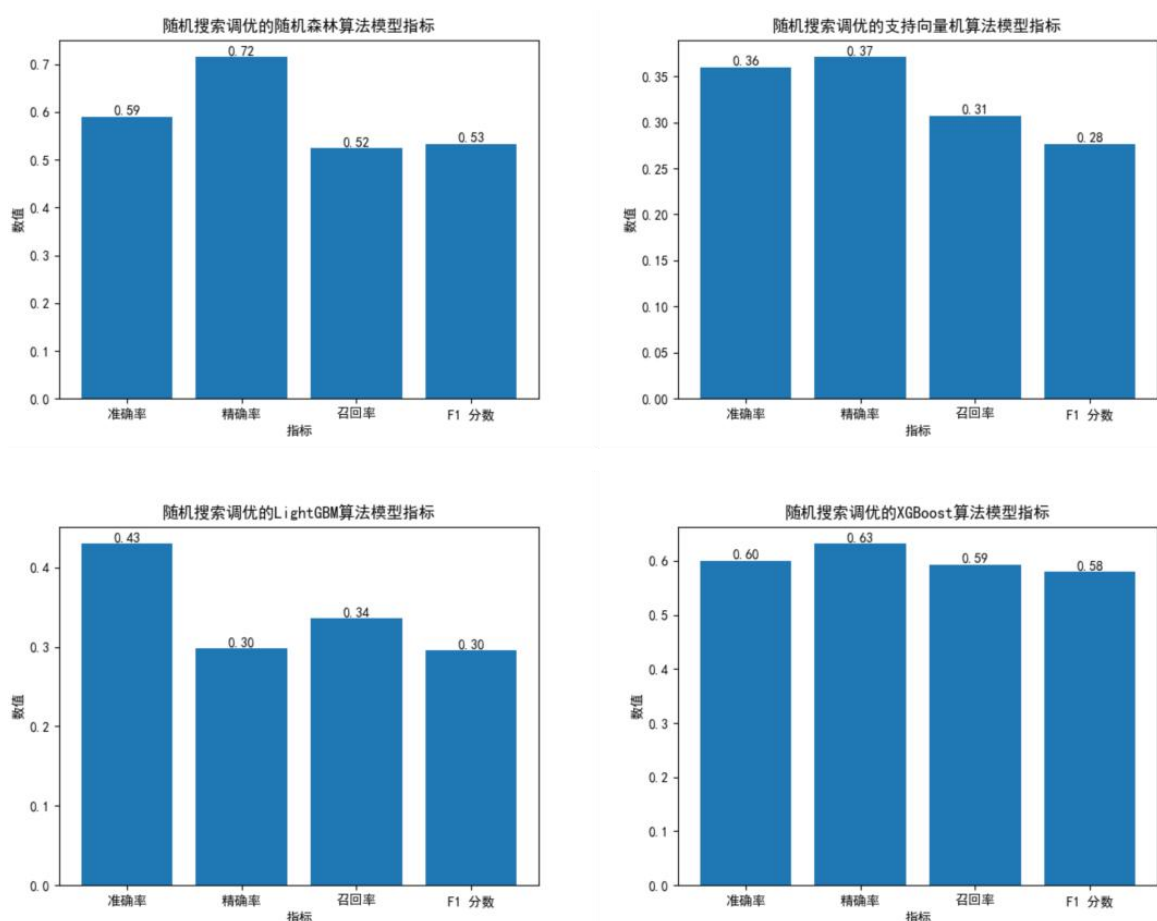


图 7.10 距离相关系数法和递归特征消除法特征筛选后用随机搜索调参后的模型指标

可以得到另一种特征筛选方法对应的模型，同样，我们用模型去预测前 100 名患者的 mRS 评分等级，并且分析准确率。在我们用方差筛选法和互信息法先筛选特征之后，再利用网格搜索和随机搜索进行超参数优化之后得到的模型，其中可以看出：在网格搜索中，网格搜索调优的 XGBoost 有比较好的效果，而随机森林算法和 LightGBM 算法的准确率比较一般。而随机搜索中，随机森林算法一样达到了好的效果，可以在前 100 名患者的 mRS 评分等级中有较高的准确率，其次一样是 XGBoost 算法，也可以有较好的预测能力。

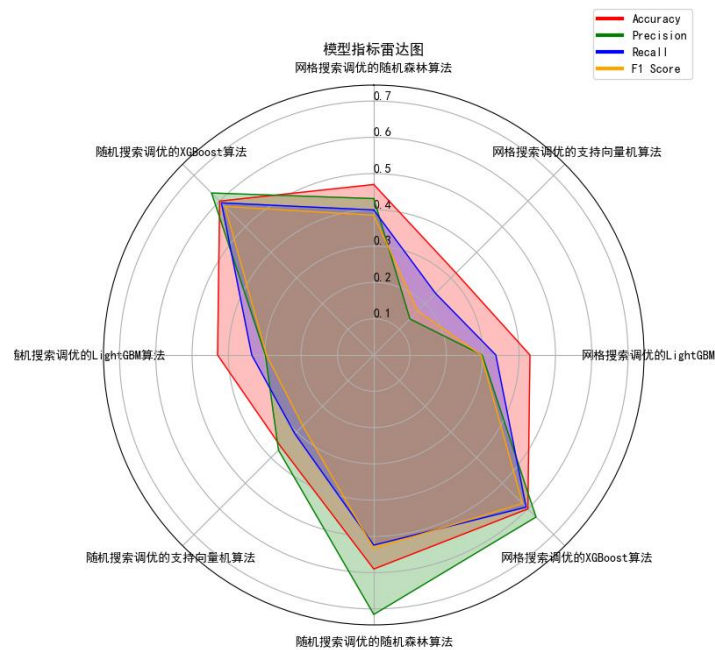


图 7.11 八个模型的指标雷达图

然后将所有模型进行排序，从以上信息我们可以得知，效果最好的且接近的六个模型分别是：

表 7.1 拟合效果最好的六个模型

	方差筛选法、互信息法		距离相关系数法、递归特征消除法	
随机搜索	随机森林	XGBoost	随机森林	XGBoost
网格搜索	XGBoost		XGBoost	

然后再将所有人的信息输入利用得到的最好模型进行预测可得结果为：（这里展示部分预测得分）

表 7.2 基于首次影像信息预测 101 号患者到 160 号患者 mRS

	预测 mRS（基于首次影像）		预测 mRS（基于首次影像）
sub101	2	sub131	5
sub102	2	sub132	3
sub103	3	sub133	1
sub104	1	sub134	2
sub105	4	sub135	6
sub106	3	sub136	1
sub107	3	sub137	5
sub108	1	sub138	1
sub109	1	sub139	5
sub110	0	sub140	2
sub111	5	sub141	2
sub112	2	sub142	3
sub113	1	sub143	1
sub114	5	sub144	3
sub115	5	sub145	1
sub116	2	sub146	0

sub117	2	sub147	4
sub118	4	sub148	1
sub119	1	sub149	4
sub120	4	sub150	2
sub121	2	sub151	1
sub122	0	sub152	5
sub123	3	sub153	3
sub124	1	sub154	2
sub125	3	sub155	3
sub126	0	sub156	6
sub127	1	sub157	2
sub128	1	sub158	6
sub129	3	sub159	3
sub130	5	sub160	5

然后再我们将所有随访信息输入进行预测可得结果为：

表 7.3 基于首次影像+随访信息预测 131 号患者到 160 号患者 mRS

	预测 mRS 基于首次影像+随访		预测 mRS 基于首次影像+随访
sub131	4	sub146	1
sub132	3	sub147	3
sub133	2	sub148	0
sub134	4	sub149	2
sub135	6	sub150	1
sub136	2	sub151	1
sub137	4	sub152	5
sub138	1	sub153	3
sub139	6	sub154	1
sub140	3	sub155	2
sub141	3	sub156	6
sub142	5	sub157	2
sub143	3	sub158	4
sub144	1	sub159	2
sub145	2	sub160	3

7.2 第三小问模型建立与求解

7.2.1 第三小问求解思路

根据任务要求, 需要根据表 1 “患者列表及临床信息” 的 100 名患者数据中的预后和个人史、疾病史、治疗方法以及表 2 “患者影像信息血肿及水肿的体积及位置” 和表 3 “患者影像信息血肿及水肿的形状及灰度分布” 给出的影像特征, 建立模型分析这些变量间的关联关系, 并给出临床诊断的决策建议。本问题求解思路流程图如图 7.12 所示。

首先, 需要对数据进行预处理。在处理完缺失值和异常之后, 建立相关变量指标, 并对指标内的变量进行分类, 以便更好地发掘变量间的关联。

其次, 将问题分为 3 个子问题: 个人史、疾病史和治疗方法对于预后的关联性分析、不同位置的血肿和水肿体积对于预后的关联性分析和信号强度特征与形状特征对于预后的关联性分析。针对这 3 个子问题, 根据其变量特征, 分别采用不同的相关性分析模型与方法: Fisher 精确检验、Levene 检验、有序 Logistics 回归模型、单因素方差分析。

最后, 根据建立的模型探究各变量间的关联关系, 参考相关临床诊疗方案和案例对出血性脑卒中患者预后提出针对性建议。

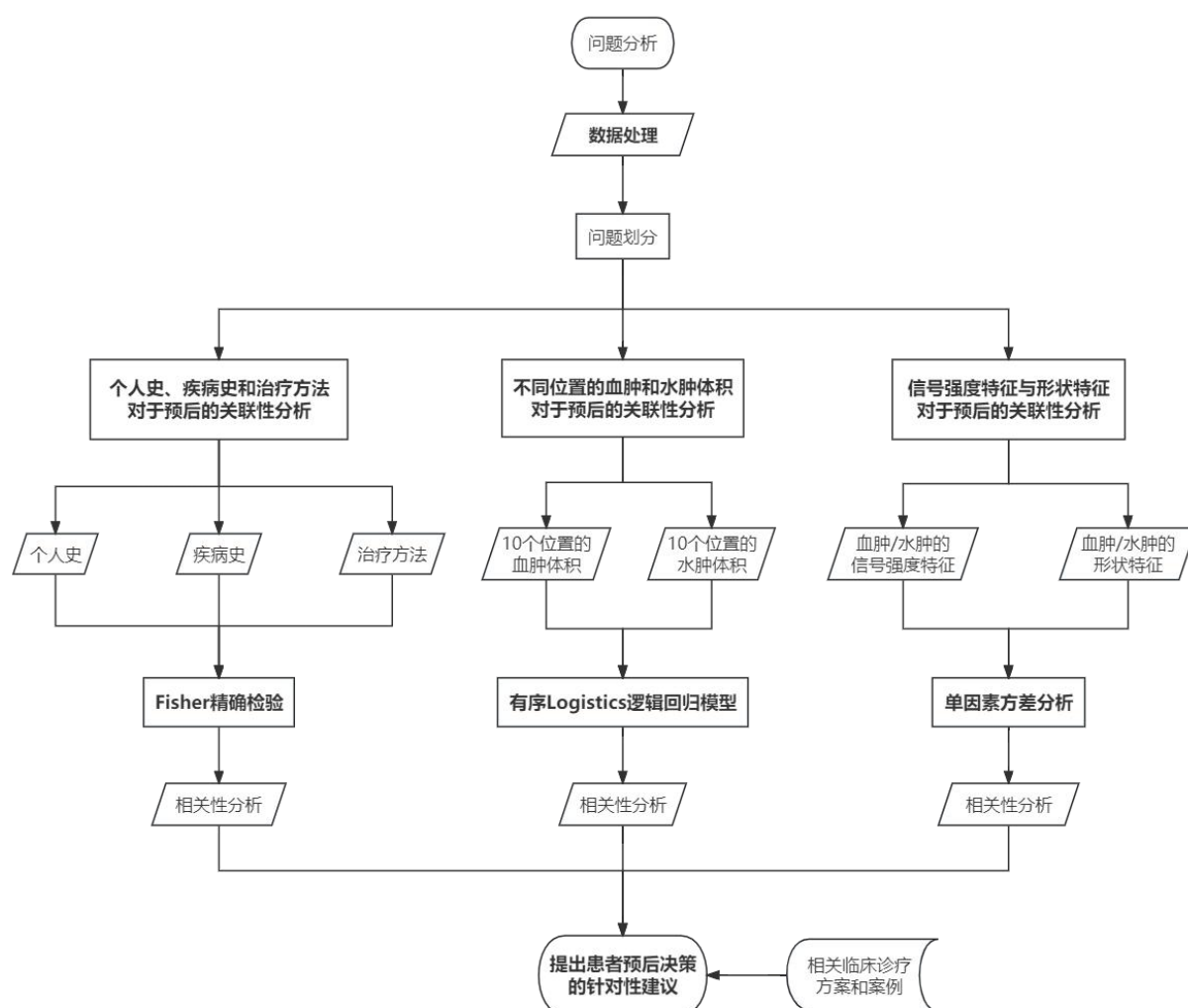


图 7.12 第三小问问题分析思路流程图

7.2.2 第三小问模型建立

(1) 问题分类

根据题目所述任务, 可将个人史、疾病史、治疗方法划归为患者个人特征变量。而影像特征 (包括血肿/水肿体积、血肿/水肿位置、信号强度特征、形状特征) 可以划分为: 探究血肿和水肿在大脑不同部位的体积对于预后评分的影响, 使用 CT 对患者脑内血肿和水肿扫描形成的目标区域内体素信号强度的分布和三维形状的描述对于预后带来的影响分析。

故将三个子问题分别表述为: ①个人史、疾病史和治疗方法对于预后的关联性分析; ②不同位置的血肿和水肿体积对于预后的关联性分析; ③信号强度特征与形状特征对于预后的关联性分析。

(2) 个人史、疾病史和治疗方法对于预后的关联性模型

①数据说明

根据表 1 数据, 可作以下定义:

个人史定义为: 年龄、性别、吸烟史、饮酒史和血压

疾病史定义为: 高血压病史、卒中病史、糖尿病史、房颤史和冠心病史

治疗方法定义为: 脑室引流、止血治疗、降颅压治疗、降压治疗、镇静镇痛治疗、止吐养胃和营养神经

②数据转换

Step 1: 提取出表 1 患者信息中的性别, 并重新定义 (男性: 1/女性: 2)。

Step 2: 提取出表 1 患者信息中的血压, 并按照以下方法重新定义

正常高压 NP: 收缩压 $<120\text{mmHg}$ 且 舒张压 $<80\text{mmHg}$

正常高值 (正常高血压) NH: 收缩压 $120-129\text{mmHg}$ 且舒张压 $<80\text{mmHg}$

高血压级别 1 (轻度高血压) H1: 收缩压 $130-139\text{mmHg}$ 或舒张压 $80-89\text{mmHg}$

高血压级别 2 (中度高血压) H2: 收缩压 $\geq 140\text{mmHg}$ 或舒张压 $\geq 90\text{mmHg}$

严重高血压 SH: 收缩压 $\geq 180\text{mmHg}$ 且/或舒张压 $\geq 120\text{mmHg}$

Step 3: 提取出表 1 患者信息中的年龄, 划分为 5 类 (1: 0-50 岁/2: 50-60 岁/3: 60-70 岁/4: 70-80 岁/5: 80-100 岁)

③分析方法

个人史:

首先, 计算不同 90mRs 下的平均年龄分布, 并使用正态性检验, 发现 7 组

(0-6 分) 人群的年龄分布不完全符合正态分布, 故使用非参数检验中的 Levene 检验, 因为非参数检验适用范围广, 对正态性敏感度不是很高。之后针对 5 类年龄、2 类性别、5 类血压、2 类吸烟 (0: 不吸烟/1: 吸烟)、2 类饮酒 (0: 不饮酒/1: 饮酒) 在 7 组 mRs 下的分布做数量统计, 因为样本数量较少, 故采用 Fisher 精确检验探究与 mRs 的显著性关系。

疾病史:

针对高血压病史 (0: 无/1: 有)、卒中病史 (0: 无/1: 有)、糖尿病史 (0: 无/1: 有)、房颤史 (0: 无/1: 有) 和冠心病史 (0: 无/1: 有) 在 7 组 mRs 下的分布做数量统计, 因为样本数量较少, 则采用 Fisher 精确检验探究与 mRs 的显著性关系。

治疗方法:

针对脑室引流 (0: 无/1: 有)、止血治疗 (0: 无/1: 有)、降颅压治疗 (0:

无/1: 有)、降压治疗 (0: 无/1: 有)、镇静镇痛治疗 (0: 无/1: 有)、止吐养胃 (0: 无/1: 有) 和营养神经 (0: 无/1: 有) 在 7 组 mRs 下的分布做数量统计, 因为样本数量较少, 则采用 Fisher 精确检验探究与 mRs 的显著性关系。

(3) 不同位置的血肿和水肿体积对于预后的关联性模型

①数据说明

根据表 2 数据即附件 2 相关概念, 如图 5.2, 将全脑位置划分为 10 个: 大脑前动脉左侧、(Left ACA)、大脑前动脉右侧 (Right ACA)、大脑中动脉左侧 (Left MCA)、大脑中动脉右侧 (Right ACA)、大脑后动脉 (Left PCA)、大脑后动脉右侧 (Right PCA)、脑桥延髓左侧 (Left Pons/Medulla)、脑桥延髓右侧 (Right Pons/Medulla)、小脑左侧 (Left Cerebellum) 和小脑右侧 (Right Cerebellum)。

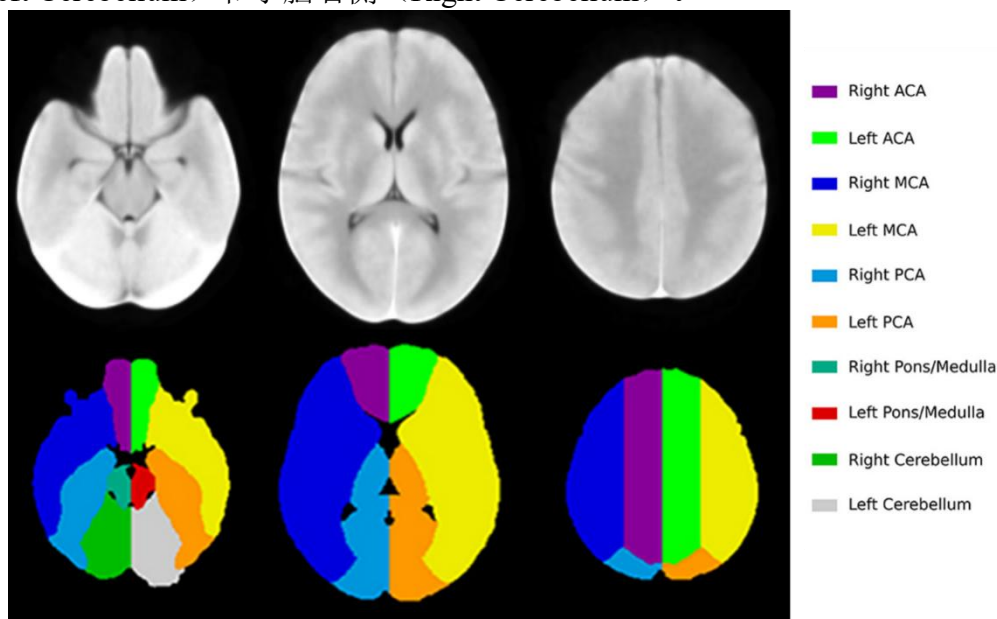


图 7.13 全脑位置分类

②数据转换

将全脑 10 个部位大脑前动脉左侧、(Left ACA)、大脑前动脉右侧 (Right ACA)、大脑中动脉左侧 (Left MCA)、大脑中动脉右侧 (Right ACA)、大脑后动脉 (Left PCA)、大脑后动脉右侧 (Right PCA)、脑桥延髓左侧 (Left Pons/Medulla)、脑桥延髓右侧 (Right Pons/Medulla)、小脑左侧 (Left Cerebellum) 和小脑右侧 (Right Cerebellum) 依次转换为 1-10 的 10 个独立因子。

③模型说明

设结果变量 Y 是具有 g 个等级 (1,2,...,g) 的有序变量, $X_1, X_2, \dots, X_m$ 为 m 个自变量。记等级为 k 的概率为 $P(Y = k|X)$, 则等级小于或等于 k 的概率为

$$P(Y \leq k|X) = P(Y = 1|X) + P(Y = 2|X) + \dots + P(Y = k|X)$$

logit 变换后得:

$$\text{logit}(P(Y \leq k|X)) = \ln \frac{P(Y \leq k|X)}{1 - P(Y \leq k|X)}$$

累积优势 Logistics 回归模型定义为:

$$\text{logit}(P(Y \leq k|X)) = \alpha_k - \sum_{i=1}^m \beta_i X_i$$

④分析方法

Step 1: 根据表 2 数据可以分析出, 每一位患者的随访次数不同, 所以采取计算体积均值的方法, 减少随访次数不同带来的差异。

Step 2: 统计不同 90mRs 分组下的患者血肿体积均值与水肿体积均值的分布, 根据正态性检验得出两组数据分布都不完全服从正态分布, 所以使用非参数检验中的 Levene 检验来检验其显著性。

Step 3: 针对 10 个不同部位开展针对性研究, 分别计算每一位患者的 10 个不同部位的血肿和水肿岁多次随访的体积均值。

Step 4: 因探究的问题是不同部位的血肿、水肿体积对于 90mRs 的影响, 而 90mRs 是一组从 0-6, 对患者功能状态和残疾程度逐渐加深的有序因子变量, 所以采用有序 Logistics 回归分析模型来分析。故将 7 个 90mRs 评分设置为从 0 到 6 的有序因子。

Step 5: 计算不同 90mRs 下的每一个大脑部位的血肿与水肿体积均值, 划分 10 个不同部位的变量为 1-10, 血肿和水肿标记为 1、2。划分后的部分数据表见表 7.4。

表 7.4 有序划分表 (部分)

Location	Category	mRs	Ave_volume
1	1	0	3168.475
1	1	1	75.684
1	1	2	2310.071
1	1	3	2012.009
1	1	4	131.301
1	1	5	3672.325
1	1	6	8105.85
1	2	0	3026.983
1	2	1	248.21
1	2	2	1959.35
1	2	3	1728.006
1	2	4	2143.803
1	2	5	2921.182
1	2	6	5321.763

并使用 R Studio 中 MASS 包的 polr 函数对位置变量 Location 和响应变量 90mRS, 将体积均值作为 weight, 按照体积均值作为权重建立有序 Logistics 回归模型。进而得到模型各参数估计值、不同部位的优势比 OR 和置信区间分布。最后进行平行性假设检验, 通过后计算以及对每一个部位的检验 p 值。

(4) 信号强度特征与形状特征对于预后的关联性模型

①数据说明

根据表 2 和表 3 数据, 可以发现表 3 记录的是每一次随访的血肿和水肿的信号强度特征与形状特征。

强度特征 (灰度特征): 基本的度量值, 反映目标区域内体素强度的分布

10Percentile、90Percentile、Energy、Entropy、InterquartileRange、Kurtosis、Maximum、MeanAbsoluteDeviation、Mean、Median、Minimum、Range、RobustMeanAbsoluteDeviation、RootMeanSquared、Skewness、Uniformity、Variance。

形状特征: 对目标区域三维形状描述

Elongation 、 Flatness 、 LeastAxisLength 、 MajorAxisLength 、 Maximum2DDiameterColumn、Maximum2DDiameterRow、Maximum2DDiameterSlice、Maximum3DDiameter、MeshVolume、MinorAxisLength、Sphericity、SurfaceArea、SurfaceVolumeRatio、VoxelVolume。

②分析方法

Step 1: 分别计算每一名患者在每一次随访时记录的 17 个强度特征和 14 个形状特征的平均值。

Step 2: 统计不同 90mRs 分组下的患者多次随访的血肿强度特征和形状特征的均值与水肿强度特征和形状特征的均值的分布。

Step 3: 根据正态性检验，计算每一组的分布数据，得出大部分数据分布都完全服从正态分布，并使用非参数检验中的 Levene 检验来检验其显著性。

7.2.3 第三小问模型求解与分析

(1) 个人史、疾病史和治疗方法对于预后的关联性模型求解与分析

①个人史对于预后的关联性

使用 R 对个人史数据进行处理与分析，采用非参数检验中的 Levene 检验检验平均年龄，使用 Fisher 精确分析检验分类年龄、性别、分类血压、吸烟和饮酒状况各变量分布的 p 值，个人史对预后的分布图以及含有检验的分布表如下表所示。

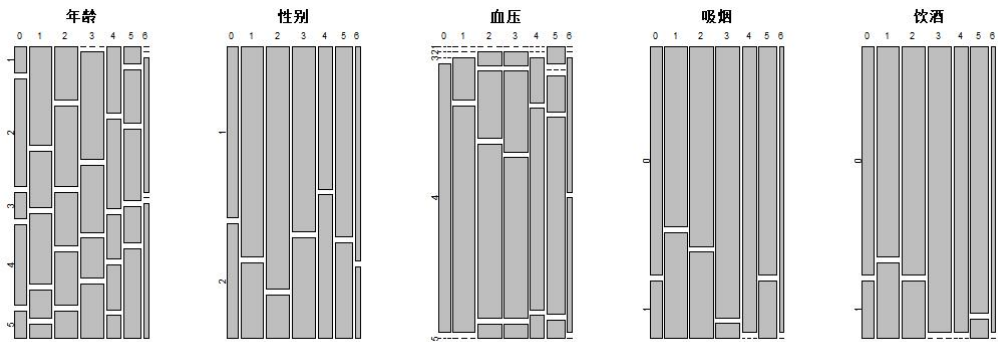


图 7.14 个人史对预后分布图
表 7.5 个人史对预后的分布表

变量	0	1	2	3	4	5	6	P value
平均年龄	63.90	57.53	62.45	66.20	61.58	69.67	74.50	0.1190
年龄								0.3163
<50	1	7	4	0	3	1	0	
50-59	4	4	6	8	4	3	0	
60-69	1	5	4	5	2	4	2	
70-79	3	2	4	3	2	2	0	
80-100	1	1	2	4	1	5	2	
性别								0.4813
男性	6	14	17	13	6	10	3	
女性	4	5	3	7	6	5	1	
血压								0.5907
NP	0	0	0	0	0	1	0	
NH	0	0	1	1	0	0	0	
H1	0	3	5	6	2	2	2	
H2	10	16	13	12	9	11	2	

SH	0	0	1	1	1	1	0	
吸烟								0.0545
是	2	7	6	1	0	5	0	
否	8	12	14	19	12	12	4	
饮酒								0.0600
是	2	5	4	0	0	1	0	
否	8	14	16	20	12	14	4	

结果与分析：个人史中的年龄、性别、血压、吸烟和饮酒因素似乎对于预后的影响在统计上都不是非常显著。然而，这并不代表它们对预后没有任何影响，可能需要更大规模的研究或其他统计方法来更准确地评估它们的影响。

②疾病史对于预后的关联性

使用 R 对疾病史数据进行处理与分析，采用 Fisher 精确分析检验高血压、卒中、糖尿病、房颤和冠心病状况各变量分布的 p 值，疾病史对预后的分布表如下表所示。

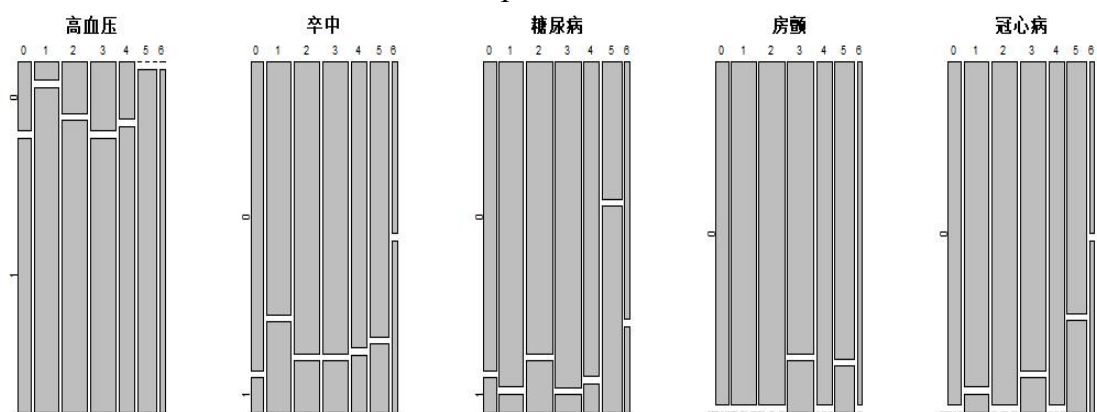


图 7.15 疾病史对预后的分布图

表 7.6 疾病史对预后的分布表

变量	0	1	2	3	4	5	6	P value
高血压								0.4338
是	8	18	17	16	10	15	4	
否	2	1	3	4	2	0	0	
卒中								0.6752
是	1	5	3	3	2	3	2	
否	9	14	17	17	10	12	2	
糖尿病								0.0010
是	1	1	3	1	1	9	1	
否	9	18	17	19	11	6	3	
房颤								0.1409
是	0	0	0	3	0	2	0	
否	10	19	20	17	12	13	4	
冠心病								0.0055
是	0	1	0	2	0	4	2	
否	10	18	20	18	12	11	2	

结果与分析：糖尿病和预后显示了糖尿病病史患者与非糖尿病病史患者在不同预后水平下的数量，p 值为 $0.0010 < 0.05$ 的阈值，这表明糖尿病与预后之间的关联性在统计上是非常显著的，可能是一个重要的因素。冠心病和预后部分显示了冠心病患者与非冠心病患者在不同预后水平下的数量。p 值为 $0.0055 < 0.05$ 的阈值，这表明冠心病与

预后之间的关联性在统计上是显著的，也可能是一个重要的因素。而疾病史中的高血压和卒中与预后的关联性不太显著。

③治疗方式对于预后的关联性

使用 R 对治疗方式数据进行处理与分析，采用 Fisher 精确分析检验脑室引流、止血治疗、降颅压治疗、降压治疗、镇静镇痛治疗、止吐养胃和营养神经治疗状况各变量分布的 p 值，治疗方式对预后的分布表如下表所示。

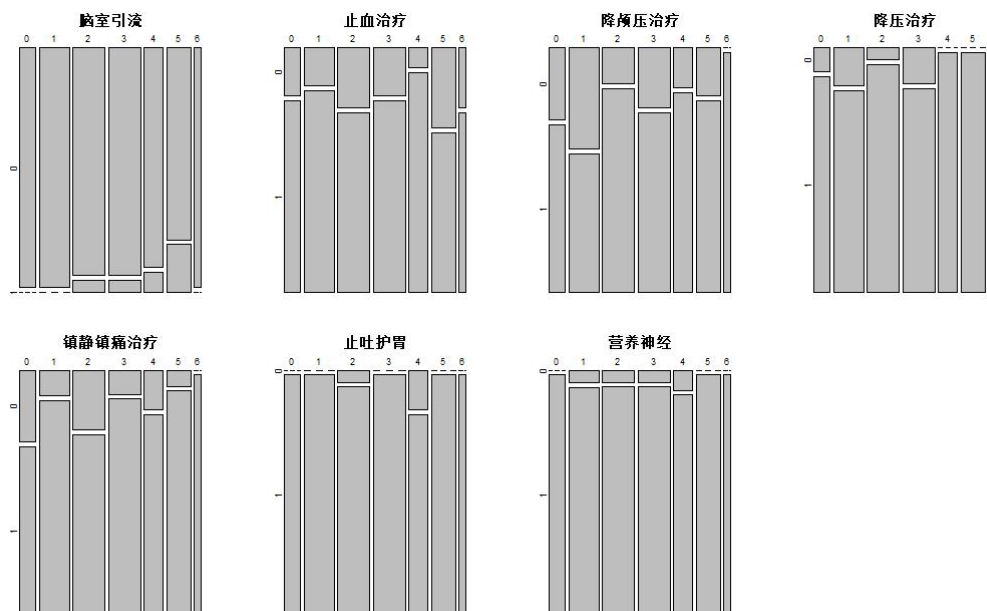


图 7.16 治疗方式对预后的分布图

表 7.7 治疗方式对预后的分布表

变量	0	1	2	3	4	5	6	P value
脑室引流								0.3303
是	0	0	1	1	1	3	0	
否	10	19	19	19	11	12	4	
止血治疗								0.7886
是	8	16	15	16	11	10	3	
否	2	3	5	4	1	5	1	
降颅压治疗								0.4713
是	7	11	17	15	10	12	4	
否	3	8	3	5	2	3	0	
降压治疗								0.5027
是	9	16	19	17	12	15	4	
否	1	3	1	3	0	0	0	
镇静、镇痛治疗								0.5347
是	7	17	15	18	10	14	4	
否	3	2	5	2	2	1	0	
止吐养胃								0.1789
是	10	19	19	20	10	15	4	
否	0	0	1	0	2	0	0	
营养神经								0.9600
是	10	18	19	19	11	15	4	
否	0	1	1	1	1	0	0	

结果与分析：治疗方式中的各项治疗措施（脑室引流、止血治疗、降颅压治疗、降压治疗、镇静镇痛治疗、止吐养胃和营养神经）与预后的关联性在这个分析中均不显著。这意味着在这个样本中，不同的治疗方式似乎没有显著影响预后的趋势。

（2）不同位置的血肿和水肿体积对于预后的关联性模型求解与分析

①分布统计分析：

表 7.8 不同位置的血肿和水肿体积对预后分布表

变量	0	1	2	3	4	5	6	P value
平均血肿 体积	24289.4 0	20377.1 3	21419.2 7	35318.3 1	39641.8 5	53313.6 8	52506.5 1	0.00013 8 ***
平均水肿 体积	20486.0 4	16744.5 1	21949.6 9	25415.5 4	43511.7 1	30564.1 4	27257.9 0	0.00423 **
ACA_R								0.0928
血肿	3168.48	75.68	2310.07	2012.01	131.30	3672.33	8105.85	
水肿	3026.98	248.21	1959.35	1728.01	2143.80	2921.18	5321.76	
PCA_R								0.264
血肿	5772.85	2147.12	3042.13	1915.90	2251.55	6376.68	7502.31	
水肿	2284.16	1311.24	1529.97	1467.3	2149.99	2520.15	1928.57	
PCA_R								0.0961
血肿	5772.85	2147.12	3042.13	1915.9	2251.55	6376.68	7502.31	
水肿	2284.16	1311.24	1529.97	1467.3	2149.99	2520.15	1928.57	
PonsMed_R								0.474
血肿	172.97	147.19	171.16	37.40	46.58	281.14	268.94	
水肿	40.10	34.45	77.86	32.75	61.14	236.31	71.50	
Cere_R								0.353
血肿	338.07	11.02	0.05	0	0	171.26	542.75	
水肿	57.06	1.73	9.50	0	0	282.30	5.00	
ACA_L								0.0283 *
血肿	221.01	1098.24	480.59	1405.8	1224.92	3981.36	5883.38	
水肿	617.61	1502.99	1071.23	1227.84	4111.46	1991.93	1967.25	
MCA_L								0.0415 *
血肿	3407.36	6829.63	6657.94	12527.2 0	18238.8 0	6020.89	7371.66	
水肿	5937.03	4856.73	8323.72	9404.70	18175.0 0	4373.11	6050.55	
PCA_L								0.0821
血肿	1536.36	2415.93	1233.10	4019.74	5338.29	5781.57	4009.91	
水肿	900.33	1062.57	1837.76	1874.12	3739.37	1121.72	858.95	
PonsMed_L								0.121
血肿	166.02	115.40	129.67	249.15	562.50	306.20	233.12	
水肿	35.44	6.78	64.20	77.73	358.25	105.66	4	
Cere_L								0.56
血肿	89.77	16.97	10.45	15.67	0.08	247.53	111.25	
水肿	2.5	0	0.55	13.4	5.47	156.63	0	

根据 90mRs 的 0-6 分，分类为

0: 没有症状，没有残疾。

- 1: 没有明显的残疾，能够独立进行日常活动。
- 2: 有轻度残疾，能够自理，但在活动中存在一些限制。
- 3: 有中度残疾，需要一定程度的帮助和照顾，但能够坐立或站立。
- 4: 有中重度残疾，需要全天候照顾和帮助，无法行走或自理。
- 5: 完全依赖他人，不能进行任何活动，床上活动有困难。
- 6: 死亡。

结果与分析：根据不同位置的血肿和水肿体积对预后分布表，可以得到平均血肿体积部分显示了不同预后水平下的平均血肿体积，并且给出了 p 值 $=0.000138 < 0.05$ ，带有三个星号 (***)，表示平均血肿体积与预后之间的关联性在统计上是非常显著的。

而平均水肿体积和预后部分显示了不同预后水平下的平均水肿体积，且 p 值 p 值为 $0.00423 < 0.05$ ，带有两个星号 (**)，表示平均水肿体积与预后之间的关联性在统计上是显著的，但不如血肿体积显著。说明

接下来的部分显示了不同脑部位置（如 ACA_R、PCA_R、PCA_R、PonsMed_R、Cere_R、ACA_L、MCA_L、PCA_L、PonsMed_L、Cere_L）下的血肿和水肿体积与预后之间的关联性以及相应的 p 值。经分析发现 ACA_L 的 p 值 $=0.0283$ ，MCA_L 的 p 值 $=0.0415$ ，都小于 0.05 。即表示大脑前动脉左侧和大脑中动脉左侧对于预后的影响是显著的，说明这两个部位的血肿/水肿体积对于患者的预后有着明显的影响，应予以重视。

②建立有序 Logistics 回归模型分析：

即各分类之间是有序的，可以考虑使用有序 Logistics 回归来处理，以便更好的建立因变量和自变量之间的联系。

使用 R 包中的 MASS 包，利用 polr() 函数建立 Logistics 回归模型，得到输出信息为：

```
Call:
polr(formula = mRs ~ Location + Category, data = p mrs loc, weights = Ave volume)
```

Coefficients:

	Value	Std. Error	t value
Location2	-0.50869	0.011177	-45.513
Location3	-0.66507	0.013827	-48.101
Location4	-0.46348	0.046768	-9.910
Location5	0.29690	0.053900	5.508
Location6	0.08835	0.014833	5.956
Location7	-0.83475	0.011372	-73.401
Location8	-0.60339	0.013612	-44.329
Location9	-0.62089	0.035002	-17.739
Location10	0.16936	0.066485	2.547
Category2	-0.32362	0.005488	-58.968

Intercepts:

	Value	Std. Error	t value
0 1	-2.8554	0.0116	-246.4484
1 2	-2.1509	0.0111	-193.0474
2 3	-1.5837	0.0109	-144.8193
3 4	-0.9754	0.0108	-90.1416
4 5	-0.1641	0.0107	-15.3134
5 6	0.8331	0.0107	77.7041

Residual Deviance: 1594489.38

AIC: 1594521.38

即各参数估计值为： $\beta_1 = -0.50869$ ， $\beta_2 = -0.66507$ ， $\beta_3 = -0.46348$ ， $\beta_4 = 0.29690$ ， $\beta_5 = 0.08835$ ， $\beta_6 = -0.83475$ ， $\beta_7 = -0.60339$ ， $\beta_8 = -0.62089$ ， $\beta_9 = 0.16936$ ， $\beta_{10} = -0.32362$ 。

可以由自变量的回归系数计算出对应的优势比 OR 和置信区间统计表如下：

表 7.9 变量间 OR 与置信区间统计表

自变量	OR	95%下限	95%上限	P value
Location2	0.6012827	0.5883583	0.6144085	<0.01
Location3	0.5142356	0.5010506	0.5278293	<0.01
Location4	0.6290934	0.5751505	0.6880092	<0.01
Location5	1.3456858	1.2129817	1.4931367	<0.01
Location6	1.0923731	1.0624927	1.1231381	<0.01
Location7	0.4339833	0.4244830	0.4418341	<0.01
Location8	0.5469564	0.5326709	0.5616231	<0.01
Location9	0.5374657	0.5025144	0.5752285	<0.01
Location10	1.1845465	1.0410703	1.3472484	<0.01
Category2	0.7235226	0.7158278	0.7313147	<0.01

从表中可以看出 Location5、Location6 和 Location10 的 OR 值都大于 1，且它们的 95%置信区间都不包含 1，并且所有不为的检验性 P 值都小于 0.01。表明 10 个部位对于 90mRS 都有明显的影响，但是小脑左侧 Cere_R、前动脉左侧 ACA_L 和小脑右侧 Cere_L 对 90mRs 的影响程度显著高于其他部位。

对模型的平行性假设做检验，指定函数 vglm()里参数 parallel 为 TRUE 和 FALSE 建立两个模型，并使用 VGAM 包的函数 lrtest()进行似然比检验，检验结果如下：

Likelihood ratio test				
Model 1: mRs ~ Location + Category				
Model 2: mRs ~ Location + Category				
#Df	LogLik	Df	Chisq	Pr(>Chisq)
1	824	-272.43		
2	774	-272.43	-50	0
				1

结果表明，两个模型的差异没有统计学意义（p 大于 0.05），因此可认为模型满足平行性假设条件，适用于有序 Logistics 回归。

（2）信号强度特征与形状特征对于预后的关联性模型求解与分析

使用 R 对信号强度特征与形状特征数据进行处理与分析，检验各项数据的正态性，统计出 17 个信号强度特征与 14 个形状特征下的平均血肿和水肿的特征值，并采用非参数检验中的 Levene 检验来各变量分布的 p 值，信号强度特征与形状特征对于预后的分布表如下表所示。

表 7.10 形状特征对预后的分布表

变量	0	1	2	3	4	5	6	P value
Elongation								0.0147
血肿	0.629	0.572	0.653	0.608	0.726	0.607	0.608	
水肿	0.589	0.646	0.671	0.632	0.733	0.683	0.615	
Flatness								0.00191
血肿	0.438	0.394	0.428	0.414	0.536	0.441	0.425	
水肿	0.367	0.373	0.419	0.382	0.483	0.435	0.405	
LeastAxis Length								4.01e-06
血肿	16.913	13.611	15.157	19.485	22.716	24.109	25.895	
水肿	22.819	18.391	22.236	25.563	30.264	28.541	26.553	
MajorAxis								0.00117

Length								
血肿	38.146	31.509	33.017	45.144	40.572	55.665	64.080	
水肿	69.000	50.382	52.096	65.229	62.039	67.989	67.750	
Maximum2D Diameter Column								0.000215
血肿	35.089	29.336	32.024	39.454	42.504	49.479	57.522	
水肿	45.453	42.808	46.870	53.985	55.375	58.921	58.455	
Maximum2D DiameterRow								0.000746
血肿	39.8700	30.166	34.289	45.710	44.025	54.562	57.677	
水肿	57.719	44.752	49.900	59.647	58.612	62.953	58.020	
Maximum2D Diameter Slice								7.04e-05
血肿	37.707	32.293	34.819	48.882	45.561	58.734	70.715	
水肿	58.575	44.700	49.376	64.430	59.191	64.818	67.505	
Maximum3D Diameter								
血肿	47.691	38.202	40.707	56.273	51.633	68.727	81.312	
水肿	58.575	44.700	49.376	64.430	59.191	64.818	67.505	
MeshVolume								0.000399
血肿	11428.756	8893.664	10554.15	16217.514	18677.327	26625.615	25787.432	
水肿	8191.018	6929.561	8948.512	9701.838	17378.782	11886.049	10881.702	
MinorAxis Length								0.00123
血肿	25.107	20.340	23.330	28.915	30.772	33.940	36.577	
水肿	37.259	33.543	36.194	42.141	46.105	45.841	40.425	
Sphericity								0.155
血肿	0.584	0.554	0.591	0.550	0.604	0.552	0.540	
水肿	0.336	0.342	0.321	0.294	0.298	0.284	0.305	
SurfaceArea								6.77e-05
血肿	3396.741	2793.019	3334.201	5014.444	5138.246	7412.479	8048.467	
水肿	5385.050	4322.139	5920.692	6834.664	10190.707	8654.771	7761.135	
Surface VolumeRatio								0.211
血肿	0.3730	0.4036	0.467	0.3350	0.3241	0.346	0.332	
水肿	1.115	1.139	0.966	1.001	0.795	0.908	0.777	
VoxelVolume								0.00039
血肿	11478.295	8933.189	10598.692	16280.285	18739.427	26713.247	25891.688	
水肿	8344.0	7047.95	9116.48	9902.21	17595.7	12122.	11087.7	

29	1	0	3	83	559	75
----	---	---	---	----	-----	----

结果与分析：根据上表，形状特征中的球度（Sphericity）和表面积体积比率（SurfaceVolumeRatio）对于预后的影响在统计上都不是非常显著（ $p=0.115$ 和 $p=0.211$ ）。但是其他的特征：Elongation、Flatness、LeastAxisLength、MajorAxisLength、Maximum2DDiameterColumn、Maximum2DDiameterRow、Maximum2DDiameterSlice、Maximum3DDiameter、MeshVolume、MinorAxisLength、SurfaceArea、VoxelVolume 对于预后的影响非常显著。

表 7.11 信号强度特征对预后的分布表

变量	0	1	2	3	4	5	6	P value
10Percentile								0.519
血肿	89.1170	84.933	94.022	95.410	95.756	96.157	93.610	
水肿	30.766	32.624	34.227	34.899	30.423	28.114	24.307	
90Percentile								0.353
血肿	160.642	151.772	165.43 3	169.91 2	174.19 1	174.87 3	168.25 7	
水肿	30.766	32.624	34.227	34.899	30.423	28.114	24.307	
Energy								0.0046
血肿	2726380 30	2140041 16	244302 173	372270 645	421647 407	599049 768	490506 904	
水肿	1521190 6	1473154 6	215761 21	246319 97	355672 18	220637 92	176364 11	
Entropy								0.525
血肿	3.183	3.000	3.201	3.181	3.409	3.434	3.495	
水肿	3.021	2.997	2.943	2.921	2.929	2.914	3.017	
Interquartile Range								0.436
血肿	45.398	41.975	44.816	45.454	48.668	48.382	44.677	
水肿	12.177	11.84	12.751	12.776	12.857	13.359	13.292	
Kurtosis								0.368
血肿	1.902	1.987	1.931	2.154	2.036	2.105	2.295	
水肿	3.165	3.068	3.176	3.190	3.355	3.432	3.277	
Maximum								0.113
血肿	197.654	192.031	202.50 3	220.04 6	215.98 1	222.52 7	221.27 0	
水肿	76.586	75.164	83.685	86.442	82.219	83.124	77.175	
MeanAbsolute Deviation								0.295
血肿	23.101	21.574	23.027	23.854	25.028	25.048	23.520	
水肿	76.586	75.164	83.685	86.442	82.219	83.124	77.175	
Mean								0.39
血肿	124.662	117.199 5	129.43 5	131.49 6	135.61 0	135.15 2	129.68 7	
水肿	43.113	44.113	46.688	47.476	43.187	41.324	37.272	
Median								0.336
血肿	124.368	115.447	128.94 0	129.91 1	136.14 5	134.49 2	128.06 0	
水肿	43.588	44.417	47.030	47.767	43.684	41.733	37.552	

Minimum								0.268
血肿	68.399	66.1405	72.651	71.782	72.057 5	70.261	67.372	
水肿	6.272	9.792	6.747	7.194	-0.716	-3.276	-5.325	
Range								0.0101
血肿	129.256	125.891	129.84 9	148.26 7	143.92 2	152.26 6	153.89 7	
水肿	70.314	65.371	76.936	79.247 5	82.935	86.398	82.497	
RobustMean AbsoluteDeviati on								0.402
血肿	18.199	16.854	18.011 0	18.367	19.550	19.462	18.057	
水肿	5.117	4.895	5.284	5.313	5.353	5.550	5.517	
RootMean Squared								0.376
血肿	127.626 0	119.972	132.28 3	134.57	138.78	138.36 8	132.69 0	
水肿	44.227	45.079	47.763	48.574	44.357	42.722	38.742	
Skewness								0.057
血肿	0.152	0.254	0.149	0.239	0.073	0.137	0.227	
水肿	-0.221	-0.155	-0.182	-0.137	-0.280	-0.203	-0.177	
Uniformity								0.0136
血肿	0.080	0.079	0.082	0.089	0.084	0.089	0.097	
水肿	0.131	0.126	0.135	0.140	0.150	0.148	0.145	
Variance								0.673
血肿	872.970	802.443	854.41 2	927.79 4	932.67 0	930.39 7	795.08 2	
水肿	98.834	89.400	102.66 9	102.43 4	99.763	110.94 6	103.72 7	

结果与分析：根据上表，信号强度特征中能量（Energy）、范围（Range）和均匀性（Uniformity）的对于预后的影响在统计上非常显著，其 p 值分别为：p=0.0046、p=0.0101 和 p=0.0136）。但是其他的特征：10Percentile、90Percentile、Entropy、InterquartileRange、Kurtosis、Maximum、MeanAbsoluteDeviation、Mean、Median、Minimum、RobustMeanAbsoluteDeviation、RootMeanSquared、Skewness、Variance 对于预后的影响在统计上并不显著。

7.2.4 对临床决策的建议

根据上述对 3 个子问题的建模与分析，可以得出本题的相关性结论为：患者疾病史中的糖尿病和冠心病对 90mRs 的影响显著，患者 CT 平扫后的观测到的血肿体积与水肿体积对 90mRs 的影响非常显著，而在大脑部位上的所有部位对 90mRs 都有影响，但是大脑前动脉左侧和大脑中动脉左侧的血肿/水肿体积对 90mRs 的影响明显，并且小脑左侧 Cere_L、前动脉左侧 ACA_L 和小脑右侧 Cere_R 对 90mRs 的影响程度显著高于其他部位，对于 CT 平扫后获取的形状特征而言，Elongation、Flatness、LeastAxisLength、MajorAxisLength、Maximum2DDiameterColumn、Maximum2DDiameterRow、Maximum2DDiameterSlice、Maximum3DDiameter、MeshVolume、MinorAxisLength、

SurfaceArea、VoxelVolume 对于预后的影响非常显著,在信号强度特征中的能量(Energy)、范围(Range)和均匀性(Uniformity)的对于预后的影响在统计上非常显著。

结合一些相关临床诊疗的案例和本题结论,可以提出如下建议:

疾病史的重要性: 疾病史中的糖尿病和冠心病对脑卒中患者的预后具有显著影响。因此,医生应该在评估患者的脑卒中风险和预后时,重点考虑他们的疾病史。对于有糖尿病和冠心病史的脑卒中患者,应当特别关注他们的病情,并采取更积极的干预措施。

影像特征的价值: 血肿和水肿的体积、位置,以及形状特征和信号强度特征都对脑卒中预后具有显著影响。这强调了影像学在脑卒中诊断和管理中的重要性。医生应充分利用 CT 等影像学工具,以获取最全面的病灶信息。特别是对于大脑前动脉左侧和大脑中动脉左侧的血肿/水肿体积,以及小脑左侧 Cere_R、前动脉左侧 ACA_L 和小脑右侧 Cere_L 这些区域,还有能量(Energy)、范围(Range)和均匀性(Uniformity)的信号强度特征,在临床应予以特别关注。

针对性治疗: 由于病灶的形状特征和信号强度特征对预后具有显著影响,这表明脑卒中患者需要个性化的治疗方案。医生应根据每个患者的具体病灶特征来制定最适合他们的治疗计划。

预防策略: 由于糖尿病和冠心病有显著影响,这也强调了预防这些疾病的重要性。医生和公共卫生专家应积极推广健康生活方式,以预防糖尿病和冠心病的发生,从而降低脑卒中的风险。

八、模型评价与推广

8.1 模型的优点

针对问题一，问题一中我们通过几个模型的对比实验结果选择了使用 XGBoost 模型，它的准确率最高，表现最好。在血肿扩张概率预测中具有很高的应用价值。XGBoost 的性能也相当高，比传统的梯度提升树算法实现的更快。适用于本题大量不同类别数量的样本，它也可以自动确定最终重要的特征，并且减轻出现过拟合的问题。

针对问题二，第一、二问中，采用分段式多项式拟合可以适应复杂的数据模式，可以更好地捕捉数据集可以清晰地解释每个区间内数据的拟合效果和趋势，更精确地拟合数据。而在第三问中主成分分析可以将高维数据集转化为低维空间，从而减少特征的数量。这有助于简化数据集的复杂性。而 K-means 算法相对简单，易于理解和实现，计算效率高，通过最小化簇内点之间的平方距离来进行聚类，能够很好地将相似的数据点聚在一起。第四问根据统计得到的条形图进行数据探索，建立多元线性回归模型来探究多种治疗方法对于患者的脑内的血肿和水肿的影响。本题中对血肿的关系模型效果最好，有很好的的解释性。并且多元线性回归模型模型具有良好的可预测性和可视化效果。

针对问题三，第一、二问中，使用了两种方法来对患者的数据信息进行特征筛选，分别是方差筛选法和互信息法结合、距离相关系数法和递归特征消除法结合，方差筛选法简单易实施非常简单、并且计算速度较快，不需要复杂的模型或算法支持；而信息法考虑特征与目标变量之间的非线性关系，不仅仅局限于线性相关性；距离相关系数法相对简单且考虑了特征与目标变量之间的相关性；递归特征消除法它通过反复训练模型并剔除最不重要的特征来选择最佳特征子集，考虑特征之间的相互作用自动化特征选择。然后使用了随机森林算法、支持向量机算法、XGBoost 算法、LightGBM 算法来进行预测，它们都有高准确性、可处理高维度数据、对小样本数据有效、鲁棒性强、可以处理非线性问题的优点。同时使用随机搜索、网格搜索来进行超参数调优，它们的主要优点有：简单易实现、可以保证找到全局最优解（如果存在）而不会陷入局部最优解、可解释性强。第三问中根据变量特征，分别采用不同的相关性分析模型与方法，包括 Fisher 精确检验、Levene 检验、有序 Logistics 回归模型和单因素方差分析。Fisher 精确检验适用于如本题的小样本数据，不受正态分布和方差齐性的限制。Levene 检验作为一种非参数方法，适用范围更广，可以降低对数据正态性的敏感度。有序 Logistics 回归模型针对 90mRs 这种有序分类因变量，可以更有效的建模，能够捕捉自变量对不同分类的影响，也可以更好的解释每个自变量对因变量的影响。对于单因素方差分析而言，能够用于多组数据间均值的比较，减少了多重对比的问题，最重要的是可以方便地检验不同因素对因变量的影响。

8.2 模型的不足

针对问题一，在使用 XGBoost 模型时，需要调整很多超参数，必须进行仔细的调整。对于特别大规模的数据集来说，XGBoost 模型的树结构需要消耗大量内存。

针对问题二，分段式多项式拟合可能会面临过拟合的风险，特别是当拟合函数的阶数很高时，容易出现在训练集上的拟合效果较好，但在未知数据上的预测效果较差的情况。降维过程中，PCA 会舍弃一部分维度较小的特征，这可能会导致一定的信息损失。因此，在选择降维的维度时需要权衡维度的减少和信息的保持。使用多元线性回归模型时，若自变量之间出现高度相关性时会不稳定，并且对于异常值较为敏感。而且当引入过多的自变量时可能出现过拟合问题，导致性能下降。

针对问题三，传统机器学习算法计算复杂度高，训练时间较长，超参数调优中，往往资源消耗较大，由于搜索需要遍历参数空间中的所有可能组合或者没有相应的先验条件，当参数数量或参数取值范围较大时，搜索过程可能需要很长时间和大量计算资源，搜索效率低。Fisher 精确检验的计算复杂度高，难以运用到大样本数据中。Levene 检验对于极端值很敏感，且不适用于顺序数据。有序 Logistics 回归必须针对有序变量，当自变量较多时，由于本身多次回归的特点，模型复杂度会提升，更难解释结果。单因素方差分析对于非正态数据和数据的方差齐性要求较高，有时需要数据转换或使用非参数方法。

8.3 模型的推广

针对问题一，XGBoost 模型是一个强大的机器学习模型，它可以使用分布式计算训练大规模数据集。为疾病预测提供更加准确的预测结果。

针对问题二，前两小问中，传统方法（如线性回归、逻辑回归）通常采用单一的全局模型来对整个数据集进行建模和预测。这意味着传统方法无法很好地处理数据集中的非线性关系或复杂的模式。当数据集的关系不是简单的线性关系时，传统方法的预测性能可能不佳。相比之下，分段式多项式（如分段线性回归、分段多项式回归）将数据集划分为多个子集，并在每个子集上建立一个局部模型。每个局部模型可以更好地适应数据集的特定区域，因此能够捕捉到非线性关系或复杂模式。分段式多项式的预测能力通常比传统方法更强，特别是在数据集存在非线性关系或复杂模式的情况下。第四小问中的多元线性回归模型，治疗方法之间可能存在相互作用，即它们的组合对体积的影响不同于单独使用时的效应。可以添加治疗方法之间的交互作用项，以更好地捕捉这种复杂性。

针对问题三，前两小问中，使用了集成学习的方法通过结合多个分类器的预测结果，能够减少模型的偏差和方差，从而提高预测性能。第三问中使用的有序 Logistics 回归模型中，可以考虑水肿和水肿体积的线性和交互作用效应，将其作为自变量引入模型。并且如果水肿和水肿体积对 90mRs 的影响不是线性的，可以添加非线性项，例如多项式项或光滑曲线，以增强模型的性能。

参考文献

- [1] 李立新,李琳琳.缺血性和出血性脑卒中危险因素分析[J].郑州大学学报(医学版),2010,45(02):313-315.DOI:10.13705/j.issn.1671-6825.2010.02.037.
- [2] 李倩,刘芸宏,吴晓慧等.基于决策树和 Logistic 回归预测出血性脑卒中手术后医院感染风险[J].中华医院感染学杂志,2021,31(23):3556-3561.
- [3] 吴海燕,陈晓磊,范国轩.一种自适应核 SMOTE- SVM 算法用于不平衡数据分类[J].北京化工大学学报(自然科学版), 2023, 50(02): 97-104. DOI:10.13543/j.bhxbzr. 2023.02.012.
- [4] 刘巧红,马雨生,蔡雨晨.基于 XGBoost 算法的糖尿病分类预测模型及应用[J].现代仪器与医疗,2023,29(04):1-6+11.
- [5] 赖海芳,顾琳,纵亚等.采用多层感知器神经网络构建亚急性期缺血性脑卒中患者短期预后的预测模型[J].中国康复理论与实践,2022,28(03):335-339.
- [6] 曹佳悦,罗冬梅.基于 Null Importance 与 GS-LGBM 的糖尿病视网膜病变因素分析与风险预测[J].中国医学物理学杂志,2023,40(08):1033-1038.
- [7] 郭虎升. 支持向量机的优化建模方法研究[D].山西大学,2014.
- [8] 吕鸿杰.脑卒中发病概率预测模型评价研究[J].神经损伤与功能重建,2011,6(03):196-198.
- [9] 张涵夏.适用于线性回归和逻辑回归的场景分析[J].自动化与仪器仪表,2022(10):1-4+8.DOI:10.14016/j.cnki.1001-9227.2022.10.001.
- [10] 刘霞,王运锋.基于最小二乘法的自动分段多项式曲线拟合方法研究[J].科学技术与工程,2014,14(03):55-58.
- [11] 莫小琴.基于最小二乘法的线性与非线性拟合[J].无线互联科技,2019,16(04):128-129.
- [12] 王建仁,马鑫,段刚龙.改进的 K-means 聚类 k 值选择算法[J].计算机工程与应用,2019,55(08):27-33.
- [13] 林海明,杜子芳.主成分分析综合评价应该注意的问题[J].统计研究,2013,30(08):25-31.DOI:10.19343/j.cnki.11-1302/c.2013.08.004.
- [14] 姚旭,王晓丹,张玉玺等.特征选择方法综述[J].控制与决策,2012,27(02):161-166+192.DOI:10.13195/j.cd.2012.02.4.yaox.013.
- [15] 李欣海.随机森林模型在分类与回归分析中的应用[J].应用昆虫学报,2013,50(04):1190-1197.
- [16] 奉国和.SVM 分类核函数及参数选择比较[J].计算机工程与应用,2011,47(03):123-124+128.

附录

附录 1

基于 XGBoost 和几种模型对比第一问的 b 求解代码

```
warnings.filterwarnings("ignore")
# 读取数据
df1 = pd.read_excel('data\\表 1-患者列表及临床信息.xlsx')
df2 = pd.read_excel('data\\表 2-患者影像信息血肿及水肿的体积及位置.xlsx')
df4 = pd.read_excel('data\\附表 1-检索表格-流水号 vs 时间.xlsx')
# 将附表的检查编号和检查时间转换成 DataFrame
ls1, ls2, ls3, ls4, ls5, ls6 = [], [], [], [], [], []
for value in df4.values:
    ID, freq = value[0], value[1]
    for f in range(freq):
        ls1.append(ID)
        ls2.append(freq)
        ls3.append(value[f*2+2])
        ls4.append(value[f*2+3])
        ls5.append(f'第{f}次检查')
        ls6.append(value[3])
df4 = pd.DataFrame({'ID': ls1, 'first_code': ls6, '重复次数': ls2, 'code': ls4, 'time': ls3, '
检查次数': ls5})
df4['code'] = df4['code'].astype('int64')
# 将附件 1 转换为易读的 DataFrame
ls1, ls2, ls3 = [], [], []
for value in df2.values:
    ID, code = value[0], value[1]
    freq = df4[df4.first_code == value[1]]['重复次数'].values[0]
    for f in range(min(freq, 9)):
        ls1.append(ID)
        ls2.append(code)
        ls3.append(f'第{f}次检查')
tem = pd.DataFrame({'ID': ls1, 'first_code': ls2, '检查次数': ls3})
time_code = []
for value in df2.values:
    freq = df4[df4.first_code == value[1]]['重复次数'].values[0]
    for f in range(min(freq, 9)):
        time_code.append(value[23*f+1])
tem['code'] = time_code
columns = df2.columns
for delta in range(22):
    ls = []
```

```

for value in df2.values:
    freq = df4[df4.first_code == value[1]]['重复次数'].values[0]
    for f in range(min(freq, 9)):
        ls.append(value[23*f+2+delta])

tem[columns[2+delta]] = ls
df2 = tem
df2 = pd.merge(df4[['code', 'time']], df2, on='code')
# 附件 3
df5 = pd.read_excel('data\\表 3-患者影像信息血肿及水肿的形状及灰度分布.xlsx')
df5['入院首次影像检查流水号'] = df5['流水号']
df2['入院首次影像检查流水号'] = df2['code']
pd.set_option('display.max_columns', None) # 展示所有列
# 合并数据集
dataset = pd.merge(df1, df2, on='入院首次影像检查流水号')
# 选择特征列
columns = ['年龄', '性别', '脑出血前 mRS 评分', '高血压病史', '卒中病史', '糖尿病史',
            '房颤史', '冠心病史', '吸烟史', '饮酒史',
            '发病到首次影像检查时间间隔', '脑室引流', '止血治疗', '降颅压治疗',
            '降压治疗', '镇静、镇痛治疗', '止吐护胃', '营养神经',
            'HM_volume', 'HM_ACA_R_Ratio', 'HM_MCA_R_Ratio',
            'HM_PCA_R_Ratio', 'HM_Pons_Medulla_R_Ratio',
            'HM_Cerebellum_R_Ratio', 'HM_ACA_L_Ratio', 'HM_MCA_L_Ratio',
            'HM_PCA_L_Ratio',
            'HM_Pons_Medulla_L_R_Ratio', 'HM_Cerebellum_L_Ratio',
            'ED_volume', 'ED_ACA_R_Ratio',
            'ED_MCA_R_Ratio', 'ED_PCA_R_Ratio',
            'ED_Pons_Medulla_R_Ratio', 'ED_Cerebellum_R_Ratio',
            'ED_ACA_L_Ratio', 'ED_MCA_L_Ratio', 'ED_PCA_L_Ratio',
            'ED_Pons_Medulla_L_Ratio',
            'ED_Cerebellum_L_Ratio']
dataset = dataset[columns]
# 处理性别特征
dataset['性别'] = dataset['性别'].apply(lambda x: 1 if x == '男' else 0)
# 处理血压特征
dataset[['收缩压', '舒张压']] = dataset['血压'].str.split('/', expand=True).astype(int)
dataset.drop('血压', axis=1, inplace=True)
train = dataset[:100]
# 读取结果文件
result = pd.read_excel('result4.xlsx')
# 提取标签
label = result['是否扩张'].values[:100]
# 使用 SMOTE 进行过采样
sm = SMOTE(random_state=0)

```

```

xres, yres = sm.fit_resample(train.values, label)
xtrain_res, xvalid_res, ytrain_res, yvalid_res = train_test_split(xres, yres,
random_state=400, test_size=0.2)
def plot_roc(models, names, dataset_type='train'):
    plt.figure(figsize=(10, 8), dpi=80, facecolor='w')
    plt.xlim((-0.01, 1.02))
    plt.ylim((-0.01, 1.02))
    plt.xticks(np.arange(0, 1.1, 0.1))
    plt.yticks(np.arange(0, 1.1, 0.1))
    line_colors = ['m']
    if dataset_type == 'test':
        for model, name, color in zip(models, names, line_colors):
            ytest_prob = model.predict_proba(xvalid_res)[:, 1]
            fpr, tpr, _ = roc_curve(yvalid_res, ytest_prob)
            plt.plot(fpr, tpr, color=color, lw=2, label=f'{name}', linewidth=3)
    else:
        for model, name, color in zip(models, names, line_colors):
            ytrain_prob = model.predict_proba(xtrain_res)[:, 1]
            fpr, tpr, _ = roc_curve(ytrain_res, ytrain_prob)
            plt.plot(fpr, tpr, color=color, lw=2, label=f'{name}', linewidth=3)
    plt.legend(loc='upper left', fontsize=15)
    plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
    plt.xlabel('假阳率', fontsize=18, fontweight='bold')
    plt.ylabel('召回率', fontsize=18, fontweight='bold')
    plt.tick_params(labelsize=23)
    plt.title('受试者工作特征曲线', fontsize=20, fontweight='bold')
    plt.savefig(f'问题 1\\roc_auc({dataset_type}).png', dpi=500)
    plt.show()
plot_roc([model[4]], ['LR'], 'test')
plot_roc([model[4]], ['LR'], 'train')

```

附录 2

分段多项式拟合代码

```

import pandas as pd
import matplotlib.pyplot as plt

# 读取数据
AllList = pd.read_excel('第二问整合后全体流水号数据.xlsx')
Numbers=AllList.iloc[:, 0]

```

```

# 读取数据
data = pd.read_excel('水肿体积随时间.xlsx') # X=data['时间进程']
Y=data['水肿体积']

# 绘制散点图
plt.figure(dpi=600)
plt.scatter(data['时间进程'], data['水肿体积'],s=5)

# 添加标题和轴标签
plt.title('Scatter Plot')
plt.xlabel('X')
plt.ylabel('Y')
plt.xlim(0, 400)
# 显示图形
plt.show()

import numpy as np
import matplotlib.pyplot as plt

# 输入数据
x = X
y = Y

combined = list(zip(x, y))
combined_sorted = sorted(combined, key=lambda tup: tup[0])

# 获取排序后的 x 和 y
x = [tup[0] for tup in combined_sorted]
y = [tup[1] for tup in combined_sorted]

# 分段点的位置
breakpoints = np.arange(0, len(x), 10)

# 分段拟合
coefficients = []
for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    segment_y = y[breakpoints[i]: breakpoints[i + 1]]

    # 低阶多项式拟合
    p = np.polyfit(segment_x, segment_y, 4)

    # 将每个分段的拟合系数存储在数组中

```

```

        coefficients.append(p)

# 预测值
predictions = np.zeros_like(y)
for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    predictions[breakpoints[i]: breakpoints[i + 1]] = np.polyval(coefficients[i],
segment_x)

# 残差
residuals = y - predictions

# 绘制原始数据点和拟合曲线
plt.scatter(x, y, c='red', label='Data')
for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    segment_y = np.polyval(coefficients[i], segment_x)
    #plt.plot(segment_x, segment_y, label='Segment {}'.format(i + 1))

plt.plot(x, predictions, label='Predictions')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()

plt.show()

# 计算残差
residuals = y - predictions

# 计算平均残差
mean_residual = np.mean(residuals)

sorted_indices = np.argsort(X)

predicted_values_original = [None] * len(x)
for i, tup in enumerate(combined_sorted):
    original_index = combined.index(tup)
    predicted_values_original[original_index] = predictions[i]

residuals_values_original = [None] * len(x)
for i, tup in enumerate(combined_sorted):
    original_index = combined.index(tup)
    residuals_values_original[original_index] = residuals[i]

```

```

# 创建 DataFrame
data = pd.DataFrame({'患者号':Numbers,'x': X, 'y': Y,'原顺序拟合值':predicted_values_original,'残差': residuals_values_original })

# 添加平均残差到 DataFrame
data.loc[len(data)] = ['-','-','-','- ',mean_residual]

# 保存到 Excel 文档
data.to_excel('residuals.xlsx', index=False)

Table = AllList.to_numpy()
UniqueNums = np.unique(Table[:, 0])

def remove_values(arr, value):
    mask = arr == value
    idx = np.argmax(mask) # 获取第一个匹配值的索引
    if idx == 0:
        return arr
    else:
        return arr[:idx]
UniqueNums=remove_values(UniqueNums,'sub101')

print(len(UniqueNums))
ErrorA = np.zeros(len(UniqueNums))

for i in range(len(UniqueNums)):
    a = np.nonzero(Table[:, 0] == UniqueNums[i])[0]
    print(a)
    error=0.0
    for j in a:
        error=error+(predicted_values_original[j] - Y[j])
    ErrorA[i] = (error)/len(a)

# 创建 DataFrame
print('平均残差为: ',ErrorA[i]/len(ErrorA))
data1 = pd.DataFrame({'残差': ErrorA})

# 保存到 Excel 文档
data1.to_excel('averageresiduals.xlsx', index=False)

```

附录 3

不同类别散点图显示代码

```
import pandas as pd
import matplotlib.pyplot as plt
# 设置字体为 SimHei
plt.rcParams['font.family'] = 'SimHei'

# 设置字体列表, 包含 SimHei 和 Arial
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial']
# 读取四个 Excel 文件并分别绘制数据点
files = ['类别 1.xlsx', '类别 2.xlsx', '类别 3.xlsx', '类别 4.xlsx']
classes = ['类别 1', '类别 2', '类别 3', '类别 4']
colors = ['blue', 'red', 'green', 'orange']

for file, color, name in zip(files, colors, classes):
    # 读取 Excel 文件
    if (name=='类别 1' and name=='类别 2' and name=='类别 3'):
        continue
    plt.figure(dpi=600)
    plt.xlabel('发病至影像检查时间')
    plt.ylabel('水肿体积')
    df = pd.read_excel(file)

    # 提取 x 和 y 列数据
    x = df.iloc[:, 1].values
    y = df.iloc[:, 2].values

    # 绘制数据点
    plt.scatter(x, y, color=color, label=name, s=5)
    plt.locator_params(axis='x', nbins=5)
    plt.legend()
    plt.title(name+'散点图')
    plt.show()

for file, color, name in zip(files, colors, classes):
    # 读取 Excel 文件
    plt.figure(dpi=600)
    plt.xlabel('发病至影像检查时间')
    plt.ylabel('水肿体积')
    df = pd.read_excel(file)
```



```

# 提取 x 和 y 列数据
x = df.iloc[:, 1].values
y = df.iloc[:, 2].values

# 绘制数据点
plt.scatter(x, y, color=color, label=name,s=5)
plt.locator_params(axis='x', nbins=5)
plt.legend()
plt.title('分类之后的散点图')
plt.show()

```

附录 4

类别 1 散点图拟合显示代码

```

import matplotlib
import pandas as pd
import matplotlib.pyplot as plt
# 设置字体为 SimHei
plt.rcParams['font.family'] = 'SimHei'

# 设置字体列表，包含 SimHei 和 Arial
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial']
# 读取数据
AllList = pd.read_excel('类别 1 整合后全体流水号数据.xlsx') # 假设数据存储在
名为 data.xlsx 的文件中
Numbers=AllList.iloc[:, 0]

# 读取数据
data = pd.read_excel('类别 1.xlsx')
X=data['时间进程']
Y=data['水肿体积']

## 绘制散点图
# plt.figure(dpi=600)
# plt.scatter(data['时间进程'], data['水肿体积'],s=5)
#
## 添加标题和轴标签
# plt.title('水肿体积随时间进展曲线的散点图')
# plt.xlabel('发病至影像检查时间')
# plt.ylabel('水肿体积')
## plt.xlim(0, 400)
## 显示图形

```

```

# plt.show()

import numpy as np
import matplotlib.pyplot as plt

# 输入数据
x = X
y = Y

combined = list(zip(x, y))
combined_sorted = sorted(combined, key=lambda tup: tup[0])

# 获取排序后的 x 和 y
x = [tup[0] for tup in combined_sorted]
y = [tup[1] for tup in combined_sorted]

# 分段点的位置
breakpoints = np.arange(0, len(x), 5)

# 分段拟合
coefficients = []
for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    segment_y = y[breakpoints[i]: breakpoints[i + 1]]

    # 低阶多项式拟合
    p = np.polyfit(segment_x, segment_y, 2)

    # 将每个分段的拟合系数存储在数组中
    coefficients.append(p)

# 预测值
predictions = np.zeros_like(y)
for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    predictions[breakpoints[i]: breakpoints[i + 1]] = np.polyval(coefficients[i],
segment_x)

# 残差
residuals = y - predictions

# 绘制原始数据点和拟合曲线
plt.figure(dpi=600)

```

```

plt.scatter(x, y, c='blue', label='真实数据点')
for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    segment_y = np.polyval(coefficients[i], segment_x)
    #plt.plot(segment_x, segment_y, label='Segment {}'.format(i + 1))
plt.plot(x, predictions, c='purple', label='预测值')
plt.xlabel('发病至影像检查时间')
plt.ylabel('水肿体积')
title = "类别 1 每{}个点分段一次 , 多项式拟合阶数 = {}".format(5, 4)
plt.title(title)

# 计算残差
residuals = y - predictions

# 计算平均残差
mean_residual = np.mean(residuals)

sorted_indices = np.argsort(X)

predicted_values_original = [None] * len(x)
for i, tup in enumerate(combined_sorted):
    original_index = combined.index(tup)
    predicted_values_original[original_index] = predictions[i]

residuals_values_original = [None] * len(x)
for i, tup in enumerate(combined_sorted):
    original_index = combined.index(tup)
    residuals_values_original[original_index] = residuals[i]

# 创建 DataFrame
data = pd.DataFrame({'患者号': Numbers, 'x': X, 'y': Y, '原顺序拟合值': predicted_values_original, '残差': residuals_values_original })

# 添加平均残差到 DataFrame
data.loc[len(data)] = ['-','-','-','- ', mean_residual]

# 保存到 Excel 文档
data.to_excel('类别 1residuals.xlsx', index=False)

Table = AllList.to_numpy()
UniqueNums = np.unique(Table[:, 0])

```

```

def remove_values(arr, value):
    mask = arr == value
    idx = np.argmax(mask) # 获取第一个匹配值的索引
    if idx == 0:
        return arr
    else:
        return arr[:idx]
UniqueNums=remove_values(UniqueNums,'sub101')

ErrorA = np.zeros(len(UniqueNums))

for i in range(len(UniqueNums)):
    a = np.nonzero(Table[:, 0] == UniqueNums[i])[0]
    error=0.0
    for j in a:

#print('predicted_values_original[j]-Y[j]=' ,predicted_values_original[j],Y[j])
        error=error+(predicted_values_original[j] - Y[j])
    #print(len(a))
    ErrorA[i] = (error)/len(a)

# 创建 DataFrame
print('平均残差为: ',ErrorA[i]/len(ErrorA))
data1 = pd.DataFrame({'残差': ErrorA})

# 保存到 Excel 文档
data1.to_excel('类别 1averageresiduals.xlsx', index=False)
ax = plt.gca()

# 在右上角添加平均残差值的文本标注
ax.annotate(f'平均残差为: : {ErrorA[i]/len(ErrorA):.2f}', xy=(1.0, 0.85),
xycoords='axes fraction', ha='right', va='top')
plt.xlim(0,1000)
plt.legend()

plt.show()

```

附录 5

类别 2 散点图拟合显示代码

```

import matplotlib
import pandas as pd

```

```

import matplotlib.pyplot as plt
# 设置字体为 SimHei
plt.rcParams['font.family'] = 'SimHei'

# 设置字体列表, 包含 SimHei 和 Arial
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial']
# 读取数据
AllList = pd.read_excel('类别 2 整合后全体流水号数据.xlsx')
Numbers=AllList.iloc[:, 0]

# 读取数据
data = pd.read_excel('类别 2.xlsx')
X=data['时间进程']
Y=data['水肿体积']

## 绘制散点图
# plt.figure(dpi=600)
# plt.scatter(data['时间进程'], data['水肿体积'],s=5)
#
## 添加标题和轴标签
# plt.title('水肿体积随时间进展曲线的散点图')
# plt.xlabel('发病至影像检查时间')
# plt.ylabel('水肿体积')
## plt.xlim(0, 400)
## 显示图形
## plt.show()

import numpy as np
import matplotlib.pyplot as plt

# 输入数据
x = X
y = Y

combined = list(zip(x, y))
combined_sorted = sorted(combined, key=lambda tup: tup[0])

# 获取排序后的 x 和 y
x = [tup[0] for tup in combined_sorted]
y = [tup[1] for tup in combined_sorted]

# 分段点的位置
breakpoints = np.arange(0, len(x),5)

```

```

# 分段拟合
coefficients = []
for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    segment_y = y[breakpoints[i]: breakpoints[i + 1]]

    # 低阶多项式拟合
    p = np.polyfit(segment_x, segment_y, 2)

    # 将每个分段的拟合系数存储在数组中
    coefficients.append(p)

# 预测值
predictions = np.zeros_like(y)
for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    predictions[breakpoints[i]: breakpoints[i + 1]] = np.polyval(coefficients[i],
segment_x)

# 残差
residuals = y - predictions

# 绘制原始数据点和拟合曲线
plt.figure(dpi=600)
plt.scatter(x, y, c='blue', label='真实数据点')
for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    segment_y = np.polyval(coefficients[i], segment_x)
    #plt.plot(segment_x, segment_y, label='Segment {}'.format(i + 1))
plt.plot(x, predictions, c='purple', label='预测值')
plt.xlabel('发病至影像检查时间')
plt.ylabel('水肿体积')
title = "类别 2 每{}个点分段一次 , 多项式拟合阶数 = {}".format(5, 4)
plt.title(title)

# 计算残差
residuals = y - predictions

# 计算平均残差
mean_residual = np.mean(residuals)

sorted_indices = np.argsort(X)

```

```

predicted_values_original = [None] * len(x)
for i, tup in enumerate(combined_sorted):
    original_index = combined.index(tup)
    predicted_values_original[original_index] = predictions[i]

residuals_values_original = [None] * len(x)
for i, tup in enumerate(combined_sorted):
    original_index = combined.index(tup)
    residuals_values_original[original_index] = residuals[i]

# 创建 DataFrame
data = pd.DataFrame({'患者号':Numbers,'x': X, 'y': Y,'原顺序拟合值':predicted_values_original,'残差': residuals_values_original })

# 添加平均残差到 DataFrame
data.loc[len(data)] = ['-','-','-','- ',mean_residual]

# 保存到 Excel 文档
data.to_excel('类别 2residuals.xlsx', index=False)

Table = AllList.to_numpy()
UniqueNums = np.unique(Table[:, 0])

def remove_values(arr, value):
    mask = arr == value
    idx = np.argmax(mask) # 获取第一个匹配值的索引
    if idx == 0:
        return arr
    else:
        return arr[:idx]
UniqueNums=remove_values(UniqueNums,'sub101')

ErrorA = np.zeros(len(UniqueNums))

for i in range(len(UniqueNums)):
    a = np.nonzero(Table[:, 0] == UniqueNums[i])[0]
    error=0.0
    for j in a:
        #print('predicted_values_original[j]-Y[j]=' ,predicted_values_original[j],Y[j])
        error=error+(predicted_values_original[j] - Y[j])

```



```

    #print(len(a))
    ErrorA[i] = (error)/len(a)

# 创建 DataFrame
print('平均残差为: ',ErrorA[i]/len(ErrorA))
data1 = pd.DataFrame({'残差': ErrorA})

# 保存到 Excel 文档
data1.to_excel('类别 1averageresiduals.xlsx', index=False)
ax = plt.gca()

# 在右上角添加平均残差值的文本标注
ax.annotate(f'平均残差为: : {ErrorA[i]/len(ErrorA):.2f}', xy=(1.0, 0.85),
xycoords='axes fraction', ha='right', va='top')
plt.xlim(0,1000)
plt.legend()

plt.show()

```

附录 6

类别 3 散点图拟合显示代码

```

import matplotlib
import pandas as pd
import matplotlib.pyplot as plt
# 设置字体为 SimHei
plt.rcParams['font.family'] = 'SimHei'

# 设置字体列表，包含 SimHei 和 Arial
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial']
# 读取数据
AllList = pd.read_excel('类别 3 整合后全体流水号数据.xlsx')
Numbers=AllList.iloc[:, 0]

# 读取数据
data = pd.read_excel('类别 2.xlsx')
X=data['时间进程']
Y=data['水肿体积']

```

```

## 绘制散点图
# plt.figure(dpi=600)
# plt.scatter(data['时间进程'], data['水肿体积'],s=5)
#
## 添加标题和轴标签
# plt.title('水肿体积随时间进展曲线的散点图')
# plt.xlabel('发病至影像检查时间')
# plt.ylabel('水肿体积')
## plt.xlim(0, 400)
## 显示图形
##plt.show()

import numpy as np
import matplotlib.pyplot as plt

# 输入数据
x = X
y = Y

combined = list(zip(x, y))
combined_sorted = sorted(combined, key=lambda tup: tup[0])

# 获取排序后的 x 和 y
x = [tup[0] for tup in combined_sorted]
y = [tup[1] for tup in combined_sorted]

# 分段点的位置
breakpoints = np.arange(0, len(x),5)

# 分段拟合
coefficients = []
for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    segment_y = y[breakpoints[i]: breakpoints[i + 1]]

    # 低阶多项式拟合
    p = np.polyfit(segment_x, segment_y, 2)

    # 将每个分段的拟合系数存储在数组中
    coefficients.append(p)

# 预测值
predictions = np.zeros_like(y)

```

```

for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    predictions[breakpoints[i]: breakpoints[i + 1]] = np.polyval(coefficients[i],
segment_x)

# 残差
residuals = y - predictions

# 绘制原始数据点和拟合曲线
plt.figure(dpi=600)
plt.scatter(x, y, c='blue', label='真实数据点')
for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    segment_y = np.polyval(coefficients[i], segment_x)
    #plt.plot(segment_x, segment_y, label='Segment {}'.format(i + 1))
plt.plot(x, predictions, c='purple', label='预测值')
plt.xlabel('发病至影像检查时间')
plt.ylabel('水肿体积')
title = "类别 3 每{}个点分段一次 , 多项式拟合阶数 = {}".format(5, 4)
plt.title(title)

# 计算残差
residuals = y - predictions

# 计算平均残差
mean_residual = np.mean(residuals)

sorted_indices = np.argsort(X)

predicted_values_original = [None] * len(x)
for i, tup in enumerate(combined_sorted):
    original_index = combined.index(tup)
    predicted_values_original[original_index] = predictions[i]

residuals_values_original = [None] * len(x)
for i, tup in enumerate(combined_sorted):
    original_index = combined.index(tup)
    residuals_values_original[original_index] = residuals[i]

# 创建 DataFrame
data = pd.DataFrame({'患者号': Numbers, 'x': X, 'y': Y, '原顺序拟合值': predicted_values_original, '残差': residuals_values_original })

```

```

# 添加平均残差到 DataFrame
data.loc[len(data)] = ['-','-','-','-',mean_residual]

# 保存到 Excel 文档
data.to_excel('类别 3residuals.xlsx', index=False)

Table = AllList.to_numpy()
UniqueNums = np.unique(Table[:, 0])

def remove_values(arr, value):
    mask = arr == value
    idx = np.argmax(mask) # 获取第一个匹配值的索引
    if idx == 0:
        return arr
    else:
        return arr[idx:]
UniqueNums=remove_values(UniqueNums,'sub101')

ErrorA = np.zeros(len(UniqueNums))

for i in range(len(UniqueNums)):
    a = np.nonzero(Table[:, 0] == UniqueNums[i])[0]
    error=0.0
    for j in a:

#print('predicted_values_original[j]-Y[j]=' ,predicted_values_original[j],Y[j])
        error=error+(predicted_values_original[j] - Y[j])
    #print(len(a))
    ErrorA[i] = (error)/len(a)

# 创建 DataFrame
print('平均残差为: ',ErrorA[i]/len(ErrorA))
data1 = pd.DataFrame({'残差': ErrorA})

# 保存到 Excel 文档
data1.to_excel('类别 3averageresiduals.xlsx', index=False)
ax = plt.gca()

# 在右上角添加平均残差值的文本标注
ax.annotate(f'平均残差为: : {ErrorA[i]/len(ErrorA):.2f}', xy=(1.0, 0.85),
xycoords='axes fraction', ha='right', va='top')
plt.xlim(0,1000)

```

```
plt.legend()
```

```
plt.show()
```

附录 7

类别 4 散点图拟合显示代码

```
import matplotlib
import pandas as pd
import matplotlib.pyplot as plt
# 设置字体为 SimHei
plt.rcParams['font.family'] = 'SimHei'

# 设置字体列表, 包含 SimHei 和 Arial
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial']
# 读取数据
AllList = pd.read_excel('类别 4 整合后全体流水号数据.xlsx')
Numbers=AllList.iloc[:, 0]

# 读取数据
data = pd.read_excel('类别 2.xlsx')
X=data['时间进程']
Y=data['水肿体积']

## 绘制散点图
# plt.figure(dpi=600)
# plt.scatter(data['时间进程'], data['水肿体积'],s=5)
#
## 添加标题和轴标签
# plt.title('水肿体积随时间进展曲线的散点图')
# plt.xlabel('发病至影像检查时间')
# plt.ylabel('水肿体积')
## plt.xlim(0, 400)
## 显示图形
##plt.show()

import numpy as np
import matplotlib.pyplot as plt

# 输入数据
x = X
y = Y
```

```

combined = list(zip(x, y))
combined_sorted = sorted(combined, key=lambda tup: tup[0])

# 获取排序后的 x 和 y
x = [tup[0] for tup in combined_sorted]
y = [tup[1] for tup in combined_sorted]

# 分段点的位置
breakpoints = np.arange(0, len(x), 5)

# 分段拟合
coefficients = []
for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    segment_y = y[breakpoints[i]: breakpoints[i + 1]]

    # 低阶多项式拟合
    p = np.polyfit(segment_x, segment_y, 2)

    # 将每个分段的拟合系数存储在数组中
    coefficients.append(p)

# 预测值
predictions = np.zeros_like(y)
for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    predictions[breakpoints[i]: breakpoints[i + 1]] = np.polyval(coefficients[i],
segment_x)

# 残差
residuals = y - predictions

# 绘制原始数据点和拟合曲线
plt.figure(dpi=600)
plt.scatter(x, y, c='blue', label='真实数据点')
for i in range(len(breakpoints) - 1):
    segment_x = x[breakpoints[i]: breakpoints[i + 1]]
    segment_y = np.polyval(coefficients[i], segment_x)
    #plt.plot(segment_x, segment_y, label='Segment {}'.format(i + 1))
plt.plot(x, predictions, c='purple', label='预测值')
plt.xlabel('发病至影像检查时间')
plt.ylabel('水肿体积')
title = "类别 3 每{}个点分段一次 , 多项式拟合阶数 = {}".format(5, 4)

```

```

plt.title(title)

# 计算残差
residuals = y - predictions

# 计算平均残差
mean_residual = np.mean(residuals)

sorted_indices = np.argsort(X)

predicted_values_original = [None] * len(x)
for i, tup in enumerate(combined_sorted):
    original_index = combined.index(tup)
    predicted_values_original[original_index] = predictions[i]

residuals_values_original = [None] * len(x)
for i, tup in enumerate(combined_sorted):
    original_index = combined.index(tup)
    residuals_values_original[original_index] = residuals[i]

# 创建 DataFrame
data = pd.DataFrame({'患者号':Numbers,'x': X, 'y': Y,'原顺序拟合值':predicted_values_original,'残差': residuals_values_original })

# 添加平均残差到 DataFrame
data.loc[len(data)] = ['-','-','-','- ',mean_residual]

# 保存到 Excel 文档
data.to_excel('类别 4residuals.xlsx', index=False)

Table = AllList.to_numpy()
UniqueNums = np.unique(Table[:, 0])

def remove_values(arr, value):
    mask = arr == value
    idx = np.argmax(mask) # 获取第一个匹配值的索引
    if idx == 0:
        return arr
    else:
        return arr[:idx]
UniqueNums=remove_values(UniqueNums,'sub101')

```

```

ErrorA = np.zeros(len(UniqueNums))

for i in range(len(UniqueNums)):
    a = np.nonzero(Table[:, 0] == UniqueNums[i])[0]
    error=0.0
    for j in a:

#print('predicted_values_original[j]-Y[j]=' ,predicted_values_original[j],Y[j])
        error=error+(predicted_values_original[j] - Y[j])
    #print(len(a))
    ErrorA[i] = (error)/len(a)

# 创建 DataFrame
print('平均残差为: ',ErrorA[i]/len(ErrorA))
data1 = pd.DataFrame({'残差': ErrorA})

# 保存到 Excel 文档
data1.to_excel('类别 4averageresiduals.xlsx', index=False)
ax = plt.gca()

# 在右上角添加平均残差值的文本标注
ax.annotate(f'平均残差为: : {ErrorA[i]/len(ErrorA):.2f}', xy=(1.0, 0.85),
xycoords='axes fraction', ha='right', va='top')
plt.xlim(0,1000)
plt.legend()

plt.show()

```

附录 8

第三问方差筛选加互信息分类预测

```

import matplotlib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from lightgbm import LGBMClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.metrics import accuracy_score

```



```

from sklearn.model_selection import train_test_split
from sklearn.feature_selection import SelectFromModel
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import VarianceThreshold
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from pyswarm import pso
import matplotlib.pyplot as plt
import numpy as np

matplotlib.rcParams['font.family'] = 'SimHei'
# 读取 Excel 文件
data = pd.read_excel('data1.xlsx')
yuce = pd.read_excel('yuce1.xlsx')
# 读取 Excel 文件
yuce_100 = pd.read_excel('yuce1_100.xlsx')
# 提取特征和目标变量
X_yuce_100 = data.iloc[:, 4:77] # 根据实际情况选择特征列
# 提取特征和目标变量
X = data.iloc[:, 4:77] # 根据实际情况选择特征列
y = data.iloc[:, 1] # 根据实际情况选择目标变量列

X_yuce = yuce.iloc[:, 4:77] # 根据实际情况选择特征列
y_yuce = yuce.iloc[:, 1] # 根据实际情况选择目标变量列
print(X.shape)
print(X_yuce.shape)
def Selector(X,X_yuce,X_yuce_100):
    # 第一次特征筛选 - 方差筛选
    var_selector = VarianceThreshold(threshold=0.1)
    X_var_filtered = var_selector.fit_transform(X)
    # 第二次特征筛选 - 互信息
    k = 10 # 选择前 k 个最相关的特征
    mi_selector = SelectKBest(score_func=mutual_info_classif, k=k)
    X_mi_filtered = mi_selector.fit_transform(X_var_filtered,y)
    ## 获取被选择的特征索引
    selected_feature_indices = mi_selector.get_support(indices=True)
    #
    ## 获取被选择的特征列表
    selected_features = X.columns[selected_feature_indices]
    print(selected_features)
    # X_yuce = X_yuce[selected_features]
    X_yuce_filtered = var_selector.transform(X_yuce)
    X_yuce_filtered_kbest = mi_selector.transform(X_yuce_filtered)

```

```

X_yuce_100_filtered = var_selector.transform(X_yuce_100)
X_yuce_100_filtered_kbest = mi_selector.transform( X_yuce_100_filtered )
return X_mi_filtered,X_yuce_filtered_kbest,X_yuce_100_filtered_kbest
X,X_yuce,X_yuce_100=Selector(X,X_yuce,X_yuce_100)

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X ,y, test_size=0.2,
random_state=42)

# 定义模型
Gridmodels = {
    'Grid-Random Forest': RandomForestClassifier(),
    'Grid-SVM': SVC(),
    'Grid-LightGBM': LGBMClassifier(),
    'Grid-XGBoost': XGBClassifier()
}

Randommodels = {
    'Random-Random Forest': RandomForestClassifier(),
    'Random-SVM': SVC(),
    'Random-LightGBM': LGBMClassifier(),
    'Random-XGBoost': XGBClassifier()
}

# 定义超参数空间
Gridparam_grid = {
    'Grid-Random Forest': {'n_estimators': np.arange(1, 100,1), 'max_depth':
np.arange(3, 4,1)},
    'Grid-SVM': {'C': np.arange(1, 100,1), 'kernel': ['poly', 'rbf']},
    'Grid-LightGBM': {"verbosity": [-1], 'num_leaves': np.arange(5, 20,1),
'learning_rate': [0.1, 0.01,0.001]},
    'Grid-XGBoost': {'max_depth': np.arange(3,10,1), 'learning_rate': [0.1,
0.01,0.001]}
}

Randomparam_grid = {
    'Random-Random Forest': {'n_estimators': np.arange(1, 100,1), 'max_depth':
np.arange(3, 7,1)},
    'Random-SVM': {'C': np.arange(0.1, 10,0.1), 'kernel': ['poly', 'rbf']},
    'Random-LightGBM': {"verbosity": [-1], 'num_leaves': np.arange(5, 20,1),
'learning_rate': [0.1, 0.01,0.001]},
    'Random-XGBoost': {'max_depth': np.arange(3,7,1), 'learning_rate': [0.1,

```

```

0.01,0.001]}
}
Y=[]

AllModels=[]
# 网格搜索调优
grid_results = {}
for model_name, model in Gridmodels.items():
    grid_search = GridSearchCV(model, Gridparam_grid[model_name],cv=3)
    grid_search.fit(X_train, y_train)
    grid_results[model_name] = grid_search.best_params_
    y_yuce = grid_search.best_estimator_.predict(X)
    AllModels.append(grid_search.best_estimator_)
    Y.append(y_yuce)

# 随机搜索调优
random_results = {}
for model_name, model in Randommodels.items():
    random_search = RandomizedSearchCV(model,
    Randomparam_grid[model_name], n_iter=10,cv=3)
    random_search.fit(X_train, y_train)
    random_results[model_name] = random_search.best_params_
    y_yuce = random_search.best_estimator_.predict(X)
    AllModels.append(random_search.best_estimator_)
    Y.append(y_yuce)

# 模型训练和评估
model_scores = {}
for model_name, model in Gridmodels.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    model_scores[model_name] = accuracy_score(y_test, y_pred)

for model_name, model in Randommodels.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    model_scores[model_name] = accuracy_score(y_test, y_pred)

import pandas as pd

df = pd.DataFrame()
classes=['网格搜索调优的随机森林算法','网格搜索调优的支持向量机算法','网格
搜索调优的 LightGBM 算法','网格搜索调优的 XGBoost 算法','随机搜索调优的随
机森林算法','随机搜索调优的支持向量机算法','随机搜索调优的 LightGBM 算法

```

```

', '随机搜索调优的 XGBoost 算法']

# 将每个模型的结果添加到 DataFrame 的每一列
for i, model_result in enumerate(Y):
    column_name = classes[i];
    df[column_name] = model_result

# 将 DataFrame 保存到 Excel 文件
df.to_excel("model_results1.xlsx", index=False)

# 打印结果
print("网格搜索结果:")
for model_name, params in grid_results.items():
    print(model_name, "-", params)

print("\n 随机搜索结果:")
for model_name, params in random_results.items():
    print(model_name, "-", params)

#print(len(model_scores))

# print("\n 模型拟合度排序:")
# sorted_scores = sorted(model_scores.items(), key=lambda x: x[1], reverse=True)
# for model_name, score in sorted_scores:
#     print(model_name, "-", score)

from sklearn.metrics import precision_score, recall_score, f1_score
y = data.iloc[:, 1].values

accuracy = []
precision = []
recall = []
f1 = []

for j, item in enumerate(AllModels):

    y_yuce_100 = item.predict(X_yuce_100)
    # 计算准确率、精确率、召回率和 F1 分数
    accuracy1 = accuracy_score(y, y_yuce_100)
    precision1 = precision_score(y, y_yuce_100, average='macro')
    recall1 = recall_score(y, y_yuce_100, average='macro')
    f11 = f1_score(y, y_yuce_100, average='macro')

```

```

accuracy.append(accuracy1)
precision.append(precision1)
recall.append(recall1)
f1.append(f11)
# 创建标签和数值的列表
labels = ['准确率', '精确率', '召回率', 'F1 分数']
values = [accuracy1, precision1, recall1, f11]

# 创建一个柱状图
plt.bar(labels, values)
#print(classes[j])
plt.title(classes[j] + '模型指标')
# 在柱状图上显示数值
for i, v in enumerate(values):
    plt.text(i, v, f'{v:.2f}', ha='center', va='bottom')

# 设置图表标题和轴标签

plt.xlabel('指标')
plt.ylabel('数值')

# 显示图表
plt.show()

# 构建雷达图数据
data = np.array([accuracy, precision, recall, f1])
categories = ['Accuracy', 'Precision', 'Recall', 'F1 Score']

# 颜色列表
colors = ['red', 'green', 'blue', 'orange']

# 绘制四个雷达图
for i in range(len(categories)):
    fig = plt.figure(figsize=(8, 8))
    ax = fig.add_subplot(111, polar=True)
    ax.set_theta_offset(np.pi / 2)
    ax.set_theta_direction(-1)
    plt.xticks(np.linspace(0, 2 * np.pi, len(classes) + 1)[-1], classes)
    ax.set_rlabel_position(0)

    # 绘制当前指标的雷达图，并使用对应的颜色
    values = data[i, :].tolist()
    values += values[:1]

```

```

    ax.plot(np.linspace(0, 2 * np.pi, len(classes) + 1), values, linewidth=1,
linestyle='solid', color=colors[i], label=categories[i])
    ax.fill(np.linspace(0, 2 * np.pi, len(classes) + 1), values, alpha=0.25,
color=colors[i])

    # 添加图例和标题
    plt.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1))
    plt.title(categories[i])

    # 显示图形
    plt.show()

# 绘制一个包含所有模型的雷达图
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, polar=True)
ax.set_theta_offset(np.pi / 2)
ax.set_theta_direction(-1)
plt.xticks(np.linspace(0, 2 * np.pi, len(classes) + 1)[:1], classes)
ax.set_rlabel_position(0)

# 绘制每个模型的雷达图，并使用对应的颜色
for i in range(len(categories)):
    values = data[i, :].tolist()
    values += values[:1]
    ax.plot(np.linspace(0, 2 * np.pi, len(classes) + 1), values, linewidth=1,
linestyle='solid', color=colors[i], label=categories[i])
    ax.fill(np.linspace(0, 2 * np.pi, len(classes) + 1), values, alpha=0.25,
color=colors[i])

# 添加图例和标题
plt.legend(loc='upper right', bbox_to_anchor=(1.1, 1))
plt.title('模型指标雷达图')

# 添加颜色和指标的图例
legend_labels = [plt.Line2D([0], [0], color=color, linewidth=3) for color in colors]
plt.legend(legend_labels, categories, loc='lower right', bbox_to_anchor=(1.1, 1))

# 显示图形
plt.show()

```

附录 9

第三问距离和 RFE 筛选分类预测

```

import matplotlib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from lightgbm import LGBMClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.feature_selection import SelectFromModel
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import VarianceThreshold
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from pyswarm import pso

from sklearn.feature_selection import RFE

from sklearn.linear_model import LinearRegression
import numpy as np
from dcor import distance_correlation
matplotlib.rcParams['font.family'] = 'SimHei'
# 读取 Excel 文件
data = pd.read_excel('data1.xlsx')
yuce = pd.read_excel('yuce1.xlsx')
X_yuce_100 = data.iloc[:, 4:77] # 根据实际情况选择特征列
# 提取特征和目标变量
X = data.iloc[:, 4:77] # 根据实际情况选择特征列
y = data.iloc[:, 1] # 根据实际情况选择目标变量列

X_yuce = yuce.iloc[:, 4:77] # 根据实际情况选择特征列
y_yuce = yuce.iloc[:, 1] # 根据实际情况选择目标变量列

print(X.shape)
print(X_yuce.shape)
def Selector(X,X_yuce,X_yuce_100):
    # 计算每个特征与目标变量之间的距离相关系数
    dist_corr = X.apply(lambda x: distance_correlation(x, y), axis=0)

    # 选择距离相关系数最高的十个特征
    top_features = dist_corr.sort_values(ascending=False).head(10).index
    print(top_features)
    X_selected = X[top_features]

```

```

## 应用递归特征消除法来进一步筛选特征
# estimator = LinearRegression() # 根据问题选择适当的估计器
# selector = RFE(estimator, n_features_to_select=10) # 选择所需数量的特征
# X_final = selector.fit_transform(X_selected, y)

X_yuce = X_yuce[top_features]
X_yuce_100=X_yuce_100[top_features]
return X_selected,X_yuce,X_yuce_100

X,X_yuce,X_yuce_100=Selector(X,X_yuce,X_yuce_100)

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 定义模型
Gridmodels = {
    'Grid-Random Forest': RandomForestClassifier(),
    'Grid-SVM': SVC(),
    'Grid-LightGBM': LGBMClassifier(),
    'Grid-XGBoost': XGBClassifier()
}

Randommodels = {
    'Random-Random Forest': RandomForestClassifier(),
    'Random-SVM': SVC(),
    'Random-LightGBM': LGBMClassifier(),
    'Random-XGBoost': XGBClassifier()
}

# 定义超参数空间
Gridparam_grid = {
    'Grid-Random Forest': {'n_estimators': np.arange(1, 100,1), 'max_depth':
np.arange(3, 10,1)},
    'Grid-SVM': {'C': np.arange(1, 100,1), 'kernel': ['poly', 'rbf']},
    'Grid-LightGBM': {'"verbosity"': [-1], 'num_leaves': np.arange(5, 20,1),
'learning_rate': [0.1, 0.01,0.001]},
    'Grid-XGBoost': {'max_depth': np.arange(3,10,1), 'learning_rate': [0.1,
0.01,0.001]}
}

Randomparam_grid = {

```



```

    'Random-Random Forest': {'n_estimators': np.arange(1, 100,1), 'max_depth':
np.arange(3, 7,1)},
    'Random-SVM': {'C': np.arange(0.1, 10,0.1), 'kernel': ['poly', 'rbf']},
    'Random-LightGBM': {"verbosity": [-1], 'num_leaves': np.arange(5, 20,1),
'learning_rate': [0.1, 0.01,0.001]},
    'Random-XGBoost': {'max_depth': np.arange(3,7,1), 'learning_rate': [0.1,
0.01,0.001]}
}
Y=[]

AllModels=[]
# 网格搜索调优
grid_results = {}
for model_name, model in Gridmodels.items():
    grid_search = GridSearchCV(model, Gridparam_grid[model_name])
    grid_search.fit(X_train, y_train)
    grid_results[model_name] = grid_search.best_params_
    y_yuce = grid_search.best_estimator_.predict(X_yuce_100)
    AllModels.append(grid_search.best_estimator_)
    Y.append(y_yuce)

# 随机搜索调优
random_results = {}
for model_name, model in Randommodels.items():
    random_search = RandomizedSearchCV(model,
Randomparam_grid[model_name], n_iter=10)
    random_search.fit(X_train, y_train)
    random_results[model_name] = random_search.best_params_
    y_yuce = random_search.best_estimator_.predict(X_yuce_100)
    AllModels.append(random_search.best_estimator_)
    Y.append(y_yuce)
# 模型训练和评估
model_scores = {}
for model_name, model in Gridmodels.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    model_scores[model_name] = accuracy_score(y_test, y_pred)

for model_name, model in Randommodels.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    model_scores[model_name] = accuracy_score(y_test, y_pred)

import pandas as pd

```

```

# 假设你的预测的一组 y 值存储在一个名为 predicted_y 的列表中

df = pd.DataFrame()

# 将每个模型的结果添加到 DataFrame 的每一列
for i, model_result in enumerate(Y):
    column_name = f'Model {i+1}'
    df[column_name] = model_result

# 将 DataFrame 保存到 Excel 文件
df.to_excel("model_results2.xlsx", index=False)
classes=['网格搜索调优的随机森林算法','网格搜索调优的支持向量机算法','网格
搜索调优的 LightGBM 算法','网格搜索调优的 XGBoost 算法','随机搜索调优的随
机森林算法','随机搜索调优的支持向量机算法','随机搜索调优的 LightGBM 算法
','随机搜索调优的 XGBoost 算法']

# 打印结果
print("网格搜索结果:")
for model_name, params in grid_results.items():
    print(model_name, "-", params)

print("\n 随机搜索结果:")
for model_name, params in random_results.items():
    print(model_name, "-", params)

print(len(model_scores))

print("\n 模型拟合度排序:")
sorted_scores = sorted(model_scores.items(), key=lambda x: x[1], reverse=True)
for model_name, score in sorted_scores:
    print(model_name, "-", score)

from sklearn.metrics import precision_score, recall_score, f1_score
y = data.iloc[:, 1].values

accuracy = []
precision = []
recall = []
f1 = []

```

```

for j,item in enumerate(AllModels):

    y_yuce_100=item.predict(X_yuce_100)
    # 计算准确率、精确率、召回率和 F1 分数
    accuracy1 = accuracy_score(y, y_yuce_100)
    precision1 = precision_score(y, y_yuce_100, average='macro')
    recall1 = recall_score(y, y_yuce_100, average='macro')
    f11 = f1_score(y, y_yuce_100, average='macro')

    accuracy.append(accuracy1)
    precision.append(precision1)
    recall.append(recall1)
    f1.append(f11)
    # 创建标签和数值的列表
    labels = ['准确率', '精确率', '召回率', 'F1 分数']
    values = [accuracy1, precision1, recall1, f11]

    # 创建一个柱状图
    plt.bar(labels, values)
    #print(classes[j])
    plt.title(classes[j] + '模型指标')
    # 在柱状图上显示数值
    for i, v in enumerate(values):
        plt.text(i, v, f'{v:.2f}', ha='center', va='bottom')

    # 设置图表标题和轴标签

    plt.xlabel('指标')
    plt.ylabel('数值')

    # 显示图表
    plt.show()

# 构建雷达图数据
data = np.array([accuracy, precision, recall, f1])
categories = ['Accuracy', 'Precision', 'Recall', 'F1 Score']

# 颜色列表
colors = ['red', 'green', 'blue', 'orange']

# 绘制四个雷达图
for i in range(len(categories)):
    fig = plt.figure(figsize=(8, 8))
    ax = fig.add_subplot(111, polar=True)

```

```

ax.set_theta_offset(np.pi / 2)
ax.set_theta_direction(-1)
plt.xticks(np.linspace(0, 2 * np.pi, len(classes) + 1)[:1], classes)
ax.set_rlabel_position(0)

# 绘制当前指标的雷达图，并使用对应的颜色
values = data[i, :].tolist()
values += values[:1]
ax.plot(np.linspace(0, 2 * np.pi, len(classes) + 1), values, linewidth=1,
linestyle='solid', color=colors[i], label=categories[i])
ax.fill(np.linspace(0, 2 * np.pi, len(classes) + 1), values, alpha=0.25,
color=colors[i])

# 添加图例和标题
plt.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1))
plt.title(categories[i])

# 显示图形
plt.show()
# 绘制一个包含所有模型的雷达图
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, polar=True)
ax.set_theta_offset(np.pi / 2)
ax.set_theta_direction(-1)
plt.xticks(np.linspace(0, 2 * np.pi, len(classes) + 1)[:1], classes)
ax.set_rlabel_position(0)

# 绘制每个模型的雷达图，并使用对应的颜色
for i in range(len(categories)):
    values = data[i, :].tolist()
    values += values[:1]
    ax.plot(np.linspace(0, 2 * np.pi, len(classes) + 1), values, linewidth=1,
linestyle='solid', color=colors[i], label=categories[i])
    ax.fill(np.linspace(0, 2 * np.pi, len(classes) + 1), values, alpha=0.25,
color=colors[i])
# 添加图例和标题
plt.legend(loc='upper right', bbox_to_anchor=(1.1, 1))
plt.title('模型指标雷达图')
# 添加颜色和指标的图例
legend_labels = [plt.Line2D([0], [0], color=color, linewidth=3) for color in colors]
plt.legend(legend_labels, categories, loc='lower right', bbox_to_anchor=(1.1, 1))
# 显示图形
plt.show()

```